

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное
образовательное учреждение высшего образования
«Череповецкий государственный университет»

Институт (факультет)	<u>Институт информационных технологий</u>
Направление подготовки	
(специальность)	<u>09.03.04 Программная инженерия</u>
Выпускающая кафедра	<u>Математического и программного обеспечения ЭВМ</u>

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Название работы	<u>Разработка программного обеспечения распознавания номера</u>
стальковшей	
Студента	<u>КраеваАлексея Александровича</u>
	Ф.И.О.

ДОПУСТИТЬ К ЗАЩИТЕ

Директор института (декан	
факультета)	<u>Ершов Евгений Валентинович</u>
Заведующий выпускающей	
кафедрой	<u>Ершов Евгений Валентинович</u>
Руководитель выпускной	
квалификационной работы	<u>Ершов Евгений Валентинович</u>
Нормоконтролер	<u>Виноградова Людмила Николаевна</u>
Выпускник	<u>КраевАлексей Александрович</u>

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное
образовательное учреждение высшего образования
«Череповецкий государственный университет»

АННОТАЦИЯ

выпускной квалификационной работы студента

по теме:

Разработка программного обеспечения распознавания номера стальной

Студент

Краев Алексей Александрович

фамилия, имя, отчество
подпись

Руководитель работы

ФГБОУВО

«Череповецкий государственный университет»

место работы

д.т.н., профессор

должность

Ершов Евгений Валентинович

фамилия, имя, отчество

(Перечислить основные вопросы, которые рассматривались; результаты работы)

В ходе работы над выпускной квалификационной работой были изучены технологии видеоаналитики, применительно к задаче оптимизации работы и сокращения затрат предприятия ПАО «Северсталь». Были рассмотрены основные способы решения данной задачи и выбран наиболее оптимальный. Результатом выполнения задачи выпускной квалификационной работы является создание программного обеспечения, предназначенного для распознавания стальной и их номеров.

Данная разработка позволит повысить эффективность производства за счет отслеживания

перемещения стальной по территории предприятия.

Оглавление

Введение.....	7
1. Основная часть.....	8
1.1 Сравнительный анализ отечественных и зарубежных аналогов проектируемой системы.....	8
1.2 Выбор технологии, среды и языка программирования.....	10
1.3 Анализ процесса обработки информации, выбор структур данных для её хранения, выбор методов и алгоритмов решения задачи.....	14
1.4 Разработка спецификаций проектируемой системы.....	18
1.4.1 Построение диаграммы вариантов использования.....	19
1.4.2 Построение контекстной диаграммы классов.....	26
1.4.3 Построение диаграмм последовательностей системы.....	28
1.4.3.1 Диаграмма последовательностей системы для варианта использования «Детектирование стальковша».....	28
1.4.3.2 Диаграмма последовательностей системы для варианта использования «Выбор источника видеопотока».....	29
1.4.3.3 Диаграмма последовательностей системы для варианта использования «Запуск/остановка системы распознавания».....	31
1.4.3.4 Диаграмма последовательностей системы для варианта использования «Логирование».....	32
1.4.3.5 Диаграмма последовательностей системы для варианта использования «Настроить».....	33
1.4.3.6 Диаграмма последовательностей системы для варианта использования «Определение номера стальковша».....	35
1.4.3.7 Диаграмма последовательностей системы для варианта использования «Проверка статуса работы камеры».....	36
1.4.3.8 Диаграмма последовательностей системы для варианта использования «Управление настройками видеопотока».....	37

1.4.4 Построение диаграмм деятельностей вариантов использования.....	40
1.4.5 Проектирование структур данных и диаграмм отношений.....	42
1.4.6 Построение диаграммы переходов состояний.....	44
1.5 Проектирование системы.....	45
1.5.1 Проектирование структуры системы и построение диаграмм пакетов	45
1.5.2 Проектирование классов в пакетах.....	47
1.5.2.1 Проектирование классов пакета «BusinessLayer».....	47
1.5.2.1.1 Построение исходной диаграммы классов пакета «BusinessLayer»	47
1.5.2.1.2 Построение диаграммы последовательности взаимодействия объектов классов пакета «BusinessLayer».....	48
1.5.2.1.3 Построение диаграммы кооперации пакета «BusinessLayer».....	51
1.5.2.1.4 Построение уточнений диаграммы классов пакета «BusinessLayer»	51
1.5.2.1.5 Построение детальной диаграммы классов пакета «BusinessLayer»	52
1.5.2.2 Проектирование классов пакета «PresentationLayer».....	55
1.5.2.2.1 Построение исходной диаграммы классов пакета «PresentationLayer».....	55
1.5.2.2.2 Построение диаграммы последовательности взаимодействия объектов классов пакета «PresentationLayer».....	56
1.5.2.2.3 Построение диаграммы кооперации пакета «PresentationLayer»....	60
1.5.2.2.4 Построение уточнений диаграммы классов пакета «PresentationLayer».....	60
1.5.2.2.5 Построение детальной диаграммы классов пакета «PresentationLayer».....	61
1.5.2.3 Проектирование классов пакета «DataLayer».....	66
1.5.2.3.1 Построение исходной диаграммы классов пакета «DataLayer».....	66
1.5.2.3.2 Построение диаграммы последовательности взаимодействия объектов классов пакета «DataLayer».....	67

1.5.2.3.3 Построение диаграммы кооперации пакета «DataLayer».....	69
5.2.3.4 Построение уточнений диаграммы классов пакета «DataLayer».....	69
1.5.2.3.5 Построение детальной диаграммы классов пакета «DataLayer».....	70
1.5.2.4 Проектирование классов пакета «Functional».....	72
1.5.2.4.1 Построение исходной диаграммы классов пакета «Functional».....	72
1.5.2.3.2 Построение диаграммы последовательности взаимодействия объектов классов пакета «Functional».....	73
1.5.2.3.3 Построение диаграммы кооперации пакета «Functional».....	74
1.5.2.3.4 Построение уточнений диаграммы классов пакета «Functional»....	74
1.5.2.3.5 Построение детальной диаграммы классов пакета «Functional».....	75
1.5.3 Построение диаграмм компонентов.....	78
1.5.4 Построение диаграмм размещения.....	89
1.6. Проектирование интерфейса пользователя.....	90
1.6.1. Построение графа диалога.....	90
1.6.2 Разработка форм ввода-вывода информации.....	92
1.7 Выбор стратегии тестирования, разработка тестов, программа и методика испытаний.....	96
1.7.1 Объект и цель испытаний.....	96
1.7.2 Требования к информационному, аппаратно-программному обеспечению.	96
1.7.2.1. Требования к функциональным характеристикам.....	96
1.7.2.2. Требования к надежности продукта.....	97
1.7.2.3. Требования к составу и параметрам технических средств.....	97
1.7.2.4. Требования к программной документации.....	98
1.7.3. Состав, порядок и методы испытаний.....	98
1.7.4. Результаты проведения испытаний.....	100
2 Технико-экономическое обоснование выполняемой разработки.....	106
2.1 Организация работ.....	106
2.2 Расчет себестоимости программного продукта.....	116
Заключение.....	124

Список литературы.....	126
Приложение 1 Техническое задание.....	129
Приложение 2 Текст программы.....	138
Приложение 3 Спецификации.....	151
Приложение 4 Руководство пользователя.....	154

Введение

Задача оптимизации неразрывно связана с развитием промышленности, так в 21 веке с развитием цифровых технологий появилось программное обеспечение способное обрабатывать огромные объемы данных, что позволило анализировать производственные процессы машинными средствами.

В данный момент на предприятии ПАО «Северсталь» существует проблема, связанная с контролем перемещения стальной ковше, а именно требуется программное обеспечение для фиксации стальной ковша и его номера на разных участках производства с указанием времени. Стальные ковши перемещаются по территории предприятия, что требует их отслеживания и идентификации по номеру на разных технологических этапах выпечки, для эффективного планирования производственного процесса. Задача осложняется неблагоприятными условиями производства, которые делают номер, нанесенный краской, непригодным для чтения за непродолжительный промежуток времени, для устранения данной проблемы текст наваривается металлом на корпус стальной ковша и для идентификации ковша автоматизированными средствами требуется изображение с тепловизионной камеры.

Основная задача, решаемая в рамках выпускной квалификационной работы, разработка ПО распознавания номера стальной ковше.

Разрабатываемое программное обеспечение позволит решить данную проблему, в частности, система будет распознавать номера стальной ковше и записывать их в базу данных, лог-файл, а также выводить информацию на экран компьютера оператора.

Целью выпускной квалификационной работы является повышение эффективности производства за счет отслеживания перемещения стальной ковше по территории предприятия.

1. Основная часть

1.1 Сравнительный анализ отечественных и зарубежных аналогов проектируемой системы

Системы машинного распознавания очень популярны в настоящее время. Такую популярность можно оправдать практически безграничной сферой их применения. Например, системы распознавания номерных знаков автомобилей, распознавания текста на изображении, идентификация лиц, контроля за техникой безопасности и т.д.

В качестве аналогов также можно рассматривать системы категории распознавания, такие как распознавания номеров автомобиля и распознавания лиц. Ниже приведены некоторые из них:

- AutoTRASSI — предназначен для автоматического распознавания автомобильных номеров, попавших в поле зрения видеокамеры. Система может использоваться для контроля въезда/выезда автотранспорта с территории предприятий, парковок. Возможно использование службами автоинспекции, на пропускных пунктах и т. д. Средствами TRASSIR обеспечивается взаимодействие с другими подсистемами (например, контроля доступа, видео- и аудиоконтроля) и оборудованием[22].

- TRASSIR Face Detector Ivideon - интеллектуальный модуль обнаружения и трекинга лиц. Состоит из двух компонентов. Сопровождение обнаруженного лица человека с последующей группировкой зафиксированных лиц и ранжирование их по людям. Формирование базы детектированных лиц (например, выборка с заданных камер и за определенный период времени) и осуществление выгрузки из архива за выбранный временной интервал[23].

- НомерОк - программно аппаратный комплекс, который используется для идентификации номерных знаков транспортных средств и управлении исполнительными устройствами. У этого ПО более высокий процент распознавания (97%). Он успешно взаимодействует с номерами из РФ, Украины

и даже Израиля. Особенности взаимодействия. Он может устанавливаться для аналоговых видеокамер и внешних устройств. Эти устройства могут выдавать и принимать сигналы от других устройств. Взаимодействие с контроллерами осуществляется посредством программного обеспечения. Модуль может взаимодействовать с базой данных. Взаимодействие сервера и базы данных осуществляется на постоянной основе. Программа даёт возможность запрограммировать активацию тревожного события для конкретных групп номерных знаков[24].

- «Система обнаружения и сопровождения движущихся объектов по признаку их движения». Данная система изобретена АО «Институт информационных технологий». Система предназначена для автоматического выделения и сопровождения малоразмерных объектов по признаку их движения на изображениях, получаемых с подвижной видеокамеры. Система реализована на базе персонального компьютера. Осуществляет ввод с видеокамеры, визуализацию и обработку цифровых изображений движущихся сцен в реальном времени [17].

- Система промышленной безопасности, разработанная «Малленом Системс». Данная разработка позволяет мгновенно выявлять такие нарушения техники безопасности, как отсутствие на сотрудниках средств индивидуальной защиты (СИЗ) или нахождение людей в опасных зонах. Для получения видеопотока система может использовать обзорные видеокамеры, установленные на предприятии. Программа на основе сверточных нейронных сетей анализирует изображения с обзорных видеокамер, детектирует на них людей и контролирует наличие на нем каски и светоотражающего жилета. В случае фиксации факта нарушения, программа формирует тревожное сообщение [25].

- Умные камеры

Компания FLiR Systems выпустила несколько камер с датчиками тепла. Устройства FLiR Duo и Duo R внешне напоминают экшн-камеру. Их можно прикрепить к любому дрону и отслеживать тепло в различных деловых и

бытовых ситуациях — например, можно обнаружить утечку в изоляции крыши или вести воздушную топографическую съемку полей и нефтяных месторождений [25].

Компания ITRI разработала систему Intelligent Vision System, которая использует глубинное обучение и компьютерное зрение, чтобы роботы могли различать объекты разного размера (фигурки, чашки) и определять их положение. Распознав объект, робот сможет взять его и принести в нужное место [25]. Элегантное устройство FridgeCam от Smarter крепится к стенке холодильника и может определять, когда истекает срок годности, сообщать, что именно находится в холодильнике, и даже рекомендовать рецепты блюд из выбранных продуктов [25].

Главным недостатком всех приведенных выше систем является то, что их невозможно применить к объектам заданной предметной области. Как следствие необходимо разработать ПО, выполняющее все требования заказчика и соответствующее предметной области.

1.2 Выбор технологии, среды и языка программирования

Исходя из требований заказчика к реализации программного обеспечения была выбрана модель прототипирования жизненного цикла (рис. 1).

Данная модель была выбрана по следующим причинам:

- взаимодействие с заказчиком на ранних этапах разработки;
- быстрое устранение неточностей в требованиях;
- возможность учесть новые пожелания заказчика;
- прототип позволяет гибко выполнять проектирование и разработку;
- заказчик всегда видит прогресс в процессе разработки программного продукта;
- возможность возникновения противоречий между разработчиками и заказчиками сведена к минимуму;

- заказчики принимают участие в процессе разработки на протяжении всего ЖЦ;
- получение прототипа ПО в кратчайшие сроки.

В результате прототипирования, разработчик демонстрирует пользователям готовый прототип, а пользователи оценивают его функционирование. После этого определяются проблемы, над устранением которых совместно работают пользователи и разработчики.



Рис.1. Модель прототипирования ЖЦ ПО

Для создания архитектуры сложного приложения необходимо построить графическое описание предметной области, которое позволит произвести моделирование системного проектирования и отображения организационных структур. Для графического описания был выбран язык UML [9]. UML обладает следующим преимуществам:

- UML объектно-ориентирован, в результате чего методы описания результатов анализа и проектирования семантически близки к методам программирования на современных объектно-ориентированных языках;
- UML позволяет описать систему практически со всех возможных точек зрения и в различных аспектах поведения системы;
- диаграммы UML сравнительно просты для чтения после достаточно быстрого ознакомления с его синтаксисом;

- UML получил широкое распространение и быстро развивается.

Объектно-ориентированное программирование (ООП) — методология программирования, основанная на представлении программы в виде совокупности объектов, каждый из которых является экземпляром определённого класса, а классы образуют иерархию наследования [1].

Для проектирования данной информационной системы был выбран следующий стек технологий:

- для обнаружения стальной шпалы и распознавания его номера на видеопотоке используется связка библиотеки компьютерного зрения OpenCV и нейронной сети YoloV3;
- для реализации системы использовался язык программирования высокого уровня C#, платформа .Net;
- пользовательский интерфейс реализован на системе WPF;
- при проектировании ПО использовался объектный подход и язык UML;
- среда разработки Microsoft Visual Studio 2019;
- в качестве места для хранения статистики будет использоваться БД SQLite.

Выбор данных технологий обусловлен требованиями к ПО приведенными в прил.1.

OpenCV (Open Source Computer Vision Library, библиотека компьютерного зрения с открытым исходным кодом) — библиотека алгоритмов компьютерного зрения, обработки изображений и численных алгоритмов общего назначения с открытым кодом. Реализована на C/C++, также разрабатывается для Python, C# и других языков [19].

За определение стальной шпалы и его номера отвечает заранее обученная сверточная нейронная сеть YOLOv3. YOLO - алгоритм обнаружения объектов для использования в реальных продуктах. [19].

Обучение модели нейронной сети происходит «с учителем», нейросеть получает на вход некоторое размеченное изображение с отмеченным стальной шпалой и его номером. Размеченные объекты представлены классами.

Обучение проводилось с использованием датасета SVHN (размеченный датасет изображений с номерами домов) и датасета со стальковшами предоставленного «Северсталью».

Для реализации программного обеспечения использовался язык C#, работающий на платформе .Net. C# — объектно-ориентированный язык программирования[2].

Пользовательский интерфейс реализован на платформе WPF. Windows Presentation Foundation (WPF) — аналог WinForms, система для построения клиентских приложений Windows с визуально привлекательными возможностями взаимодействия с пользователем, графическая (презентационная) подсистема в составе .NET Framework (начиная с версии 3.0), использующая язык XAML[12].

Разработка производилась в среде Microsoft Visual Studio 2019. Microsoft Visual Studio — линейка продуктов компании Microsoft, включающих интегрированную среду разработки программного обеспечения и ряд других инструментальных средств[12].

SQLite — компактная встраиваемая СУБД [12]. Данная СУБД отличается простотой и удобством в использовании, имеет библиотеку для языка C# и подробную документацию на русском языке, что значительно упрощает работу с ней.

Так же для проектирования ПО были использованы следующие CASE-средства:

- Microsoft Visio — векторный графический редактор, редактор диаграмм и блок-схем для Windows;
- ERwin Data Modeler — это компьютерная программа для проектирования и документирования баз данных.

Разработка программной документации проводилась в соответствии с Единой системой программной документации (ЕСПД), позволяющая облегчить процесс сопровождения программного решения как самим создателем ПО, так

и сторонним лицам, сопровождающим разработанный программный продукт.

1.3 Анализ процесса обработки информации, выбор структур данных для её хранения, выбор методов и алгоритмов решения задачи

В последнее время широкое распространение для решения данной проблемы получили системы компьютерного зрения. Компьютерное зрение (Computer Vision, CV), в том числе машинное зрение (Machine Vision, MV) – это автоматическая фиксация и обработка изображений, как неподвижных, так и движущихся объектов при помощи компьютерных средств. В России также используется термин «техническое зрение», обозначающий в совокупности компьютерное и машинное зрение. В последние годы CV стало активно использоваться в промышленности, в частности металлургии [19].

Одним из основных видов сырья для получения стали является чугун. Суть процесса заключается в уменьшении содержания углерода в металле и удалении примесей – серы и фосфора, а также в доведении содержания кремния, марганца и других элементов до необходимых значений.

Процесс состоит из следующих этапов: выплавка стали в конвертере (или электропечи), внепечная обработка, разливка стали. В нашей предметной области будет рассмотрен этап внепечной обработки.

Задача внепечной обработки - получить заданный химический состав (в т.ч. произвести удаление примесей S, N при необходимости) и обеспечить требуемую температуру для разливки. Для реализации данной задачи используются стальковши.

Сталеразливочный ковш (стальковш) – предназначен для транспортировки жидкой стали от плавильного агрегата к месту расположения изложниц, внепечной обработки стали и последующей разливки. В настоящее время большинство цехов для производства крупных кузнечных слитков оснащено установками ковш-печь и вакуумирования в ковше.

Стальковши перемещаются по территории предприятия, что требует их отслеживания и идентификации по номеру на разных технологических этапах

внепечной обработки, для эффективного планирования производственного процесса. Задача осложняется неблагоприятными условиями производства, которые делают номер, нанесенный краской непригодным для чтения за непродолжительный промежуток времени, для устранения данной проблемы текст наваривается металлом на корпус стальной ковша и для идентификации ковша автоматизированными средствами требуется изображение с тепловизионной камеры.

Стальной ковш обладает следующими характеристиками (рис.2):

- Диаметр максимальный – 6 метров;
- Высота стальной ковша – 10 метров.



Рис. 2. Стальной ковш

Для решения поставленной задачи подразумевается следующий алгоритм работы:

1. Получение видеопотока содной из камер тепловизора, установленной в цехе или загрузка видеопотока из внешнего файла;
2. Вывод видеопотока на экран программы оператору;
3. Обработка видеопотока библиотекой OpenCV: получение каждого кадра видеопотока для дальнейшей обработки;
4. Поиск на кадре стальковша и его номера с использованием нейронной сети YoloV3.
5. В случае нахождения в кадре перечисленных объектов, происходит выделение их контуров.
6. Если объект найден, то производится запись вбазу данных (БД) и вывод сведений на экран программы оператора.

Схема процесса обработки информации представлена на рис.3.

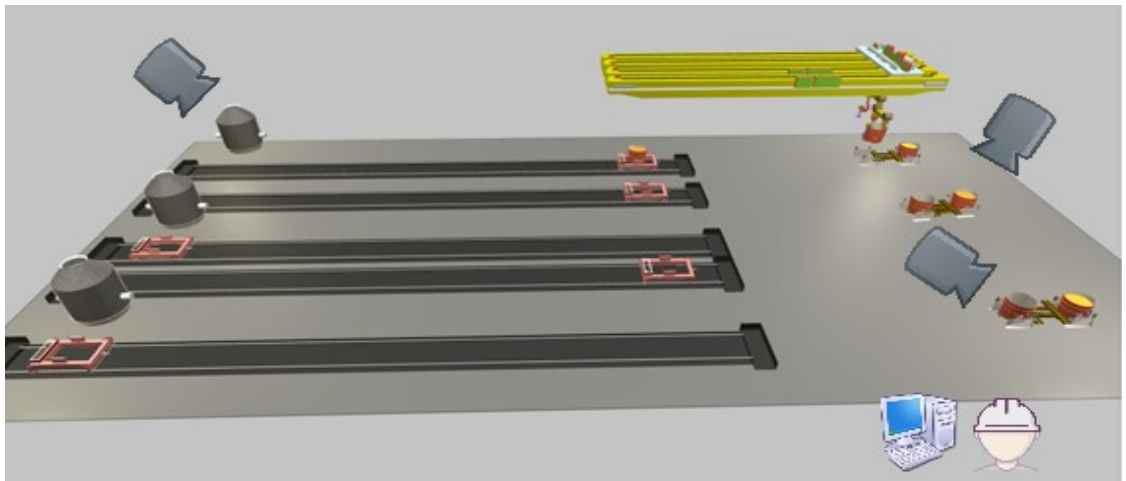


Рис. 3. Схема процесса обработки информации

Схема размещения программного обеспечения на производстве показывает то, как будет происходить процесс обработки информации. Камера, установленная напротив стальковшей, захватывает и формирует видеопоток для системы распознавания номера, установленная на персональном компьютере оператора.

Предполагаемый алгоритм решения задачи обеспечит производство своевременным выявлением факта нарушения, что сыграет положительную

роль для уменьшения травматизма на производстве и увеличения ответственности персонала.

Объект стальной представляет из себя емкость имеющую высокую температуру, как следствие на видеопотоке имеет желто-алую окраску. Номер стальной наплавлен металлом на корпус объекта и как следствие имеет более низкую температуру, отображается более темным цветом, что позволяет его детектировать.

Работу по определению стальной и его номера на видео берет на себя заранее обученная нейронная сеть YOLOv3. Это сверточная нейронная сеть. YOLO - алгоритм обнаружения объектов для использования в реальных продуктах. Архитектура нейронной сети Yolo. Детектирование объектов осуществляется с некоторой точностью.

Система детектирования работает следующим образом. Изображение разбивается на несколько областей. Каждая область предсказывает степень уверенности в том, что этой области содержится объект. Каждая из областей предсказывает 5 переменных: x , y , w , h , $confidence$. (x, y) – координаты центра объекта относительно ячейки. W , h – ширина и высота объекта соответственно. $Confidence$ показывает вероятность присутствия объекта. Каждая ячейка так же предсказывает C – вероятность принадлежности объекта к определенному классу. В результате, получается матрица, размерности $S \times S \times (B*5 + C)$ элементов [19].

В YOLO удалены два полносвязных слоя на выходе, вместо них на используются сверточные слои. Вместо координат, сеть предсказывает смещения относительно якорных значений (anchor boxes). Для YOLO эти anchor boxes получаются с использованием метода K-means с использованием обучающей выборки. Вместе с архитектурой YOLO состоящей из 22 сверточных слоев была представлена и уменьшенная модель сети, в 9 слоев и с меньшим количеством фильтров свертки. Уменьшенная модель необходима для увеличения скорости работы, хотя приходится пожертвовать точностью работы

сети. Авторы отмечают, что такая топология сети занимает меньший объем памяти, что критически важно на встраиваемых системах.

После разработки алгоритма работы системы следует перейти к его программной реализации. Необходимо определить структуры данных для решения поставленной задачи.

В результате детектирования получаем экземпляр класса «Стальковш», который содержит в себе данные о номере, времени детектирования и участке производства, камере. После данные заносятся в базу данных или лог файл.

Объект стальковш представляет из себя емкость имеющую высокую температуру, как следствие на видеопотоке имеет желто-алую окраску. Номер стальковша наплавлен металлом на корпус объекта и как следствии имеет более низкую температуру, отображается более темным цветом, что позволяет его детектировать.

Задача детектирования осложняется спецификой производства, а именно условия запыленности помещения, высокие температуры, работа с ИК-видеопотоком.

1.4 Разработка спецификаций проектируемой системы

В основе объектного подхода к разработке программного обеспечения лежит объектная декомпозиция, т. е. представление разрабатываемого программного обеспечения в виде совокупности объектов, в процессе взаимодействия, которых через передачу сообщений и происходит выполнение требуемых функций.

Спецификация разрабатываемого программного обеспечения при использовании UML объединяет несколько моделей: использования, логическую, реализации, процессов, развертывания.

1.4.1 Построение диаграммы вариантов использования

Одним из стандартных элементов языка UML является диаграмма вариантов использования, которая позволяет наглядно представить ожидаемое поведение системы. Основными понятиями диаграмм вариантов использования являются: действующее лицо, вариант использования, связь. Диаграмма вариантов может дополняться пояснительным текстом, который раскрывает смысл или семантику составляющих ее компонентов. Такой пояснительный текст получил название примечания или сценария [7].

Отдельный вариант использования обозначается на диаграмме эллипсом, внутри которого содержится его краткое название или имя в форме глагола с пояснительными словами [7].

Действующее лицо (actor) – внешняя по отношению к разрабатываемому программному обеспечению сущность, которая взаимодействует с ним с целью получения или предоставления какой-либо информации. Действующими лицами могут быть пользователи, другое программное обеспечение или какие-либо технические средства, взаимодействующие с разрабатываемым программным обеспечением [7].

Вариант использования - некоторая очевидная для действующего лица процедура, решающая конкретную задачу. Все варианты использования, так или иначе, связаны с требованиями к функциональности разрабатываемой системы и могут сильно отличаться по объему выполняемой работы.

Связь - взаимодействие действующих лиц и соответствующих вариантов использования [7].

Варианты использования также могут быть связаны между собой. При этом фиксируют связи использования и расширения.

Диаграмма вариантов использования для системы определения номера стальковша представлена на рис.4.

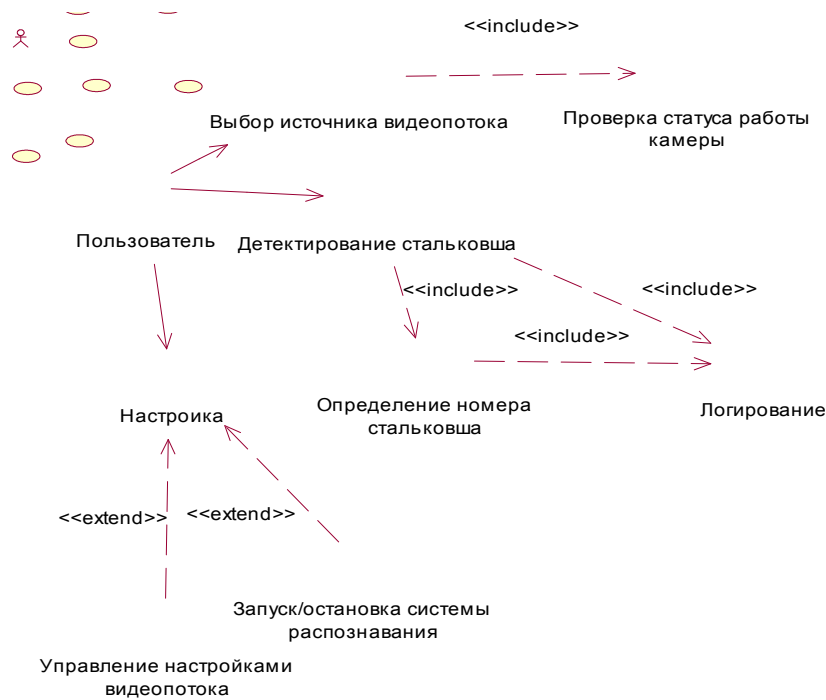


Рис. 4. Диаграмма вариантов использования

У данной диаграммы одно действующее лицо – «Пользователь». Оно совершает все действия с программой: выбор источника видеопотока, детектирование столкновения (включает логирование и определение номера столкновения), настройка (расширенная управлением настройками видеопотока и запуском, остановкой системы).

Описание блока «Проверка статуса работы камеры»

Краткое описание варианта использования «Проверка статуса работы камеры» представлено в табл. 1.

Таблица 1

Краткое описание варианта «Проверка статуса работы камеры»

Название	Проверка статуса работы камеры
Назначение	Определяет наличие видеопотока, поступающего от камеры
Действующие лица	Пользователь
Краткое описание	Обеспечивает механизм проверки поступающего сигнала от камеры

Типичный ход событий для данного варианта представлен в табл. 2.

Таблица 2

Типичный ход событий варианта «Проверка статуса работы камеры»

Действие пользователя	Отклик системы
1) Выбор источника видеопотока	2) Проверка наличие сигнала от выбранной камеры, вывод результата проверки
3) Пользователь подтверждает выбор источника	4) Переход к окну просмотра видеопотока

Альтернатива:

- Не удаётся получить сигнал с камеры. В этом случае выводится сообщение о неработоспособности камеры.

Описание блока «Логирование»

Краткое описание варианта использования «Логирование» представлено в табл. 3.

Таблица 3

Краткое описание варианта «Логирование»

Название	Логирование
Назначение	Сохранение результатов работы программы в отдельном файле
Действующие лица	Пользователь
Краткое описание	Обеспечивает механизм сохранения результатов работы системы распознавания, с указанием времени, источника видеосигнала и номера обнаруженного объекта

Типичный ход событий для данного варианта представлен в табл. 4.

Таблица 4

Типичный ход событий варианта «Логирование»

Действие пользователя	Отклик системы
1) Запуск системы распознавания	1) Создается новый файл, в который заносятся результаты работы
1) Остановка системы распознавания	1) Сохранение и закрытие созданного файла

Альтернатива:

- Работа программы завершается в результате сбоя или некорректных действий пользователя. В этом случае файл лога сохраняется вплоть до предыдущего до сбоя события.

Описание блока «Запуск/остановка системы распознавания»

Краткое описание варианта использования «Запуск/остановка системы распознавания» представлено в табл. 5.

Таблица 5

Краткое описание варианта «Запуск/остановка системы распознавания»

Название	Запуск/остановка системы распознавания
Назначение	Запуск и остановка распознавания номеров стальной
Действующие лица	Пользователь
Краткое описание	Обеспечивает механизм старта и завершения системы распознавания номеров стальной

Типичный ход событий для данного варианта представлен в табл. 6.

Таблица 6

Типичный ход событий варианта

Действие пользователя	Отклик системы
1) Нажатие кнопки «Запуск»	1) Система начинает распознавание объектов на выбранном видеопотоке
2) Нажатие кнопки «Стоп»	3) Система останавливает распознавание объектов на выбранном видеопотоке

Описание блока «Управление настройками видеопотока»

Краткое описание варианта использования «Запуск/остановка системы распознавания» представлено в табл. 7.

Таблица 7

Краткое описание варианта «Управление настройками видеопотока»

Название	Управление настройками видеопотока
Назначение	Управление параметрами видеопотока

Действующие лица	Пользователь
Краткое описание	Обеспечивает задание настроек видеопотоку

Типичный ход событий для данного варианта представлен в табл. 8.

Таблица 8

Типичный ход событий варианта «Управление настройками видеопотока»

Действие пользователя	Отклик системы
1) Изменение поля «Частота кадров»	1) Система распознавание работает с видеопотоком с заданной частотой кадров
1) Изменение поля «Разрешение»	1) Система распознавание работает с видеопотоком с заданным разрешением
1) Нажатие кнопки «Сохранение»	1) Параметры сохраняются
1) Нажатие кнопки «Отмена»	1) Изменения не сохраняются

Описание блока «Выбор источника видеопотока»

Краткое описание варианта использования «Выбор источника видеопотока» представлено в табл. 9.

Таблица 9

Краткое описание варианта «Выбор источника видеопотока»

Название	Выбор источника видеопотока
Назначение	Подключение IP – камеры к ЭВМ по её адресу
Действующие лица	Пользователь
Краткое описание	Пользователю предоставляется список подключенных камер к компьютеру

Типичный ход событий для данного варианта представлен в табл. 10.

Таблица 10

Типичный ход событий варианта «Выбор источника видеопотока»

Действие пользователя	Отклик системы
1) Нажатие кнопки «Выбор камеры»	1) Система предоставляет поле ввода адреса камеры
1) Нажатие кнопки «Применить»	1) Система проверяет камеру на наличие видеосигнала

Описание блока «Детектирование стальной»

Краткое описание варианта использования «Детектирование стальной»

представлено в табл. 11.

Таблица 11

Краткое описание варианта «Детектирование стальковша»

Название	Детектирование стальковша
Назначение	Поиск объекта стальковш на видео
Действующие лица	Пользователь
Краткое описание	Обеспечивает механизм поиска стальковша на видео и выделение его контура

Типичный ход событий для данного варианта представлен в табл. 12.

Таблица 12

Типичный ход событий варианта «Детектирование стальковша»

Действие пользователя	Отклик системы
1) Нажатие кнопки «Запуск»	1) Система начинает поиск объекта стальковш на видеопотоке и выделяет его контур, в случае обнаружение 2) Заносит запись в лог файл
1) Нажатие кнопки «Стоп»	1) Система останавливает поиск объекта стальковш на видеопотоке

Описание блока «Определение номера стальковша»

Краткое описание варианта использования «Определение номера стальковша» представлено в табл. 13.

Таблица 13

Краткое описание варианта «Определение номера стальковша»

Название	Определение номера стальковша
Назначение	Определение номера стальковша на видеопотоке
Действующие лица	Пользователь
Краткое описание	Обеспечивает механизм поиска номера стальковша на видео и выделение его контура

Типичный ход событий для данного варианта представлен в табл. 14.

Таблица 14

Типичный ход событий варианта «Определение номера стальной»

Действие пользователя	Отклик системы
1) Нажатие кнопки «Запуск»	1) Если стальной обнаружен, система начинает поиск номера объекта на видеопотоке и выделяет его контур, в случае обнаружения 2) Заносит запись в лог файл
1) Нажатие кнопки «Стоп»	1) Система останавливает поиск объекта номера стальной на видеопотоке

Альтернатива:

- Если номер отсутствует на корпусе либо не был распознан, то в поле номер заносится запись «Отсутствует».

Описание блока «Настроить»

Краткое описание варианта использования «Настроить» представлено в табл. 15.

Таблица 15

Краткое описание варианта «Настроить»

Название	Настроить
Назначение	Осуществление взаимодействия с настройками программы
Действующие лица	Пользователь
Краткое описание	Обеспечивает механизм настройки работы программы

Типичный ход событий для данного варианта представлен в табл. 16.

Таблица 16

Типичный ход событий варианта «Настроить»

Действие пользователя	Отклик системы
1) Нажатие кнопки «Настроить»	1) Система предоставляет список возможных настроек
1) Выбор из списка	1) Переход к окну настройки выбранного параметра

1.4.2 Построение контекстной диаграммы классов

Диаграммы классов - центральное звено объектно-ориентированных методов разработки программного обеспечения, поэтому все существующие методы используют диаграммы классов в одной из известных нотаций. Однако в основном диаграммы классов в этих методах применяют на этапе проектирования, для того чтобы показать особенности построения конкретных классов[7].

Концептуальные модели в соответствии с определением оперируют понятиями предметной области, атрибутами этих понятий и отношениями между ними. Понятию в предметной области разрабатываемого программного обеспечения могут соответствовать как материальные предметы, так и абстракции, которые применяют специалисты предметной области [7].

Основным понятиям в модели ставятся в соответствие классы. Класс при этом традиционно понимают, как совокупность общих признаков заданной группы объектов предметной области. В соответствии с этим определением на диаграмме классов каждому классу соответствует группа объектов, общие признаки которых и фиксирует класс[7].

Для заполнения параметров используются специальные формы пользовательского интерфейса.

Контекстная диаграмма классов представлена на рис. 5. В данной диаграмме существуют следующие обобщения:

- пользователь (оператор), взаимодействует с системой;
- камера тепловизора, предоставляет видеопоток для системы;
- система, производящая взаимодействие с БД, обрабатывающая входной видеопоток и осуществляющая распознавания стальковша и его номера (взаимодействуя с YOLO);
- YOLO, осуществляет взаимодействие с системой и определяет номер стальковша.

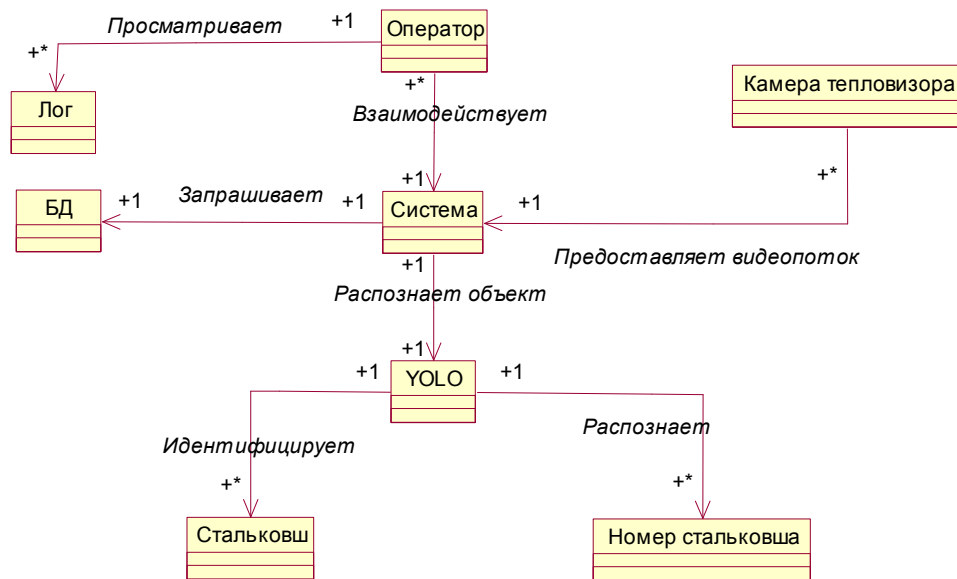


Рис. 5. Контекстная диаграмма классов

Разработанная контекстная диаграмма классов подтверждает высокую степень обособленности сущностей, их единообязанность применительно к своим задачам, а также высокий уровень взаимодействия между объектами классов предметной области. Исходя из этого, выбор объектно-ориентированного подхода при разработке программного обеспечения полностью оправдан.

1.4.3 Построение диаграмм последовательностей системы

Диаграмма последовательности (англ. sequence diagram) — диаграмма UML, на которой для некоторого набора объектов на единой временной оси показан жизненный цикл объекта (создание-деятельность-уничтожение некой сущности) и взаимодействие действующих лиц ПО в рамках прецедента[7].

Основными элементами диаграммы последовательности являются обозначения объектов (прямоугольники с названиями объектов), вертикальные «линии жизни», отображающие течение времени, прямоугольники,

отражающие деятельность объекта или исполнение им определенной функции (прямоугольники на пунктирной «линии жизни»), и стрелки, показывающие обмен сигналами или сообщениями между объектами[7].

Оператор системы выполняет настройку системы перед запуском: выставляет необходимые параметры детектирования и указывает источник видеопотока, запускает систему, либо сразу запустить систему с предустановленными параметрами.

1.4.3.1 Диаграмма последовательностей системы для варианта использования «Детектирование стальнойковша»

Диаграмма последовательностей варианта использования «Детектирование стальнойковша» представлена на рис.6.

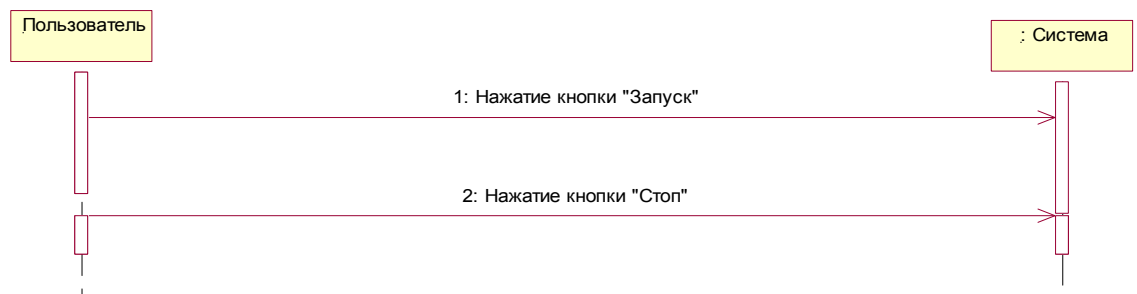


Рис. 6. Диаграмма последовательностей варианта использования «Детектирование стальнойковша»

Описание операции «Нажатие кнопки «Запуск»» представлено в табл. 17.

Таблица 17

Описание операции «Нажатие кнопки «Запуск»»

Раздел	Описание
--------	----------

Имя	Нажатие кнопки «Запуск»
Обязанности	Обеспечивает механизм запуска системы поиска стальнойковша на видео и выделение его контура
Тип	Системная
Ссылки	Вариант использования «Детектирование стальнойковша»
Примечания	Дать пользователю представление об успешности запуска системы детектирования
Исключения	Видеопоток отсутствует
Вывод	На экранепоявляется выделенный контур объекта
Предусловия	Выбран источник видеопотока
Постусловие	Выделенный объект /сообщение об ошибке

Описание операции «Нажатие кнопки «Стоп»» представлено в табл. 18

Таблица 18

Описание операции «Нажатие кнопки «Стоп»»

Раздел	Описание
Имя	Нажатие кнопки «Стоп»
Обязанности	Обеспечивает механизм запуска поиска стальнойковша на видео и выделение его контура
Тип	Системная
Ссылки	Вариант использования «Детектирование стальнойковша»
Примечания	Дать пользователю представление об успешном завершении работы системы детектирования
Исключения	-
Вывод	На экране проигрывается видеопоток без обработки
Предусловия	Система запущена
Постусловие	-

1.4.3.2 Диаграмма последовательностей системы для варианта использования «Выбор источника видеопотока»

Диаграмма последовательностей варианта использования «Выбор источника видеопотока» представлена на рис.7.

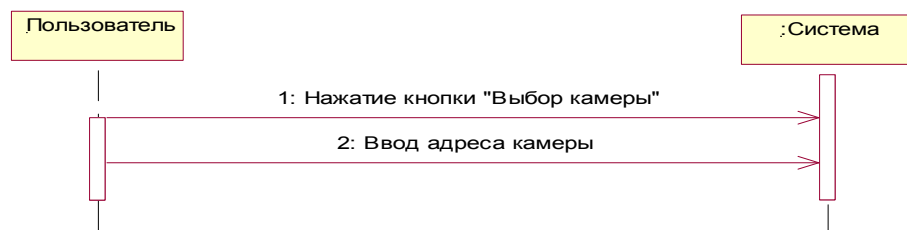


Рис. 7. Диаграмма последовательностей варианта использования «Выбор источника видеопотока»

Описание операции «Выбор источника видеопотока» представлено в табл.19.

Таблица 19

Описание операции «Выбор камеры»

Раздел	Описание
Имя	Выбор камеры
Обязанности	Предоставляет пользователю список подключенных камер к ЭВМ
Тип	Системная
Ссылки	Вариант использования «Выбор источника видеопотока»
Примечания	Предоставить пользователю обработку необходимого ему видеопотока
Исключения	Ошибка при загрузке списка камер
Вывод	На экране появляется список камер

Предусловия	Запущена система
Постусловие	Список видеокамер /сообщение об ошибке

Описание операции «Ввод адреса камеры» представлено в табл. 20.

Таблица 20

Описание операции «Ввод адреса камеры»

Раздел	Описание
Имя	Ввод адреса камеры
Обязанности	Возможность ввода адреса IP- камеры пользователем для последующего с ней соединения
Тип	Системная
Ссылки	Вариант использования «Выбор источника видеопотока»
Примечания	Предоставить пользователю обработку необходимого ему видеопотока
Исключения	Ошибка при загрузке видеопотока
Вывод	На экране появляется видеопоток
Предусловия	Запущена система
Постусловие	Видеопоток /сообщение об ошибке

1.4.3.3 Диаграмма последовательностей системы для варианта использования «Запуск/остановка системы распознавания»

Диаграмма последовательностей варианта использования «Запуск/остановка системы распознавания» представлена на рис.8.

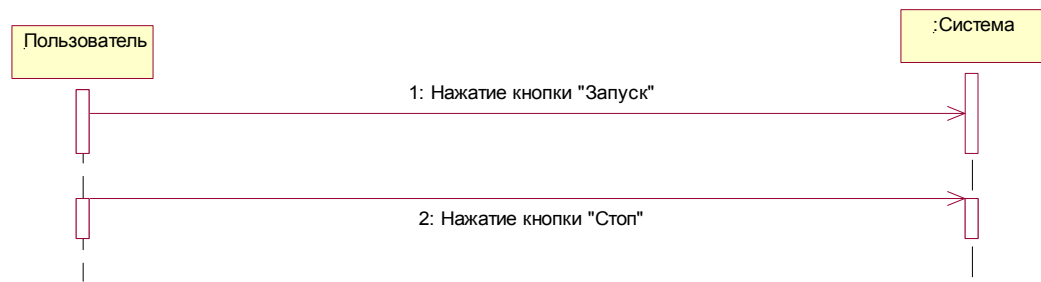


Рис. 8. Диаграмма последовательностей варианта использования
«Запуск/остановка системы распознавания»

Описание операции «Нажатие кнопки «Запуск»» представлено в табл. 21.

Таблица 21

Описание операции «Нажатие кнопки «Запуск»»

Раздел	Описание
Имя	Нажатие кнопки «Запуск»
Обязанности	Обеспечивает механизм запуска системы поиска стальковша на видео и выделение его контура
Тип	Системная
Ссылки	Вариант использования «Запуск/остановка системы распознавания»
Примечания	Предоставляет пользователю механизм старта системы распознавания номеров стальковшей
Исключения	Ошибка при запуске системы
Вывод	На экране появляется обрабатываемый видеопоток
Предусловия	-
Постусловие	Обрабатываемый видеопоток /сообщение об ошибке

Описание операции «Нажатие кнопки «Стоп»» представлено в табл. 22.

Таблица 22

Описание операции «Нажатие кнопки «Стоп»»

Раздел	Описание
Имя	Нажатие кнопки «Стоп»
Обязанности	Обеспечивает механизм запуска поиска стальнойковша на видео и выделение его контура
Тип	Системная
Ссылки	Вариант использования «Запуск/остановка системы распознавания»
Примечания	Предоставляет пользователю механизм остановки системы распознавания номеров стальнойковшей
Исключения	-
Вывод	На экране появляется необрабатываемый видеопоток
Предусловия	-
Постусловие	Обрабатываемый видеопоток /сообщение об ошибке

1.4.3.4 Диаграмма последовательностей системы для варианта использования «Логирование»

Диаграмма последовательностей варианта использования «Логирование» представлена на рис.9.

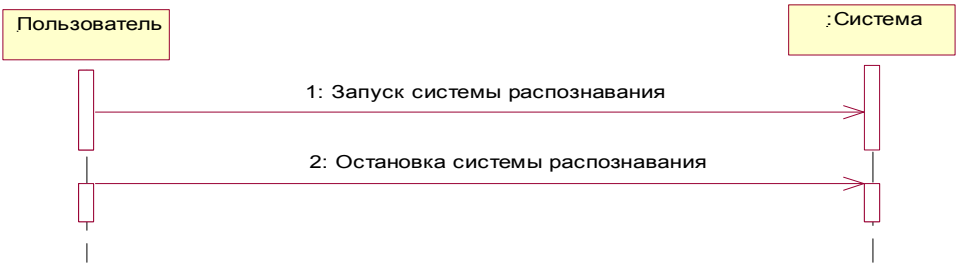


Рис.9. Диаграмма последовательностей варианта использования «Логирование»

Описание операции «Запуск системы распознавания» представлено в табл. 23.

Таблица 23

Описание операции «Запуск системы распознавания»

Раздел	Описание
Имя	Запуск системы распознавания
Обязанности	Обработка видеопотока и логирование распознанных объектов
Тип	Системная
Ссылки	Вариант использования «Логирование»
Примечания	Предоставляет пользователю текстовую и визуальную информацию о ранее распознанных объектах
Исключения	Ошибка при сохранении и формировании лога, отсутствие источника видеопотока
Вывод	Текстовый файл лога и таблица на экране пользователя

Предусловия	Запущена система
Постусловие	Лог файл /сообщение об ошибке

Описание операции «Остановка системы распознавания» представлено в табл. 24.

Таблица 24

Описание операции «Остановка системы распознавания»

Раздел	Описание
Имя	Остановка системы распознавания
Обязанности	Отключение логирования распознанных объектов
Тип	Системная
Ссылки	Вариант использования «Логирование»
Примечания	Предоставляет пользователю текстовую и визуальную информацию о ранее распознанных объектах
Исключения	Ошибка при сохранении и формировании лога
Вывод	Текстовый файл лога и таблица на экране пользователя
Предусловия	Запущена система
Постусловие	Лог файл /сообщение об ошибке

1.4.3.5 Диаграмма последовательностей системы для варианта использования «Настроить»

Диаграмма последовательностей варианта использования «Настроить» представлена на рис.10.

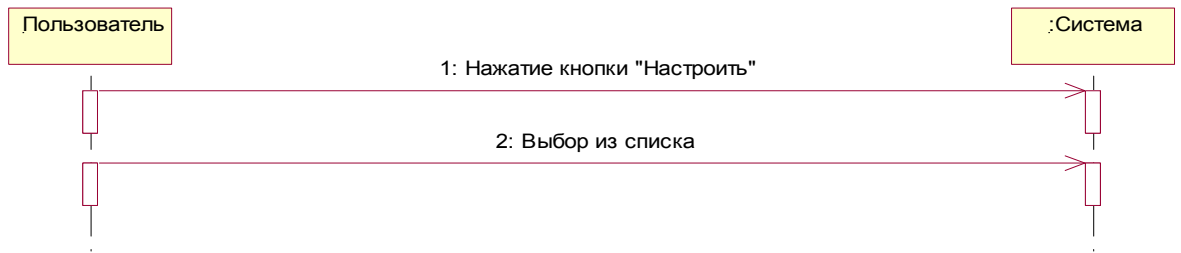


Рис. 10. Диаграмма последовательностей варианта использования
«Настроить»

Описание операции «Нажатие кнопки «Настроить»» представлено в табл. 25.

Таблица 25

Описание операции «Нажатие кнопки «Настроить»»

Раздел	Описание
Имя	Нажатие кнопки «Настроить»
Обязанности	Осуществление взаимодействия с настройками программы
Тип	Системная
Ссылки	Вариант использования «Настроить»
Примечания	Предоставляет пользователю список настроек работы программы
Исключения	-
Вывод	Список параметров
Предусловия	-
Постусловие	Запущена система /сообщение об ошибке

Описание операции «Выбор из списка» представлено в табл. 26.

Таблица 26

Описание операции «Выбор из списка»

Раздел	Описание
Имя	Выбор из списка
Обязанности	Осуществление взаимодействия пользователя с настройками программы
Тип	Системная
Ссылки	Вариант использования «Настроить»
Примечания	Управление настройками программы пользователем
Исключения	Некорректное значение
Вывод	Успешное изменение настроек
Предусловия	Открыто окно выбора настроек
Постусловие	Запущена система /сообщение об ошибке

1.4.3.6 Диаграмма последовательностей системы для варианта использования «Определение номера стальной»

Диаграмма последовательностей варианта использования «Определение номера стальной» представлена на рис.11.

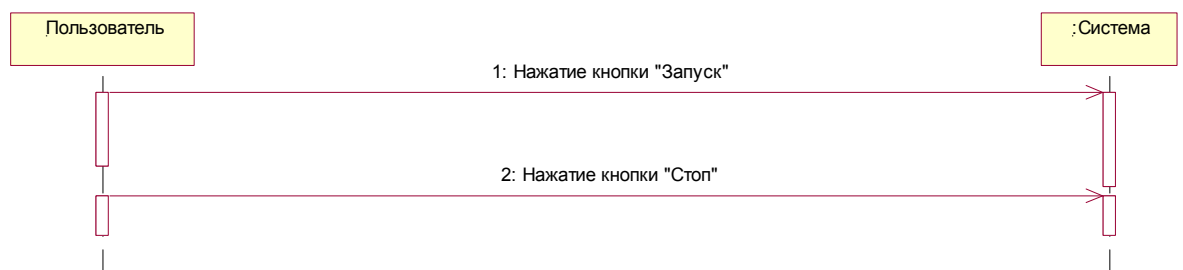


Рис. 11. Диаграмма последовательностей варианта использования «Определение номера стальной»

Описание операции «Нажатие кнопки «Запуск»» представлено в табл. 27.

Таблица 27

Описание операции «Нажатие кнопки «Запуск»»

Раздел	Описание
Имя	Нажатие кнопки «Запуск»
Обязанности	Определение номера стальнойковша на видеопотоке
Тип	Системная
Ссылки	Вариант использования «Определение номера стальнойковша на видеопотоке»
Примечания	Дать пользователю представление об успешности детектирования номера объекта
Исключения	Некорректноераспознавание номера
Вывод	На экране появляется выделенный контур номера объекта
Предусловия	Запущена система
Постусловие	Выделенный номер объекта /сообщение об ошибке

Описание операции «Нажатие кнопки «Стоп»» представлено в табл. 28.

Таблица 28

Описание операции «Нажатие кнопки «Стоп»»

Раздел	Описание
Имя	Нажатие кнопки «Стоп»
Обязанности	Завершение определения номера стальнойковша на видеопотоке
Тип	Системная
Ссылки	Вариант использования «Определение номера стальнойковша на видеопотоке»

Примечания	Видеопоток не обрабатывается
Исключения	-
Вывод	Необработанный видеопоток
Предусловия	Запущена система
Постусловие	-

1.4.3.7 Диаграмма последовательностей системы для варианта использования «Проверка статуса работы камеры»

Диаграмма последовательностей варианта использования «Проверка статуса работы камеры» представлена на рис.12.

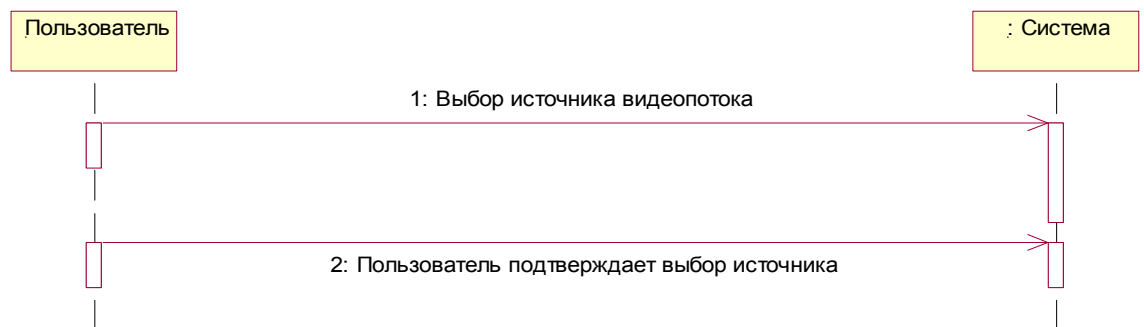


Рис. 12. Диаграмма последовательностей варианта использования «Проверка статуса работы камеры»

Описание операции «Проверка статуса работы камеры» представлено в табл.29.

Таблица 29

Описание операции «Выбор источника видеопотока»

Раздел	Описание
Имя	Выбор источника видеопотока
Обязанности	Определяет наличие видеопотока, поступающего от камеры
Тип	Системная
Ссылки	Вариант использования «Проверка статуса работы камеры»
Примечания	Дать пользователю представление о работоспособности камеры тепловизора
Исключения	Ошибка при подключении к камере
Вывод	Вывод сообщения об успешном подключении
Предусловия	Выбор из списка камер
Постусловие	Переход к окну подтверждения /сообщение об ошибке

Описание операции «Пользователь подтверждает выбор источника» представлено в табл. 30.

Таблица 30

Описание операции «Пользователь подтверждает выбор источника»

Раздел	Описание
Имя	Пользователь подтверждает выбор источника
Обязанности	Задаёт видеопоток для обработки
Тип	Системная
Ссылки	Вариант использования «Проверка статуса работы камеры»
Примечания	Распознавание происходит на выбранном пользователем видео потоке
Исключения	Ошибка при подключении к камере

Вывод	Вывод сообщения об успешном подключении
Предусловия	Выбор из списка камер
Постусловие	Переход к главному окну /сообщение об ошибке

1.4.3.8 Диаграмма последовательностей системы для варианта использования «Управление настройками видеопотока»

Диаграмма последовательностей варианта использования «Управление настройками видеопотока» представлена на рис.13.

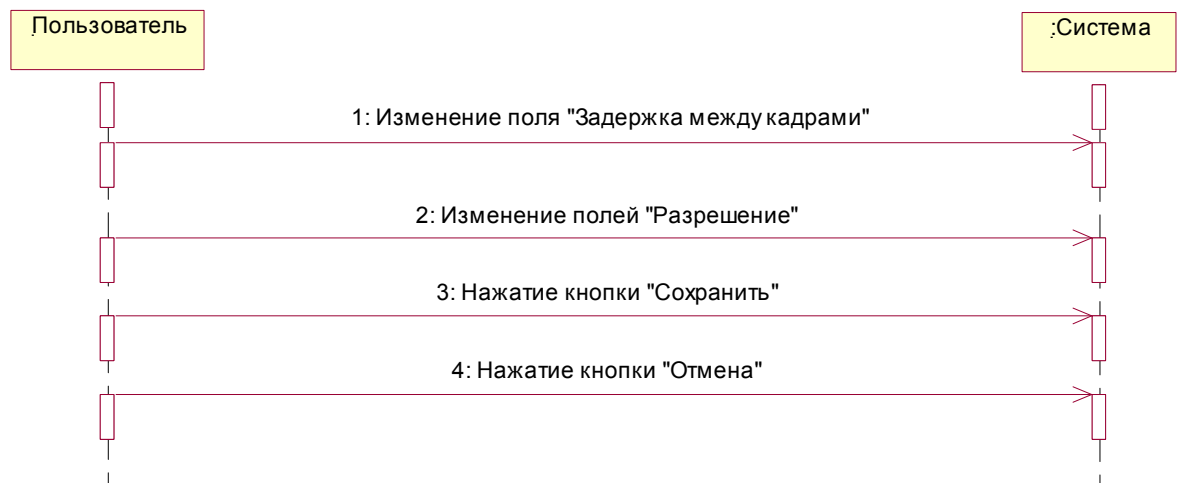


Рис. 13. Диаграмма последовательностей варианта использования «Управление настройками видеопотока»

Описание операции «Изменение поля «Задержка между кадрами»» представлено в табл. 31.

Таблица 31

Описание операции «Изменение поля «Задержка между кадрами»»

Раздел	Описание
Имя	Изменение поля «Задержка между кадрами»
Обязанности	Управление параметрами длительности задержки между кадрами видеопотока
Тип	Системная
Ссылки	Вариант использования «Управление параметрами видеопотока»
Примечания	Дать пользователю возможность настраивать длительности задержки между кадрами видеопотока
Исключения	Ошибка при изменении параметров
Вывод	Сообщение об успешном изменении параметров
Предусловия	Настроить
Постусловие	Запущена система /сообщение об ошибке

Описание операции «Изменение поле «Разрешение»» представлено в табл. 32.

Таблица 32

Описание операции «Изменение поле «Разрешение»»

Раздел	Описание
Имя	Изменение полей «Разрешение»
Обязанности	Управление параметрами разрешения видеопотока
Тип	Системная
Ссылки	Вариант использования «Управление параметрами видеопотока»
Примечания	Дать пользователю возможность настраивать разрешение видеопотока

Исключения	Ошибка при изменении параметров
Вывод	Сообщение об успешном изменении параметров
Предусловия	Настроить
Постусловие	Запущена система /сообщение об ошибке

Описание операции «Нажатие кнопки «Сохранить»» представлено в табл. 33.

Таблица 33

Описание операции «Нажатие кнопки «Сохранить»»

Раздел	Описание
Имя	Нажатие кнопки «Сохранить»
Обязанности	Применение пользовательских настроек
Тип	Системная
Ссылки	Вариант использования «Управление параметрами видеопотока»
Примечания	Дать пользователя возможность сохранить настройки
Исключения	Ошибка при изменении параметров
Вывод	Сообщение об успешном изменении параметров
Предусловия	Настроить
Постусловие	Запущена система /сообщение об ошибке

Описание операции «Нажатие кнопки «Сохранить»» представлено в табл. 34.

Таблица 34

Описание операции «Нажатие кнопки «Отмена»»

Раздел	Описание
Имя	Нажатие кнопки «Отмена»
Обязанности	Откат пользовательских настроек
Тип	Системная
Ссылки	Вариант использования «Управление параметрами видеопотока»
Примечания	Дать пользователя возможность отменить внесенные изменения
Исключения	Ошибка при изменении параметров
Вывод	Сообщение об успешном изменении параметров
Предусловия	Настроить
Постусловие	Запущена система /сообщение об ошибке

1.4.4 Построение диаграмм деятельности вариантов использования

В зависимости от степени детализации диаграммы деятельности так же, как диаграммы классов, используют на разных этапах разработки. На этапе анализа требований и уточнения спецификаций диаграммы деятельности позволяют конкретизировать основные функции разрабатываемого программного обеспечения.

Под деятельностью в данном случае понимают задачу (операцию), которую необходимо выполнить вручную или с помощью средств автоматизации. Каждому варианту использования соответствует своя последовательность задач. В теоретическом плане диаграммы деятельности являются обобщенным представлением алгоритма, реализующего анализируемый вариант использования. На диаграмме деятельность обозначается прямоугольником с закругленными углами [7].

Диаграммы деятельности позволяют описывать альтернативные и параллельные процессы. Для обозначения альтернативных процессов используют ромб [10].

Проанализируем один из сценариев варианта использования «Определение номера стальнойковша». Разбиваем процесс на отдельные операции. Полученные операции показаны на диаграмме деятельности (рис. 14).

Описание деятельности представлено в табл. 35.

Таблица 35

Описание деятельности для варианта использования «Определение номера стальнойковша»

№	Деятельность	Описание
1	2	3
1	Нажатие кнопки «Запуск»	Система распознавания номера стальнойковша запущена
2	Поиск номера на стальнойковше	Система пытается распознать номер нанесенный на стальнойковше
3	Запись в лог файл ошибки	В случае неудачи при распознавании номера в лог файл заносится ошибка, что стальнойковш был обнаружен, но его номер считать не удалось

Продолжение табл. 35

1	2	3
4	Выделение контура номера	Контур номера выделяется рамкой в окне просмотра видеопотока
5	Запись номера в лог файла	При успешном распознавании в лог файл заносится номер стальнойковша
6	Продолжается поиск стальнойковша	Система продолжает искать объект стальнойковш и его номер

Оператор системы осуществляет первичную настройку ПО после включения системы перед выбором источника видеопотока.

По умолчанию, система использует исходные настройки, которые задал разработчик. Если оператора они не устраивают, он вносит корректировки и подтверждает их.

Следующим шагом является подключение к источнику видеопотока либо загрузки его из видеофайла. Оператор выбирает источник или подключается к IP-камере.

Если видеосигнал присутствует, оператор запускает процесс детектирования нажатием на кнопку «Запуск системы».

Когда необходимо, оператор запускает процедуру остановки процесса детектирования нажатием кнопки «Остановить систему».

Во время работы программы нейронная сеть ищет контур объекта стальной на видеопотоке, после его нахождения выделяет контур и в пределах данной области пытается найти номер.

Данные полученные на этапе распознавания заносятся в БД или лог файл и в таблицу на экране оператора. Оператор имеет возможность сохранять результаты работы как в базу данных, так и лог-файл одновременно или по отдельности.

Если номер стальной не найден, то в поле номер заносится текст «Отсутствует».

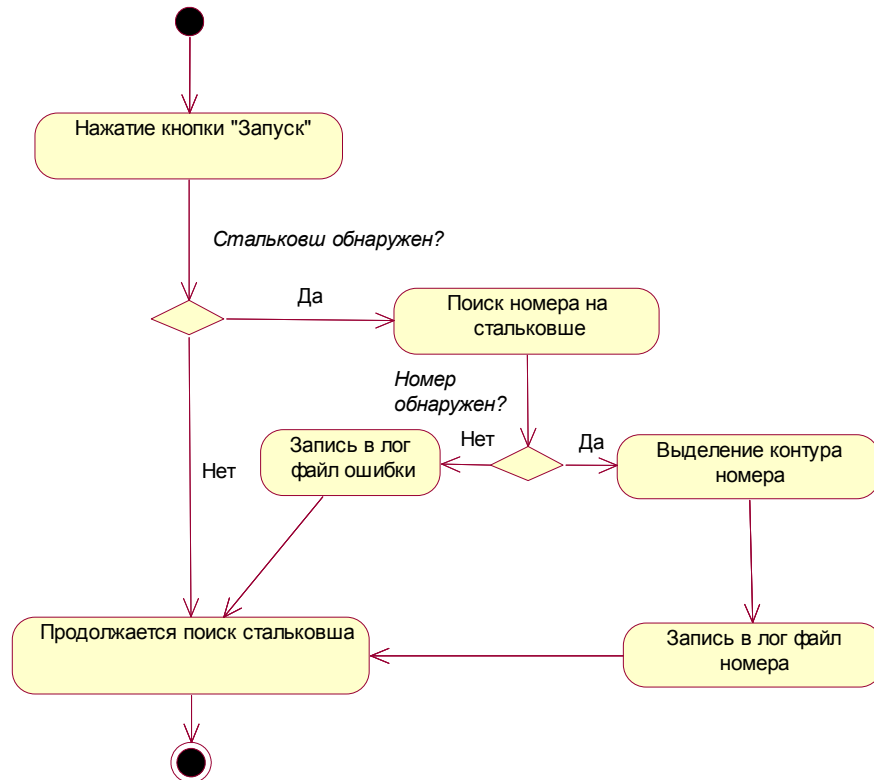


Рис. 14. Диаграмма деятельности варианта использования «Определение номера стальковша»

1.4.5 Проектирование структур данных и диаграмм отношений

Любая информационная система работает с данными, которые используются: в работе программы, для сбора статистики, формирования отчетов и т.д. Все собранные данные необходимо где-то хранить. Существуют разные способы хранения данных, которые имеют свои преимущества и недостатки, но самый распространенный и оптимальный способ хранения данных — это база данных.

База данных (БД) — это организованная структура, предназначенная для хранения, изменения и обработки взаимосвязанной информации [8].

Для проектирования ER-диаграммы было реализовано CASE-инструментом ERwin Data Modeler (рис. 15).

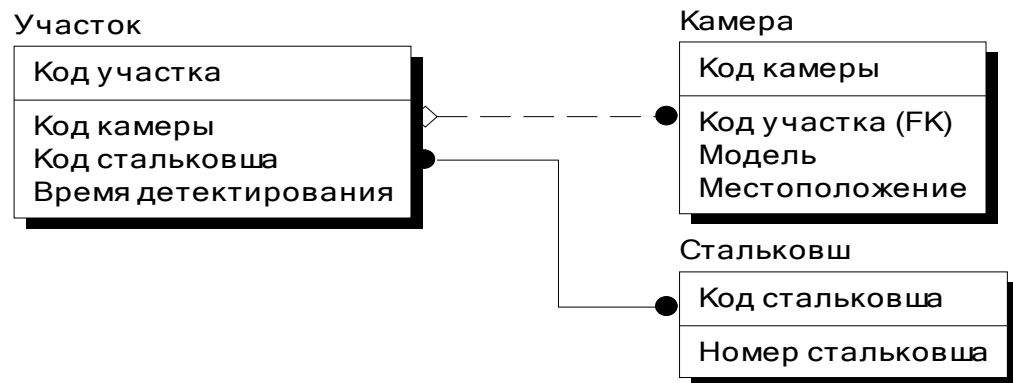


Рис. 15. ER-диаграмма

В ходе проектирования были выделены три сущности: «Участок», «Камера», «Стальковш».

Сущность «Камера» содержит в себе данные о камере, а именно: её модель, место установки, код участка и код камеры тепловизора (табл.36).

Таблица 36

Спецификации таблицы Камера

Поле	Тип данных	Модификаторы	Назначение
CodPoint	Number	Первичный ключ	Код
Model	String	-	Модель камеры
ID	String	-	Код камеры
Position	String	-	Местоположение

Сущность «Стальковш» содержит в себе данные о номере стальковша, и его код (необходим для идентификации стальковша при нераспознанном номере) (табл.37).

Таблица 37

Спецификации таблицы Стальковш

Поле	Тип данных	Модификаторы	Назначение
ID	Number	-	Код стальнойковша
Number	String	-	Номер стальнойковша

Сущность «Участок» является главной сущностью и содержит информацию о коде участка, о коде камеры, о стальнойковше и времени детектирования. С данной сущностью взаимодействуют сущности «Камера», «Стальнойковш» (табл.38).

Таблица 38

Спецификации таблицы Участок

Поле	Тип данных	Модификаторы	Назначение
CodStellcovsh	Number	-	Код стальнойковша
IDCamera	String	-	Код камеры
ID	String	-	Код участка
Time	String	-	Время детектирования

1.4.6 Построение диаграммы переходов состояний

Диаграмма переходов состояний – описание возможных последовательностей состояний и переходов, которые в совокупности характеризуют поведение элемента модели в течение его жизненного цикла.

Диаграмма состояний представляет динамическое поведение сущностей, на основе спецификации их реакции на восприятие некоторых конкретных событий.

С её помощью можно моделировать последующее функционирование системы на основе ее предыдущего и текущего функционирования. Моделируемая система в любой заданный момент времени находится точно в одном из конечного множества состояний. С течением времени она может

изменить свое состояние, при этом переходы между состояниями должны быть точно определены.

На рис. 16 представлена диаграмма состояний системы, начальным состоянием является открытие главного окна приложения, затем выставляются необходимые параметры детектирования, выбирается источник видеопотока и запускается процесс распознавания.

После процесс может быть остановлен. Конечным состоянием системы является закрытие главного окна программы.

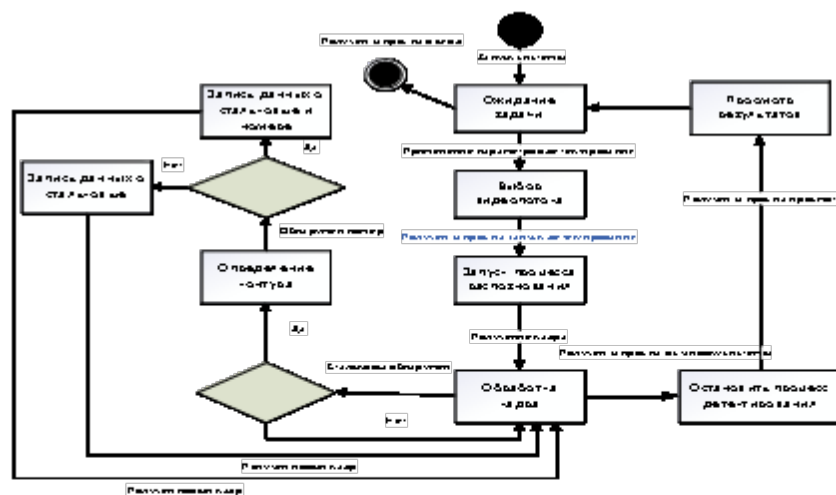


Рис. 16. Диаграмма переходов состояний системы

1.5 Проектирование системы

Основной задачей логического проектирования при объектном подходе является разработка классов для реализации объектов, полученных при объектной декомпозиции, что предполагает полное описание полей и методов каждого класса.

Физическое проектирование при объектном подходе включает объединение классов и других программных ресурсов в программные компоненты, а также размещение этих компонентов на конкретных вычислительных устройствах[7].

1.5.1 Проектирование структуры системы и построение диаграмм пакетов

Пакетом при объектном подходе называется совокупность описаний классов и других программных ресурсов, в том числе и самих пакетов. Объединение в пакеты используют для удобства создания больших проектов, количество классов в которых велико. При этом в один пакет обычно собирают классы и другие ресурсы одинакового назначения[7].

Диаграмма пакетов показывает, из каких частей состоит проектируемая программная система, и как эти части связаны друг с другом[7].

Связь между пакетами определяется внешними связями классов и других ресурсов, объединенных в пакет[7].

Анализ диаграммы вариантов использования (см. рис. 2) и контекстной диаграммы классов (см. рис. 3). Описание пакетов представлено в табл. 39. Диаграмма пакетов представлена на рис. 17.

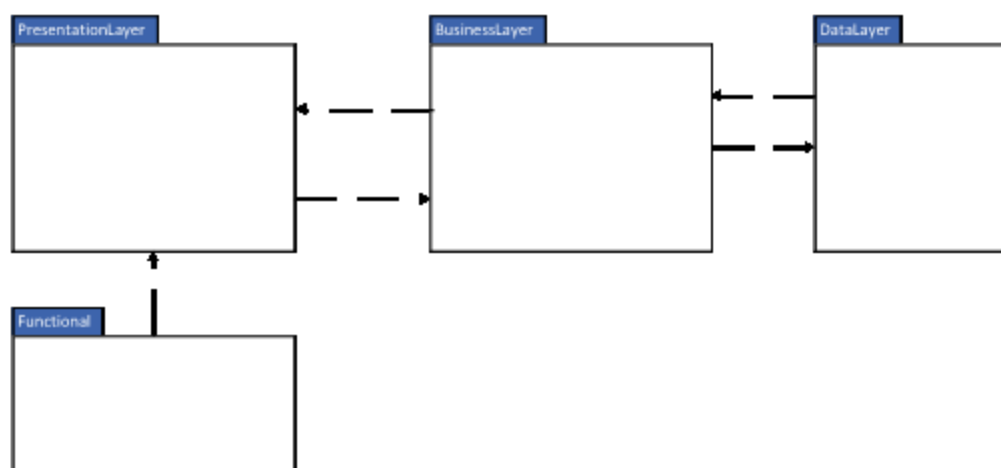


Рис. 17. Диаграмма пакетов

Таблица 39

Описание пакетов

Пакет	Описание
Presentation Layer	Пакет, содержащий классы, определяющие пользовательский интерфейс и средства взаимодействия с ним, обработку видеопотока
Business Layer	Пакет, содержащий классы, которые содержат логику работы программы и являющиеся связующим звеном между пакетами Presentation Layer и Data Layer. Определяют механизмы по работе с данными и их отображению, работают с пакетом вспомогательных функций Functional
DataLayer	Пакет, содержащий классы, описывающие данные предметной области используемые в программе, а также логику по взаимодействию с ними
Functional	Пакет, содержащий классы, отвечающие за вспомогательные функции на прямую, не относящиеся к предметной области

1.5.2 Проектирование классов в пакетах

После определения основных пакетов разрабатываемого программного обеспечения переходят к детальному проектированию классов, входящих в каждый пакет.

Классы-кандидаты, которые предположительно должны войти в конкретный пакет, показывают на диаграмме классов этапа проектирования и уточняют отношения между объектами указанных классов[7].

Ниже будет рассмотрено проектирование пакетов и их классов.

1.5.2.1 Проектирование классов пакета «BusinessLayer»

Пакет «BusinessLayer» содержит основную логику программы, отвечает за взаимодействия с данными, их обработкой, пользовательским интерфейсом.

Определяют механизмы по работе с данными и их отображению, работают с пакетом вспомогательных функций.

1.5.2.1.1 Построение исходной диаграммы классов пакета «BusinessLayer»

Диаграмма классов служит для представления структуры модели системы в терминологии классов объектно-ориентированного программирования. Диаграмма классов может отражать различные взаимосвязи между отдельными сущностями предметной области, такими как объекты и подсистемы, а также описывает их внутреннюю структуру (поля, методы и т.д.) и типы отношений. На данной диаграмме не указывается информация о временных аспектах функционирования системы. С этой точки зрения диаграмма классов является дальнейшим развитием концептуальной модели проектируемой системы. На этом этапе принципиально знание ООП подхода и паттернов проектирования[18].

В табл.40 представлено описание классов пакета«BusinessLayer». Исходная диаграмма классов представлена на рис. 18.

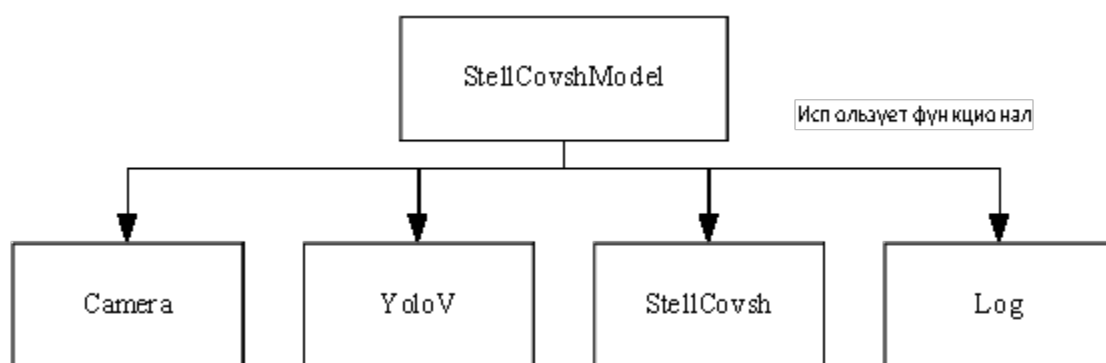


Рис. 18. Исходная диаграмма классов пакета «BusinessLayer»

Описание классов пакета «BusinessLayer»

Класс	Описание
StellCovshModel	Класс используют функционал классов Camera, Log, StellCovsh и результат работы класса YoloV и отвечает за корректное отображение, запись обнаруженных объектов
StellCovsh	Класс, реализующий тип данных характеризующий стальковш
YoloV	Класс, определяющий работу нейросети и ее конфигурацию
Log	Класс, определяющий формат записи в лог файл и БД
Camera	Класс, реализующий тип данных, характеризующий камеру тепловизора

1.5.2.1.2 Построение диаграммы последовательности взаимодействия объектов классов пакета «BusinessLayer»

Диаграммы последовательностей этапа проектирования отображают взаимодействие объектов, упорядоченное по времени. В отличие от диаграмм последовательности этапа анализа на ней показывают внутренние объекты, а также последовательность сообщений, которыми обмениваются объекты в процессе реализации фрагмента варианта использования, называемого сценарием [7].

Анализ описания варианта использования показывает, что необходимо рассмотреть три варианта последовательности действий:

- 1) нормальный ход событий;
- 2) прерывание процесса оператором;
- 3) прерывание процесса системой.

На рис. 19-21 представлены диаграммы последовательностей взаимодействия объектов классов пакета «BusinessLayer»

При нормальном ходе событий, в момент запуска системы происходит инициализация класса YoloV, после выбора источника видеопотока определяются поля статического класса Camera, в дальнейшем при работе системы и удачном распознавании происходит вызов метода Logging класса StellCovshModel, в котором происходит инициализация объекта класса StellCovshc соответствующими полями. Затем происходит инициализация объекта класса Logv который заносятся данные объекта StellCovsh.

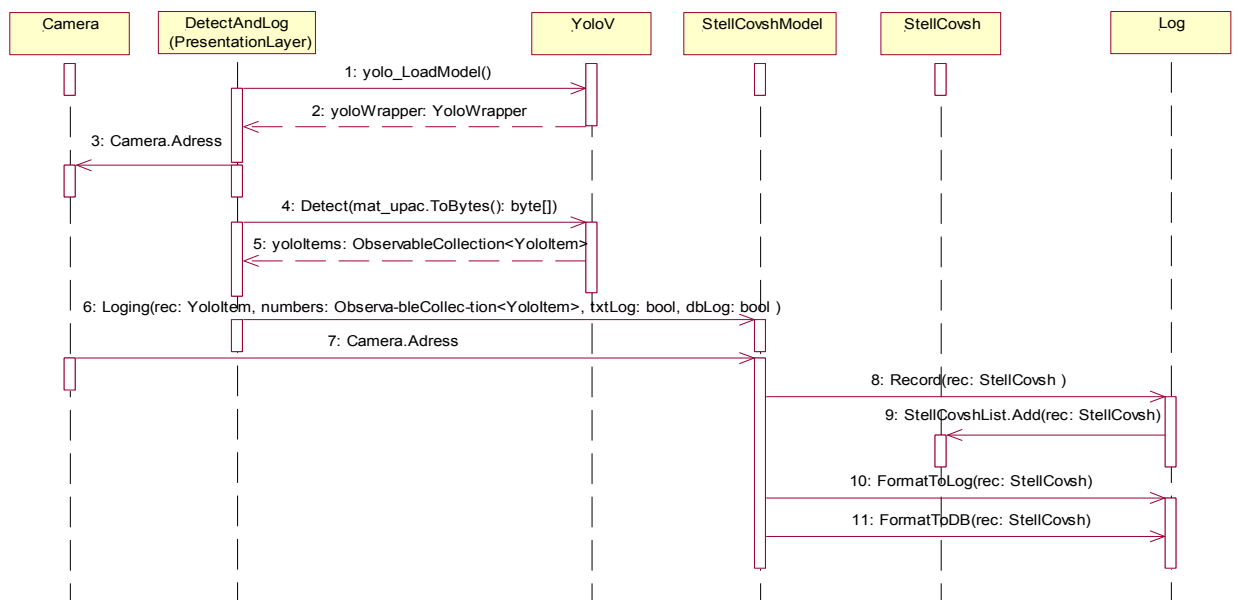


Рис.19. Диаграмма последовательности взаимодействия объектов классов пакета «BusinessLayer». Нормальный ход событий

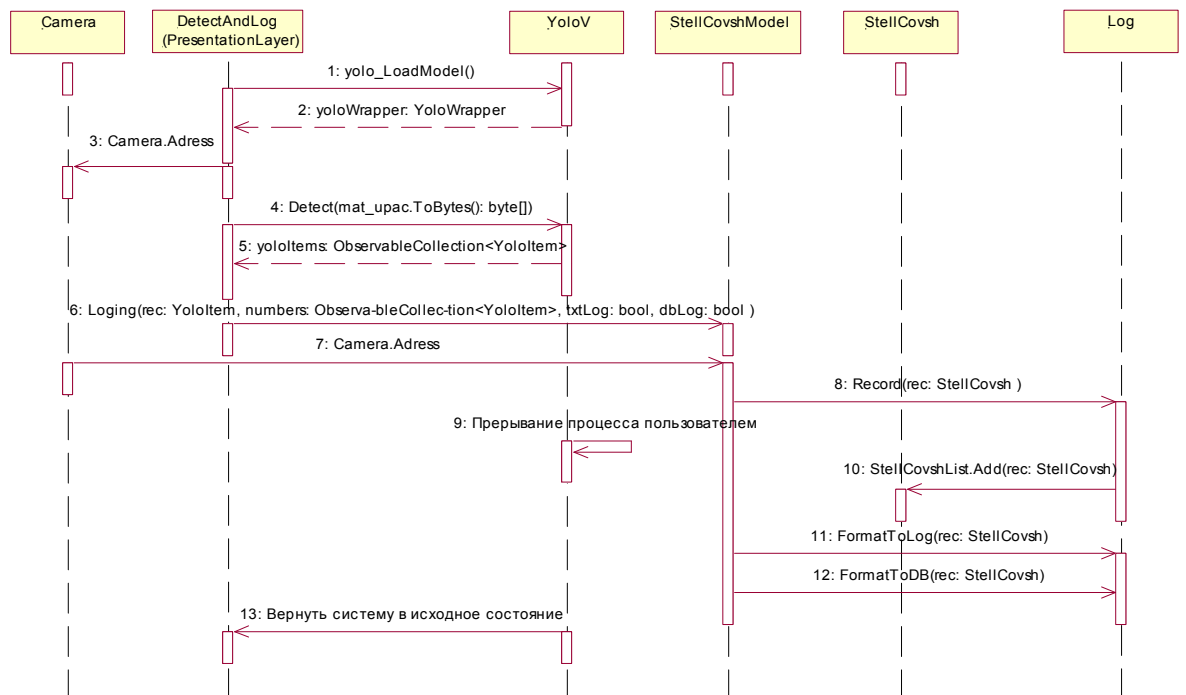


Рис.20. Диаграмма последовательности взаимодействия объектов классов пакета «BusinessLayer». Прерывание процесса оператором

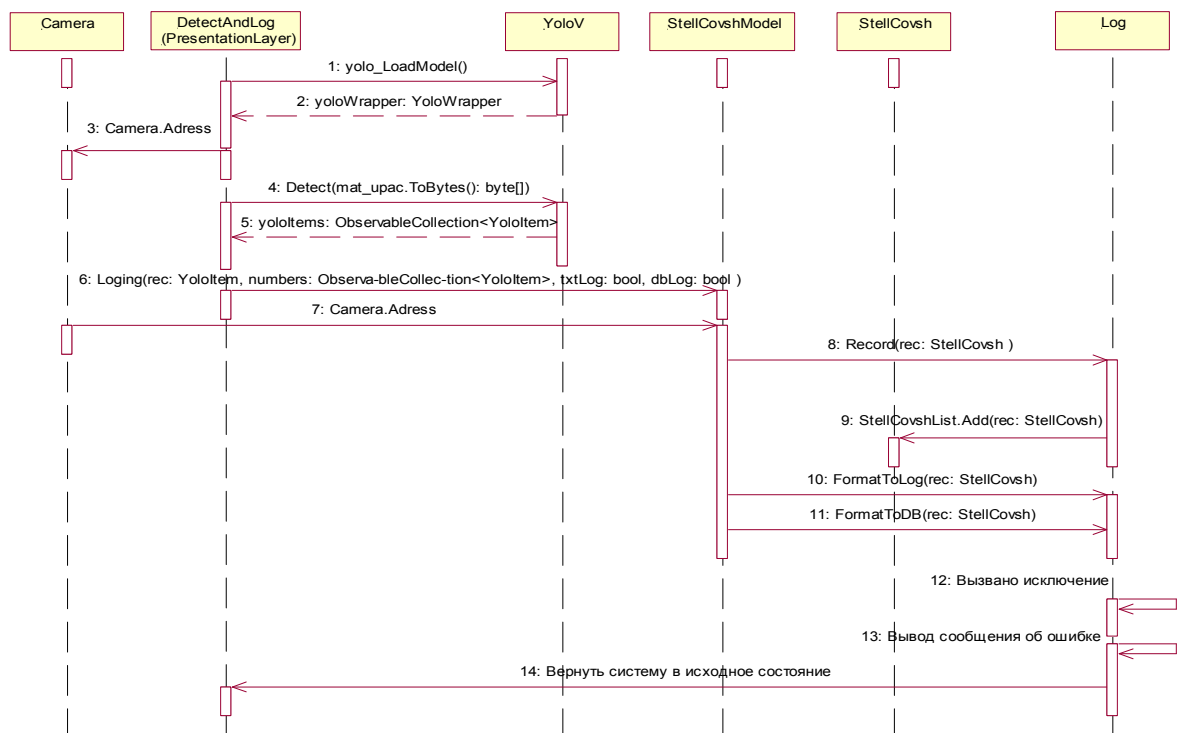


Рис.21. Диаграмма последовательности взаимодействия объектов классов пакета «BusinessLayer». Прерывание процесса системой
1.5.2.1.3 Построение диаграммы кооперации пакета «BusinessLayer»

Диаграмма кооперации — это альтернативный способ представления взаимодействия объектов в процессе реализации сценария, который позволяет по-другому взглянуть на ту же информацию. В отличие от диаграмм последовательностей диаграммы кооперации показывают потоки данных между объектами классов, что позволяет уточнить связи между ними [7].

Диаграмма кооперации рассматриваемого пакета «BusinessLayer» представлена на рис.22.

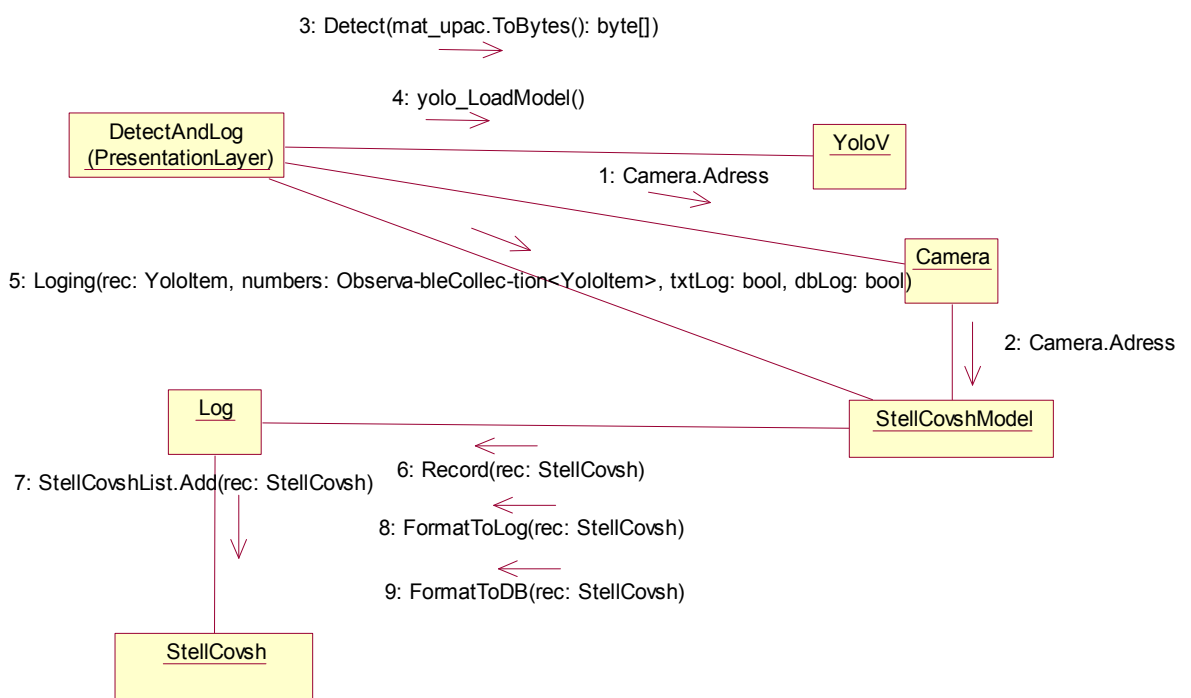


Рис.22. Диаграмма кооперации

1.5.2.1.4 Построение уточнений диаграммы классов пакета «BusinessLayer»

Процесс проектирования классов начинают с уточнения отношений между ними.

Класс `StellCovshModel` используют функционал классов `Camera`, `Log`, `StellCovshi` результат работы класса `YoloV` отвечает за корректное отображение обнаруженных объектов. `YoloV` содержит внутри конфигурацию нейронной сети. Класс `StellCovsh` содержит функционал для описания объекта предметной области «стальковш». Класс `Log` содержит функционал для формирования записи в лог файл и БД.

Уточненная диаграмма классов пакета представлена на рис.23.

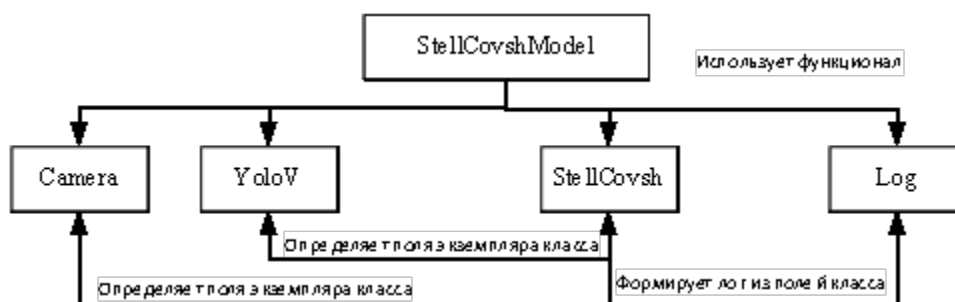


Рис. 23. Уточненная диаграмма классов пакета «BusinessLayer»

1.5.2.1.5 Построение детальной диаграммы классов пакета «BusinessLayer»

Построение детальной диаграммы классов предполагает проектирование окончательной структуры и поведения его объектов. Структура объектов определяется совокупностью атрибутов и операций класса[7].

Диаграмма классов — это набор статических, декларативных элементов модели. Диаграммы классов могут применяться и при прямом проектировании,

то есть в процессе разработки новой системы, и при обратном проектировании - описании существующих и используемых систем[7].

Детальная диаграмма классов представлена на рис. 24.

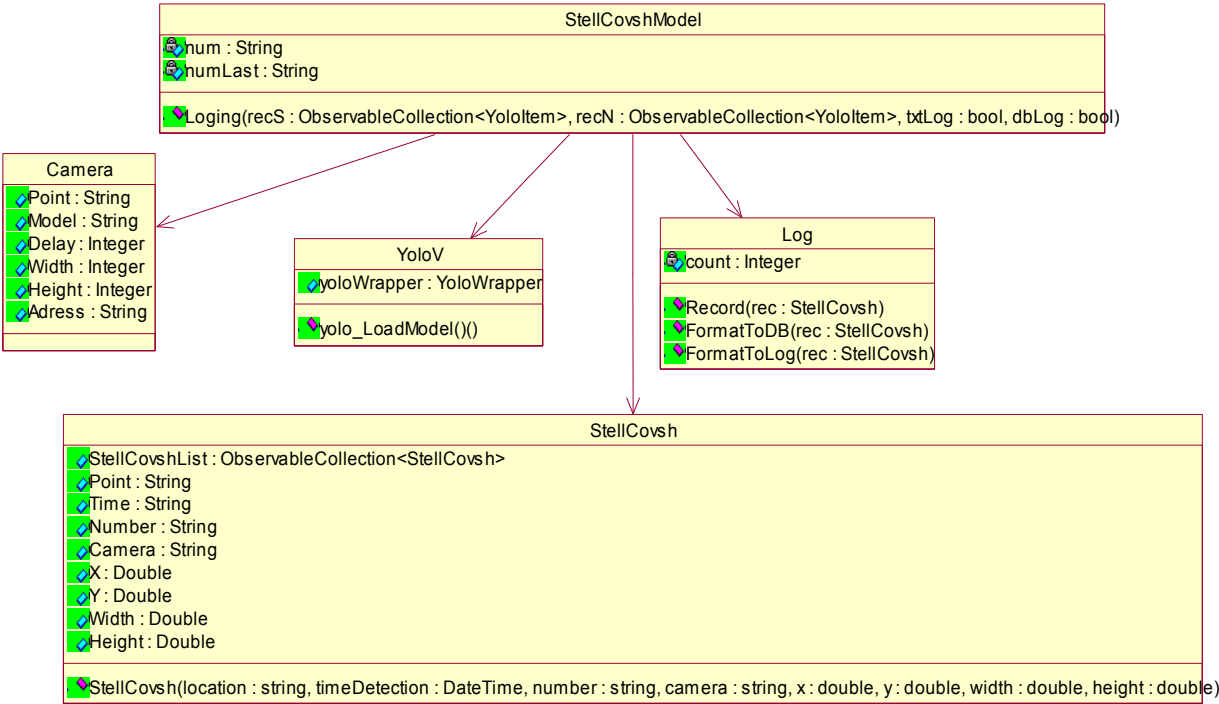


Рис. 24. Детальная диаграмма классов пакета «BusinessLayer»

Описаниеметодови полей класса «StellCovshModel» представлено в табл. 41,42.

Таблица 41

Описаниеметодов класса «StellCovshModel»

Название метода	Параметры	Возвращаемое значение	Описание
Logging	recS:ObservableCollection<YoloItem>,recN: ObservableCollection<YoloItem>, txtLog : bool, dbLog : bool	Отсутствует	Метод определения, фильтрация и передачи данных для записи обнаруженных нейросетью объектов

Таблица 42

Описание полей«StellCovshModel»

Название поля	Описание
num: String	Номер стальнойковши
numLast: String	Номер предыдущего стальнойковши

Описание полей и методов класса «YoloV» представлено в табл. 43, 44.

Таблица 43

Описание методов«YoloV»

Название метода	Параметры	Возвращаемое значение	Описание
yolo_LoadModel	Отсутствует	Отсутствует	Метод загрузки нейросети детектирующей стальнойковши
yoloNumber_LoadModel	Отсутствует	Отсутствует	Метод загрузки нейросети детектирующей номера

Таблица 44

Описание полей«YoloV»

Название поля	Описание
yoloWrapper: YoloWrapper	Экземпляр класса нейросети детектирующей стальнойковши
yoloWrapperNumber: YoloWrapper	Экземпляр классанейросети детектирующей номера

Описание полей и методов класса «StellCovsh» представлено в табл. 45, 46.

Таблица 45

Описание методов класса «StellCovsh»

Название метода	Параметры	Возвращаемое значение	Описание
StellCovsh	location: string, timeDetection: DateTime, number: string, camera: string, x: double, y: double, width: double, height: double	Отсутствует	Конструктор спараметрами

Таблица 46

Описание полей класса «StellCovsh»

Название поля	Назначение
1	2
StellCovshList: ObservableCollection<StellCovsh >	Список детектированных стальных
Point : String	Участок детектирования
Time : String	Время детектирования
Number : String	Номер стальной

Продолжение табл.46

1	2
Camera : String	Адрес камеры
X : Double	Координата стальной
Y : Double	Координата стальной
Width : Double	Ширина стальной
Height : Double	Высота стальной

Описание полей и методов класса «Log» представлено в табл. 47, 48.

Таблица 47

Описание методов и полей класса «Log»

Название	Назначение
count: Integer	Номер записи в файле

Описание методов класса «Log»

Название метода	Параметры	Возвращаемое значение	Описание
Record	rec: StellCovsh	Отсутствует	Добавление стельковша в список
FormatToDB	rec: StellCovsh	Отсутствует	Метод, конвертирующий запись в формат пригодный для записи в БД
FormatToLog	rec: StellCovsh	Отсутствует	Метод, конвертирующий запись в формат пригодный для записи в лог файл

1.5.2.2 Проектирование классов пакета «PresentationLayer»

Пакет «PresentationLayer» содержит логику взаимодействия пользовательского интерфейса с логикой работы программы, работу с видеопотоком, нейросетью.

1.5.2.2.1 Построение исходной диаграммы классов пакета «PresentationLayer»

В табл.49 представлено описание классов пакета «PresentationLayer». Исходная диаграмма классов представлена на рис. 25.

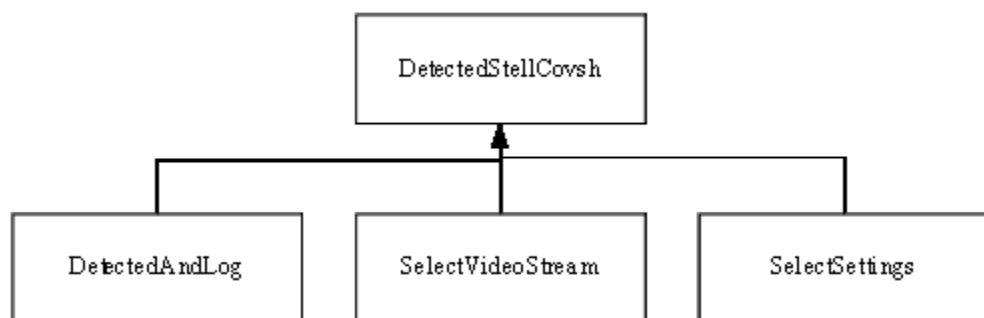


Рис. 25. Исходная диаграмма классов пакета «PresentationLayer»

Описание классов пакета «PresentationLayer»

Класс	Описание
DetectedStellCovsh	Класс, в котором определен базовый функционал остальных моделей представления
DetectedAndLog	Класс, реализующий алгоритм распознавания номера стальной, выделения контура найденного объекта и работу модуля логирования
SelectVideoStream	Класс, определяющий работу модуля выбора источника видео потока и его проверку
SelectSettings	Класс, отвечающий за применение пользовательских параметров к системе

1.5.2.2.2 Построение диаграммы последовательности взаимодействия объектов классов пакета «PresentationLayer»

На рис. 26-28 представлены диаграммы последовательностей взаимодействия объектов классов пакета «PresentationLayer» для варианта использования распознавание номера стальной. Классы данного пакета используют функционал пакета DetectedStellCovsh. Основной функционал сосредоточен в классе DetectedAndLog.

При запуске системы происходит следующее. В первую очередь инициализируется класс SelectSettingsc параметрами программы, в дальнейшем происходит инициализация объекта класса SelectVideoStream отвечающий за выбор источника видеопотока. Непосредственное детектирование и логирование происходит в классе DetectedAndLog.



Рис.26. Диаграмма последовательности взаимодействия объектов классов пакета «PresentationLayer». Нормальный ход событий

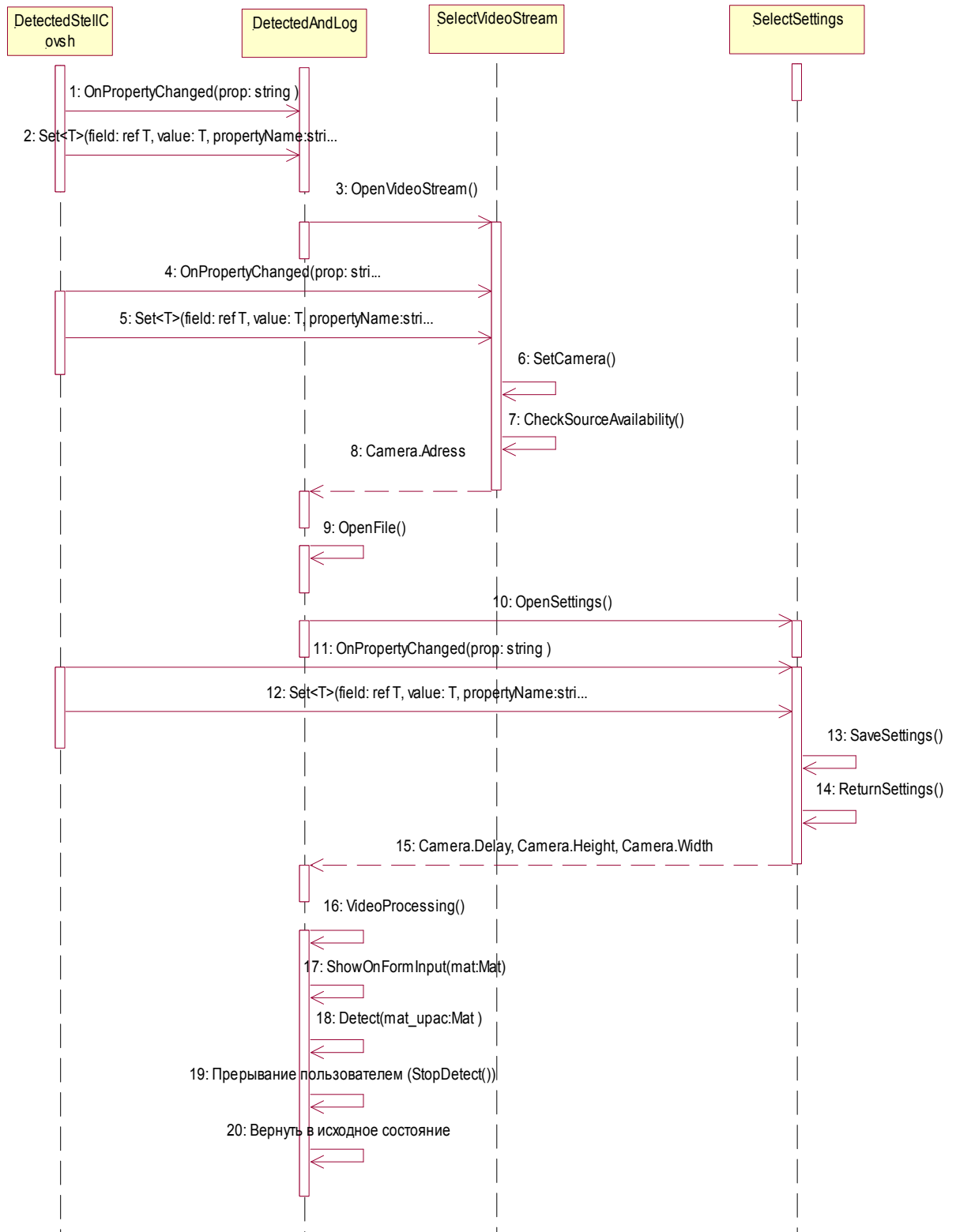


Рис.27. Диаграмма последовательности взаимодействия объектов классов пакета «PresentationLayer». Прерывание процесса оператором

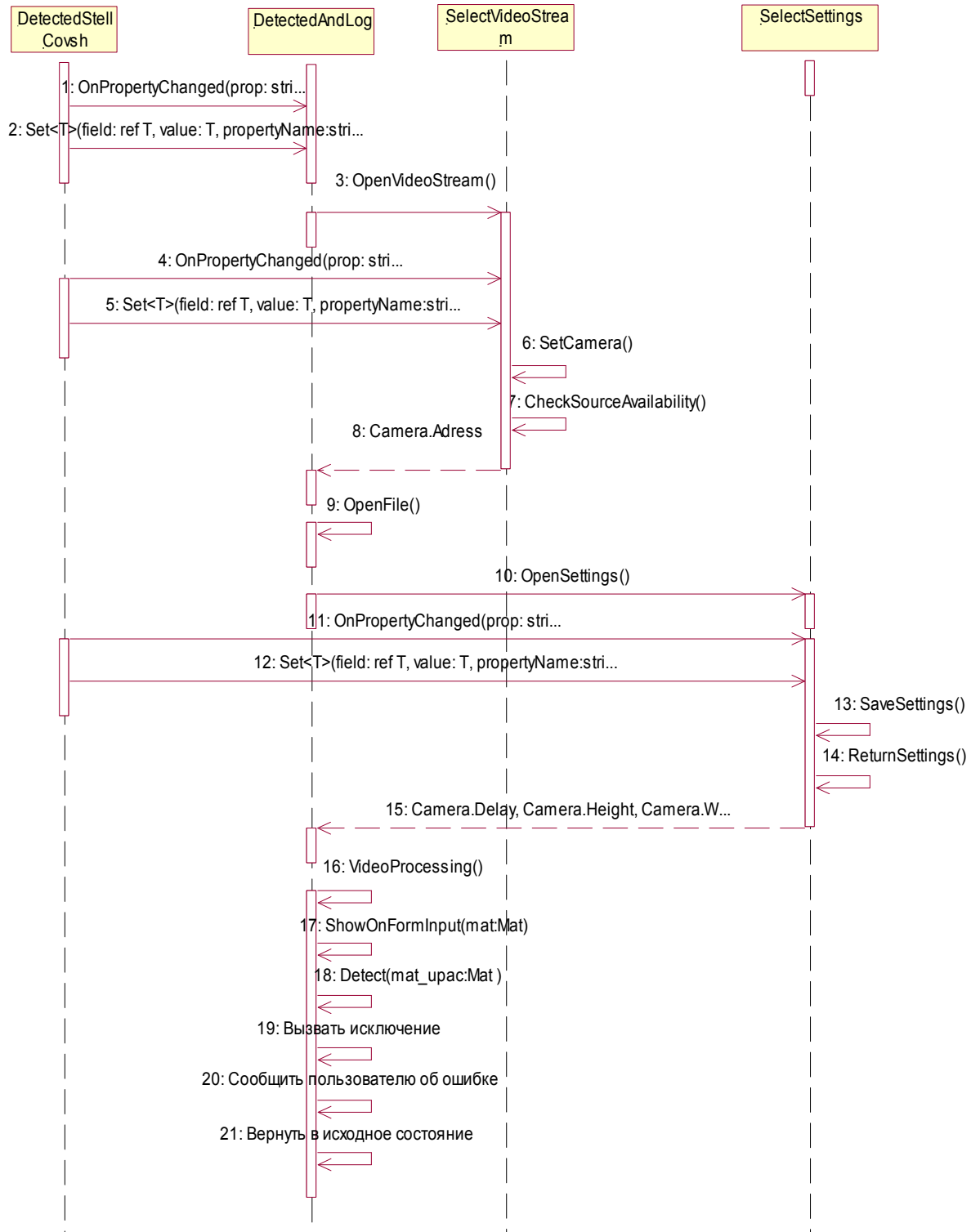


Рис.28. Диаграмма последовательности взаимодействия объектов классов пакета «PresentationLayer». Прерывание процесса системой

1.5.2.2.3 Построение диаграммы кооперации пакета «PresentationLayer»

Диаграмма кооперации рассматриваемого пакета «PresentationLayer» представлена на рис.29.

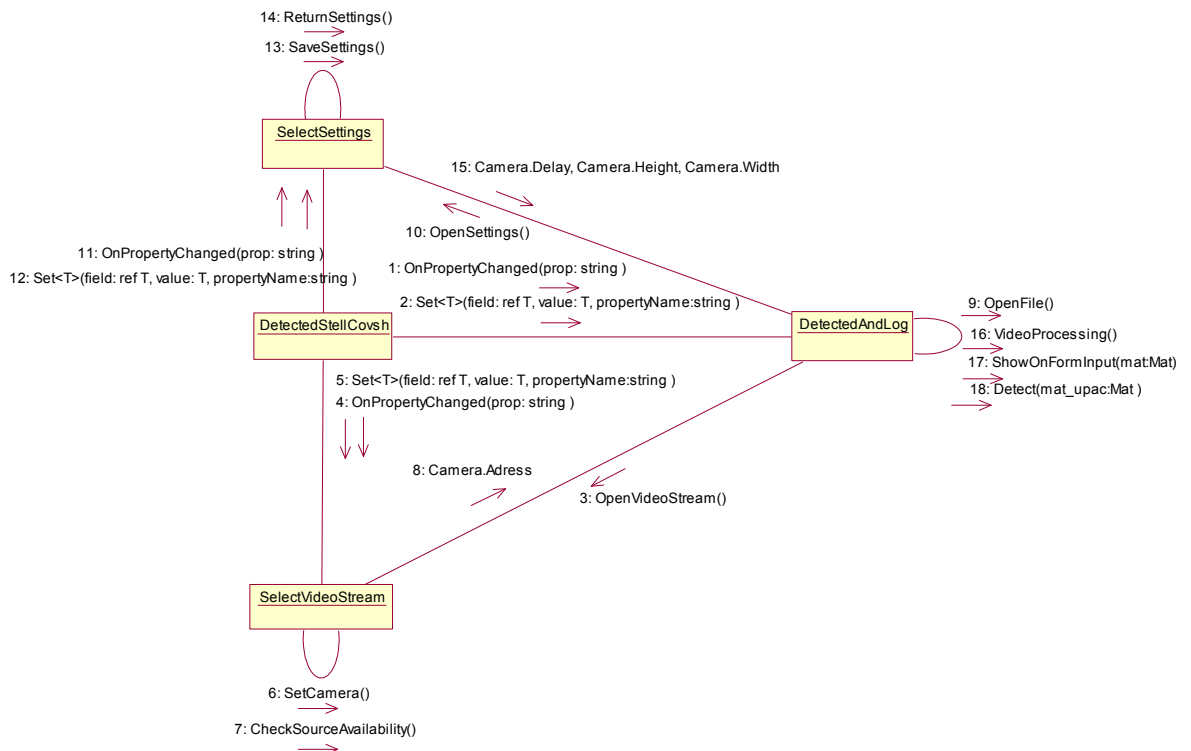


Рис.29. Диаграмма кооперации

1.5.2.2.4 Построение уточнений диаграммы классов пакета «PresentationLayer»

Процесс проектирования классов начинают с уточнения отношений между ними.

Анализ диаграммы кооперации показывает, что основное взаимодействие между классами в пакете происходит в классе «DetectedAndLog», а именно работа с видеопотоков и логирование. Базовый функционал наследуется из класса «DetectedStellCovsh».

Уточненная диаграмма классов пакета представлена на рис.30.

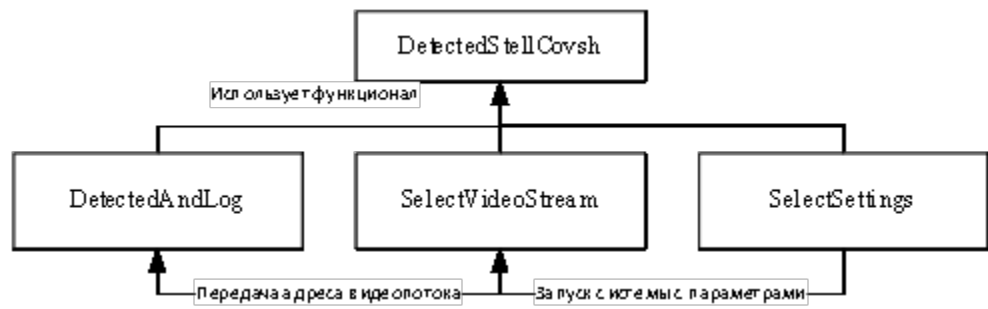


Рис. 30. Уточненная диаграмма классов пакета «PresentationLayer»

1.5.2.2.5 Построение детальной диаграммы классов пакета «PresentationLayer»

Детальная диаграмма классов представлена на рис. 31.

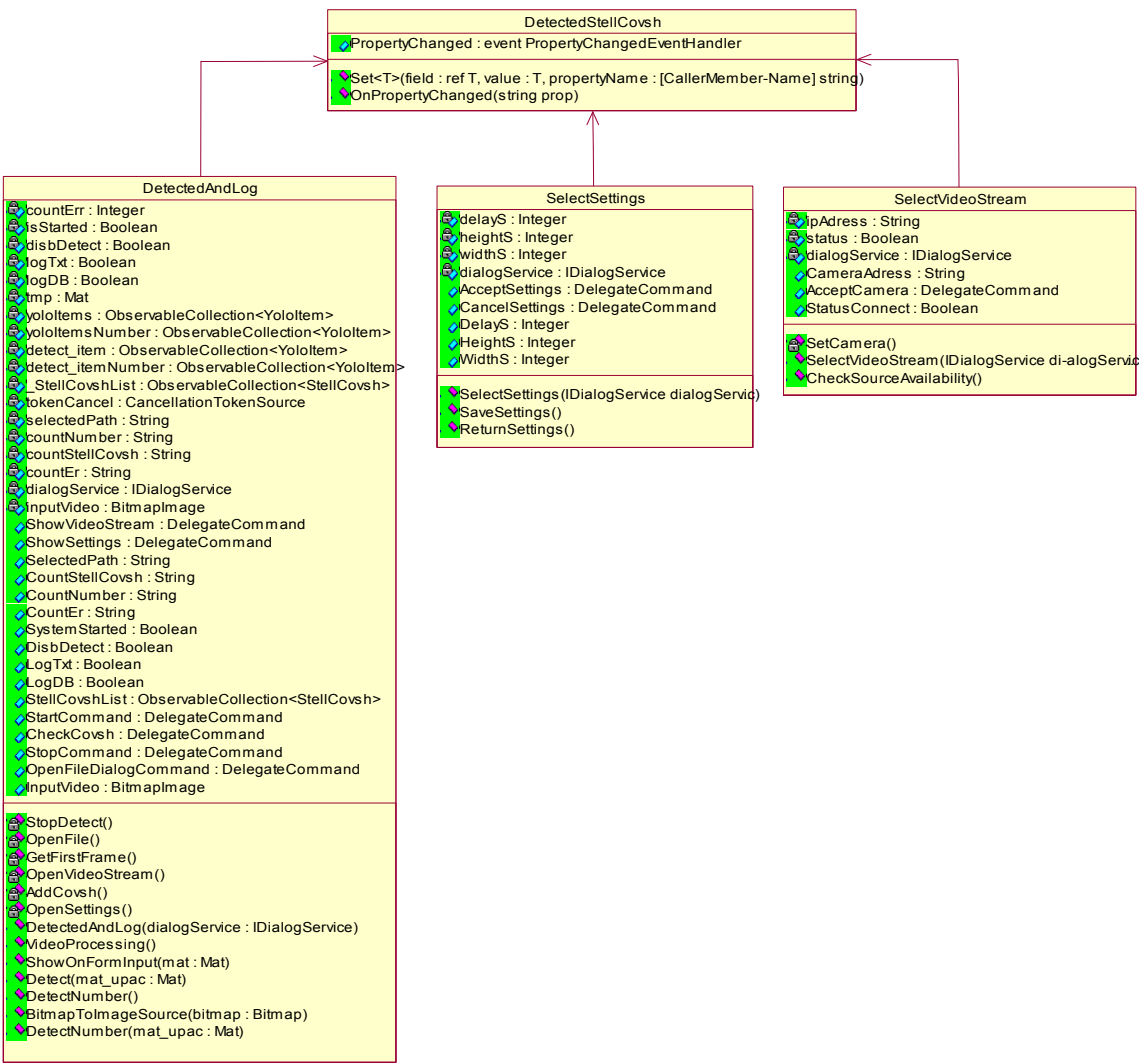


Рис. 31. Детальная диаграмма классов пакета «PresentationLayer»

Описание методов и полей класса «DetectedStellCovsh» представлено в табл. 50, 51.

Таблица 50

Описание методов класса «DetectedStellCovsh»

Название метода	Параметры	Возвращаемое значение	Описание
Set<T>	field: ref T, value: T, propertyName: [CallerMemberName] string	bool	Метод изменяет значение свойства и меняет его состояние на форме
OnPropertyChanged	prop: [CallerMemberName] string	Отсутствует	Метод изменяет состояние объекта на форме

Таблица 51

Описание полей класса «DetectedStellCovsh»

Название	Описание
PropertyChanged: PropertyChangedEventHandler	Событие изменения состояния объекта на форме

Описание полей и методов класса «DetectedAndLog» представлено в табл. 52, 53.

Таблица 52

Описание методов «DetectedAndLog»

Название метода	Параметры	Возвращаемое значение	Описание
1	2	3	4
DetectedAndLog	dialogService : IDialogService	Отсутствует	Конструктор с параметрами
StopDetect	Отсутствует	Отсутствует	Метод останавливает работу системы
OpenFile	Отсутствует	Отсутствует	Метод вызывает диалоговое окно выбора видеофайла для распознавания

Продолжение табл. 52

1	2	3	4
GetFirstFrame	Отсутствует	Отсутствует	Метод вывода стартового кадра на экран заданного видеопотока
VideoProcessing	Отсутствует	Отсутствует	Метод воспроизведения видео
ShowOnFormInput	mat: Mat	Отсутствует	Метод обработки кадра видеопотока и вывода его на экран

Detect	mat_upac: Mat	ObservableCollection <YoloItem>	Метод передачи кадра видеопотока для детекции нейросетью, по типу - стальковш
DetectNumber	mat_upac: Mat	ObservableCollection <YoloItem>	Метод передачи кадра видеопотока для детекции нейросетью, по типу - число
BitmapToImageSource	bitmap: Bitmap	BitmapImage	Метод перевода пикселей графического изображения в изображение для вывода на экран
OpenVideoStream	Отсутствует	Отсутствует	Метод вызывает окно подключения к IP-камере
AddCovsh	Отсутствует	Отсутствует	Метод подсчета распознанных стальковшей
OpenSettings	Отсутствует	Отсутствует	Метод вызывает окно настроек

Таблица 53

Описание полей класса «DetectedAndLog»

Название	Описание
1	2
countErr : Integer	Количество ошибок распознавания, обнаруженных пользователем
isStarted : Boolean	Флаг запуска системы
disbDetect : Boolean	Флаг отключения системы
logTxt : Boolean	Флаг записи в текстовый лог-файл
logDB : Boolean	Флаг записи в БД
tmp : Mat	Массив пикселей кадра
yoloItems : ObservableCollection<YoloItem>	Список обнаруженных объектов нейросетью на кадре
detect_item : ObservableCollection<YoloItem>	Список обнаруженных объектов нейросетью на кадре

Продолжение табл. 53

1	2
detect_itemNumber : ObservableCollection<YoloItem>	Список обнаруженных чисел нейросетью на кадре
_StellCovshList : ObservableCollection<StellCovsh >	Список обнаруженных стальковшей за время работы системой
tokenCancel :	Переменная токена, необходима для прерывания

CancellationTokenSource	асинхронной операции
selectedPath : String	Путь к источнику видеопотока
countNumber : String	Количество распознанных номеров стальной
countStellCovsh : String	Количество распознанных стальной
countEr : String	Количество ошибок при распознавании
dialogService : IDialogService	Обработчик диалоговых окон
inputVideo : BitmapImage	Входной кадр видеопотока
ShowVideoStream DelegateCommand	: Свойство привязки элемента интерфейса к функции обработки видеопотока
ShowSettings : DelegateCommand	Свойство привязки элемента интерфейса к функции открытия окна настроек
SelectedPath : String	Путь к источнику видеопотока
CountStellCovsh : String	Количество обнаруженных стальной – вывод в интерфейс
CountNumber : String	Количество обнаруженных номеров стальной – вывод в интерфейс
CountEr : String	Количество обнаруженных ошибок – вывод в интерфейс
SystemStarted : Boolean	Свойство флага запуска системы
DisbDetect : Boolean	Свойство флага остановки системы
LogTxt : Boolean	Свойство флага логирования в текстовый файл
LogDB : Boolean	Свойство флага записи в БД
StellCovshList ObservableCollection<StellCovsh >	: Список обнаруженных стальной
StartCommand DelegateCommand	: Свойство запуска системы
CheckCovsh : DelegateCommand	Свойство отметки нераспознанного стальной
StopCommand DelegateCommand	: Свойство остановки системы
OpenFileDialogCommand DelegateCommand	: Свойство открытия диалогового окна выбора видеофайла
InputVideo : BitmapImage	Свойство, представляющее входной кадр видеопотока

Описание методов и полей класса «SelectVideoStream» записи представлено в табл. 54,55.

Описание методов «SelectVideoStream»

Название метода	Параметры	Возвращаемое значение	Описание
SetCamera	Отсутствует	Отсутствует	Метод устанавливает адрес камеры соответствующему классу и запускает метод проверки сигнала
SelectVideoStream	IDialogService dialogService	Отсутствует	Конструктор с параметром
CheckSourceAvailability	Отсутствует	Отсутствует	Метод проверки работы IP-камеры

Таблица 55

Описание полей класса «SelectVideoStream»

Название	Описание
ipAddress: String	IP – адрес камеры
status: Boolean	Флаг подключения к камере
dialogService: IDialogService	Обработчик диалоговых окон
AcceptCamera: DelegateCommand	Свойство подтверждения выбора IP-камеры со стороны пользователя
StatusConnect: Boolean	Свойство подключения к камере

Описание полей и методов класса «SelectSettings» в табл. 56, 57.

Таблица 56

Описание методов класса «SelectSettings»

Название метода	Параметры	Возвращаемое значение	Описание
SelectSettings	IDialogService dialogService	Отсутствует	Конструктор с параметром
SaveSettings	Отсутствует	Отсутствует	Метод применения пользовательских настроек
ReturnSetting	Отсутствует	Отсутствует	Метод, отменяющий

s			пользовательские настройки
---	--	--	-------------------------------

Таблица 57

Описание полей класса «SelectSettings»

Название	Описание
1	2
delayS : Integer	Задержка между кадрами видеопотока
heightS : Integer	Разрешение видеопотока (по высоте)
widthS : Integer	Разрешение видеопотока (по ширине)
dialogService IDialogService	: Обработчик диалоговых окон
AcceptSettings DelegateCommand	: Свойство применения пользовательских настроек
CancelSettings DelegateCommand	: Свойство отмены пользовательских настроек
DelayS : Integer	Свойство, устанавливающее задержку между кадрами
HeightS : Integer	Свойство, устанавливающее разрешение видеопотока (по высоте)
WidthS: Integer	Свойство, устанавливающее разрешение видеопотока (по ширине)

1.5.2.3 Проектирование классов пакета «DataLayer»

Пакет, содержащий классы, описывающие данные предметной области используемые в программе, логику по взаимодействию с ними, а именно логирование в текстовый файл и БД с помощью SQL-команд.

1.5.2.3.1 Построение исходной диаграммы классов пакета «DataLayer»

В табл.58 представлено описание классов пакета «DataLayer». Исходная диаграмма классов представлена на рис. 32.

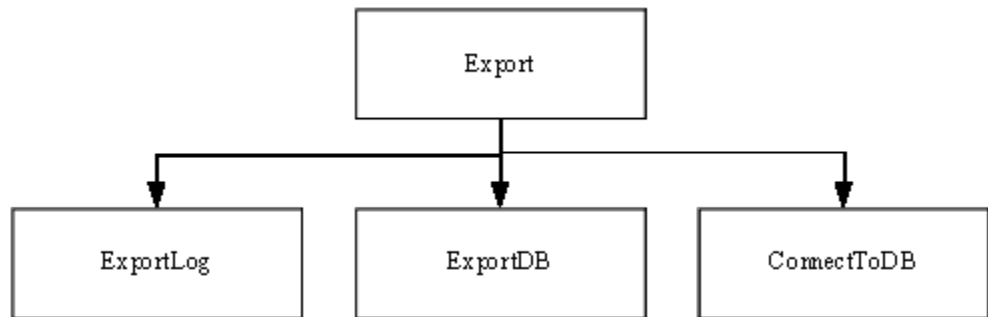


Рис. 32. Исходная диаграмма классов пакета «DataLayer»

Таблица 58

Описание классов пакета«DataLayer»

Класс	Описание
Export	Класс, в котором определен базовый функционал пакета работы с данными
ExportLog	Класс, реализующий алгоритм формирования лог файла на основе данных из пакета PresentationLayer
ExportDB	Класс, реализующий алгоритм записи в БД на основе данных из пакета PresentationLayer
ConnectToDB	Класс, отвечающий за подключение к БД

1.5.2.3.2 Построение диаграммы последовательности взаимодействия объектов классов пакета «DataLayer»

На рис. 33-35 представлены диаграммы последовательностей взаимодействия объектов классов пакета «DataLayer».

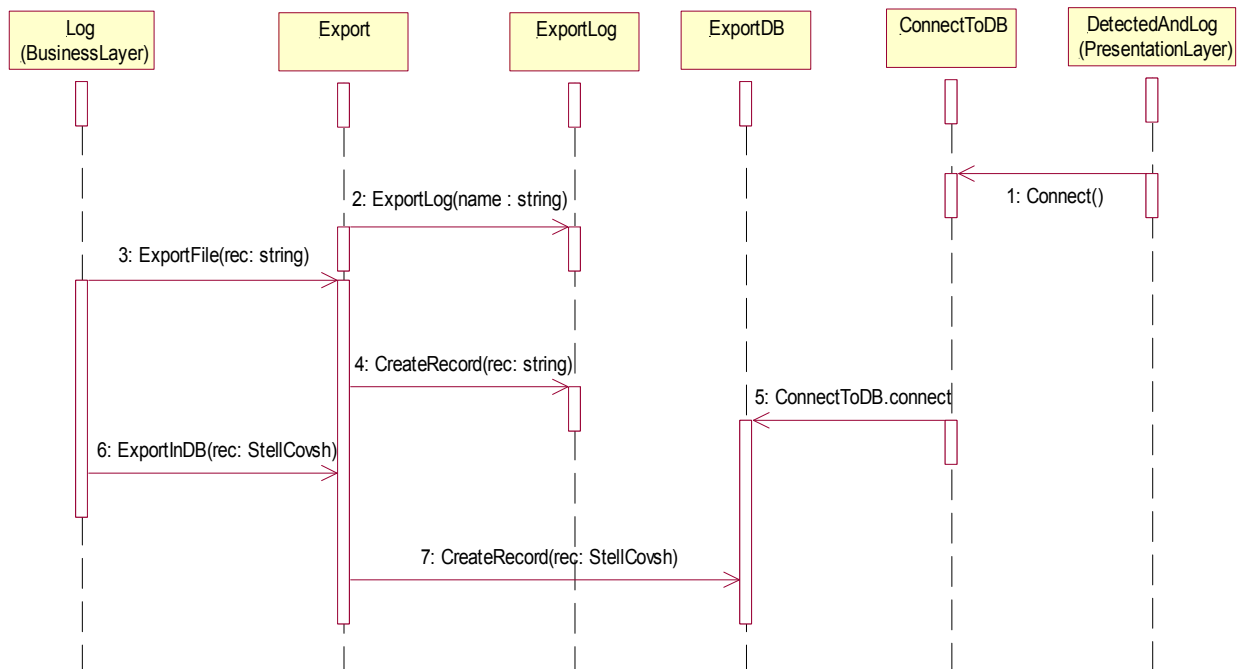


Рис.33. Диаграмма последовательности взаимодействия объектов классов пакета «DataLayer». Нормальный ход событий

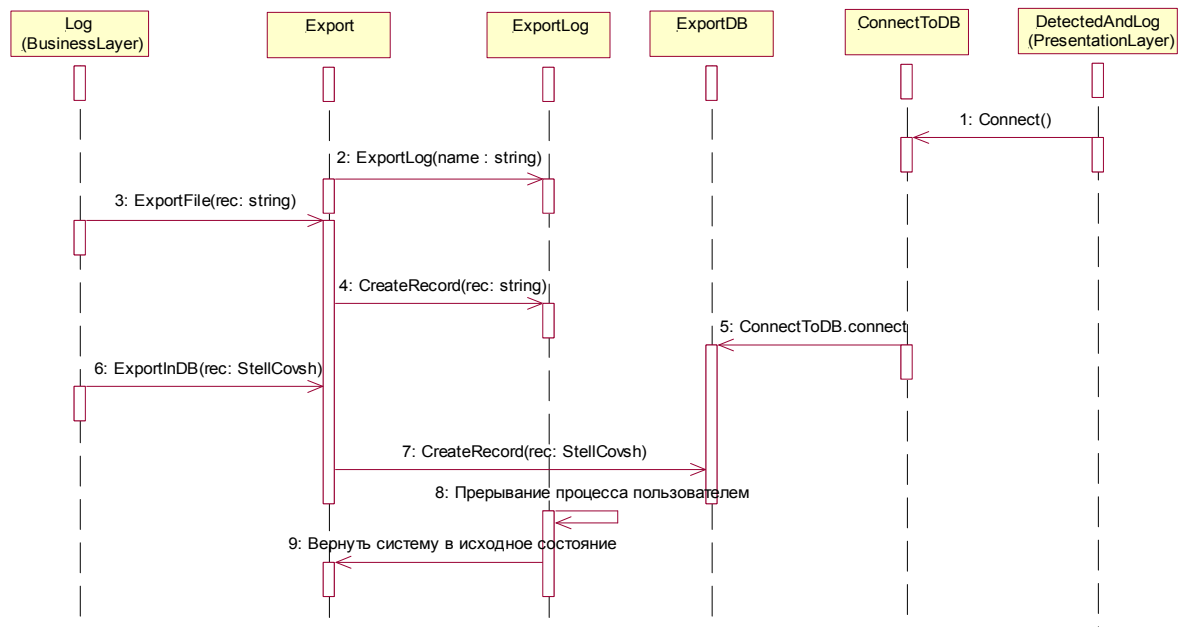


Рис.34. Диаграмма последовательности взаимодействия объектов классов пакета «DataLayer». Прерывание процесса оператором

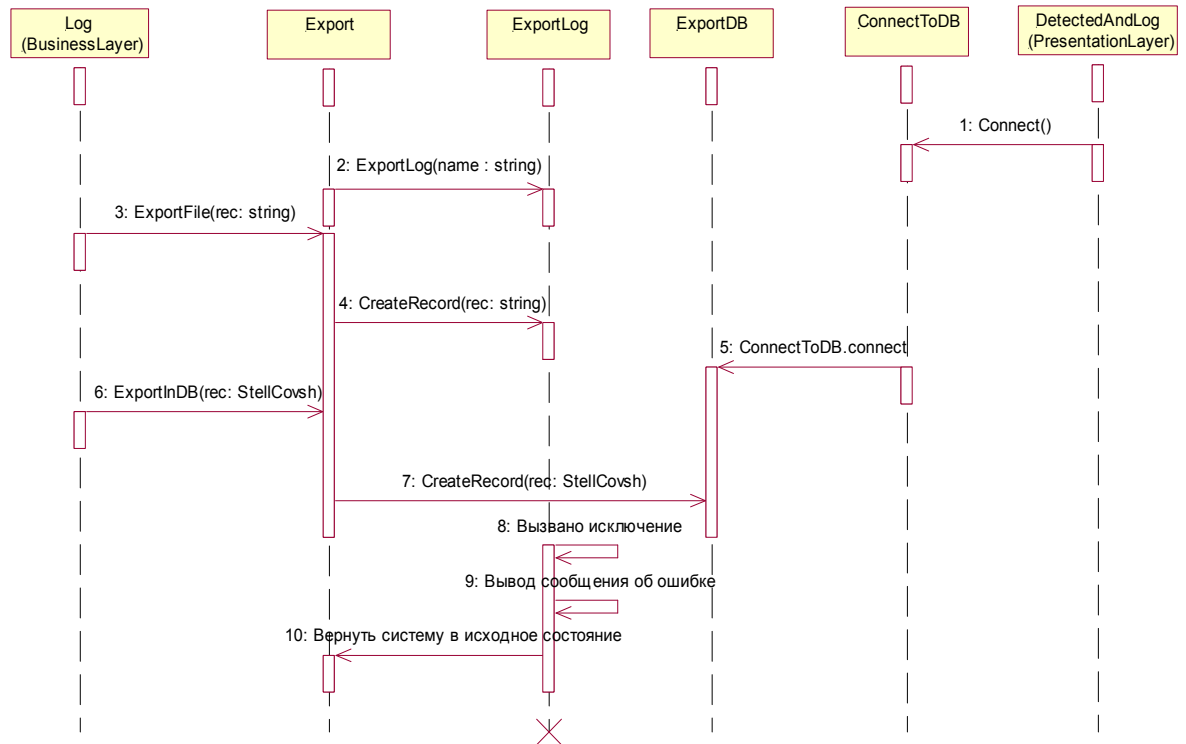


Рис.35. Диаграмма последовательности взаимодействия объектов классов пакета «DataLayer». Прерывание процесса системой

При запуске системы происходит следующее. В первую очередь инициализируется класс ConnectToDB с параметрами подключения к БД, в дальнейшем происходит инициализация объекта класса ExportLog отвечающий за логирования данных в лог файл. С БД, подключенной в самом начале, работает статический класс ExportDB. Обращения к функциям логирования ExportLog, ExportDB происходит из статического класса Export, являющегося своего рода прослойкой между бизнес-логикой и непосредственной записи данных в файл и БД.

1.5.2.3.3 Построение диаграммы кооперации пакета «DataLayer»

Диаграмма кооперации рассматриваемого пакета «DataLayer» представлена на рис.36. На одной диаграмме изображены три возможных случая реализации.

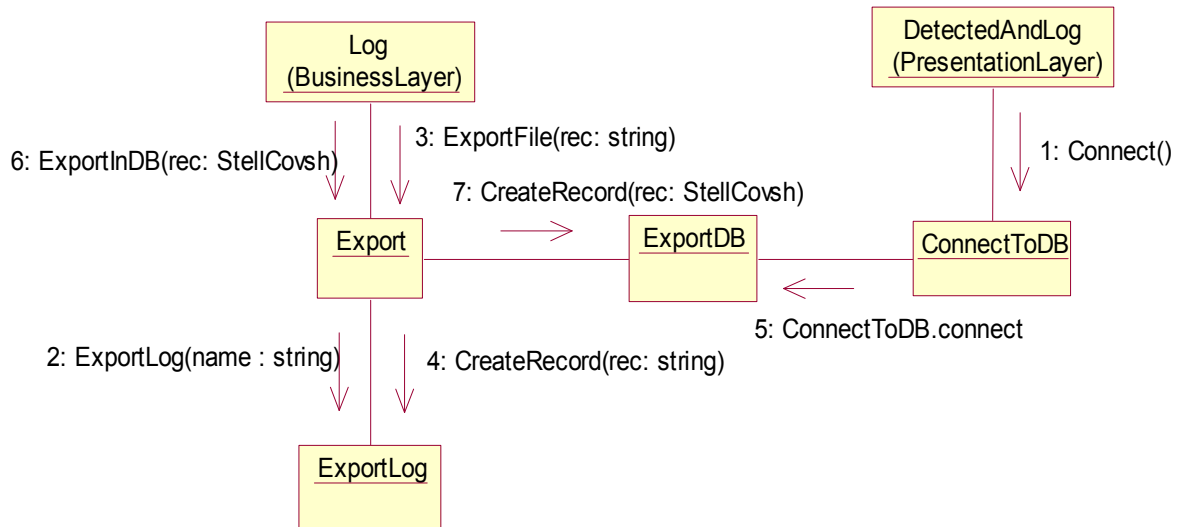


Рис.36. Диаграмма кооперации

5.2.3.4 Построение уточнений диаграммы классов пакета «DataLayer»

Класс **Export** использует функционал классов **ExportLog**, **ExportDB**, **ConnectToDB**. Классы **ExportDB**, **ConnectToDB** являются взаимозависимыми.

Уточненная диаграмма классов пакета представлена на рис.37.

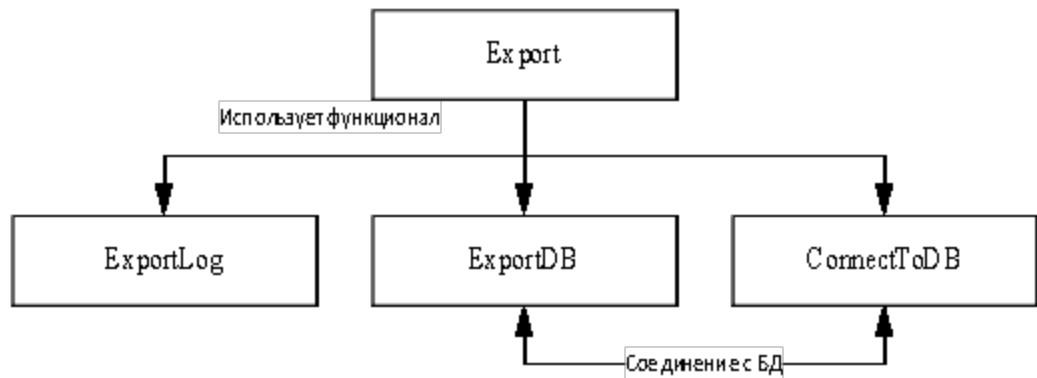


Рис. 37. Уточненная диаграмма классов пакета «DataLayer»

1.5.2.3.5 Построение детальной диаграммы классов пакета «DataLayer»

Детальная диаграмма классов представлена на рис. 38.

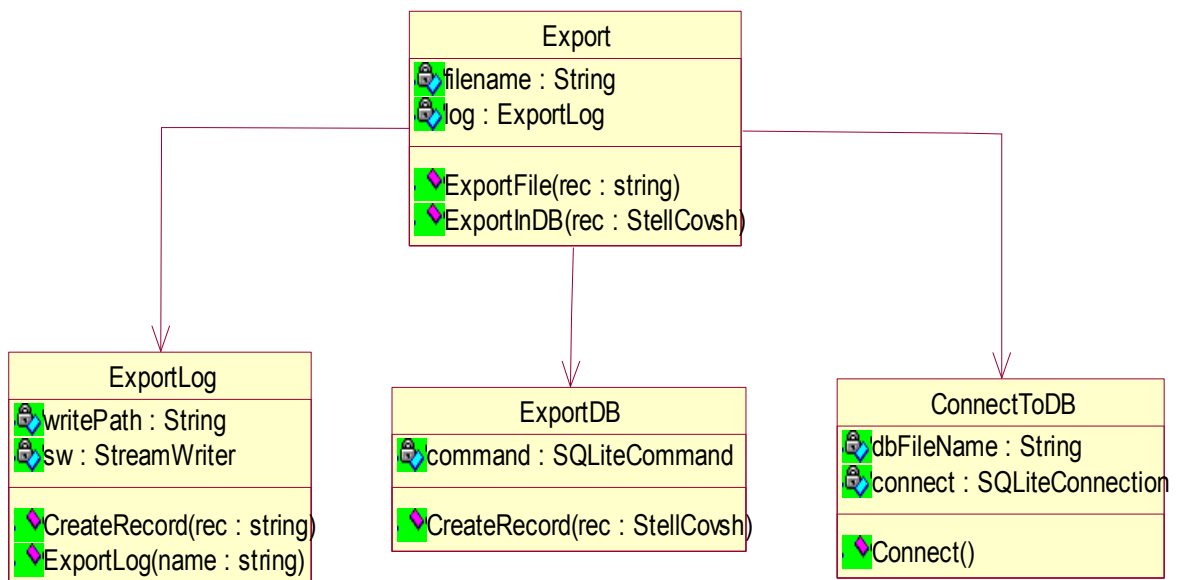


Рис. 38. Детальная диаграмма классов пакета «DataLayer»

Описание методов и полей класса «Export» представлено в табл. 59, 60.

Таблица 59

Описание методов класса «Export»

Название метода	Параметры	Возвращаемое значение	Описание
ExportFile	rec: string	Отсутствует	Метод передачи лога на запись в текстовый файл
ExportInDB	rec: StellCovsh	Отсутствует	Метод передачи лога на запись в БД

Таблица 60

Описание полей класса «Export»

Название	Описание
filename : String	Путь к лог-файлу
log : ExportLog	Экземпляра класса лог-файл

Описание полей и методов класса «ConnectToDB» представлено в табл. 61,62.

Таблица 61

Описание методов класса «ConnectToDB»

Название метода	Параметры	Возвращаемое значение	Описание
Connect	Отсутствует	Отсутствует	Метод подключения к БД

Таблица 62

Описание полей класса «ConnectToDB»

Название	Описание
dbFileName : String	Путь к лог-файлу
connect :	Экземпляр класса, отвечающего за подключение к БД

SQLiteConnectio	
-----------------	--

Описание методов и полей класса «ExportLog» записи представлено в табл. 63,64.

Таблица 63

Описание методов класса «ExportLog»

Название метода	Параметры	Возвращаемое значение	Описание
CreateRecord	rec: string	Отсутствует	Метод записи в лог-файл
ExportLog	name : string	Отсутствует	Конструктор с параметром

Таблица 64

Описание полей класса «ExportLog»

Название	Описание
writePath : String	Путь к лог-файлу
sw : StreamWriter	Экземпляр класса потоковой записи в текстовый файл

Описание методов и полей класса «ExportDB» в табл. 65, 66.

Таблица 65

Описание методов класса «ExportDB»

Название метода	Параметры	Возвращаемое значение	Описание
CreateRecord	rec: StellCovsh	Отсутствует	Создание записи в БД

Таблица 66

Описание полей класса «ExportDB»

Название	Описание
command:	Экземпляр класса, отвечающего за выполнение SQL-команд в БД

SQLiteCommand	
---------------	--

1.5.2.4 Проектирование классов пакета «Functional»

Пакет, содержащий классы, отвечающие за вспомогательные функции на прямую, не относящиеся к предметной области. Основной задачей пакета является организации поддержки паттерна MVVM в приложении, а именно связь кода программы с элементами интерфейса.

1.5.2.4.1 Построение исходной диаграммы классов пакета «Functional»

В табл. 67 представлено описание классов пакета «PresentationLayer». Исходная диаграмма классов представлена на рис. 39.

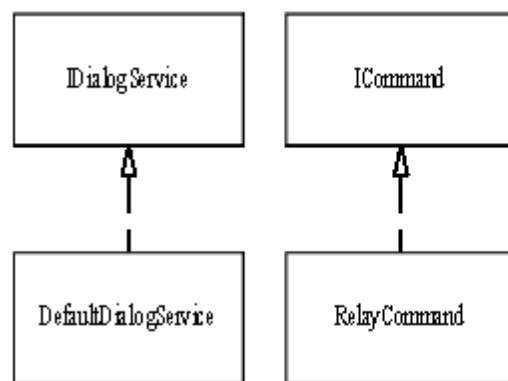


Рис. 39. Исходная диаграмма классов пакета «Functional»

Таблица 67

Описание классов пакета «Functional»

Класс	Описание
IDialogService	Класс, в котором определен базовый функционал пакета работы с данными
DefaultDialogService	Класс, реализующий алгоритм

	формирования лог файла на основе данных из пакета PresentationLayer
RelayCommand	Класс, реализующий алгоритм записи в БД на основе данных из пакета PresentationLayer
ICommand	Класс, отвечающий за подключение к БД

1.5.2.3.2 Построение диаграммы последовательности взаимодействия объектов классов пакета «Functional»

На рис. 40 представлена диаграмма последовательностей взаимодействия объектов классов пакета «Functional».

При загрузке формы главного окна инициализируется экземпляр класса-интерфейса «IDialogService», функционал которого реализован в классе «DefaultDialogService» предназначении данного класса работа с диалоговыми окнами. Класс «RelayCommand» реализует интерфейс системного класса «ICommand» и служит для передачи событий пользовательского интерфейса системе, и обратно.

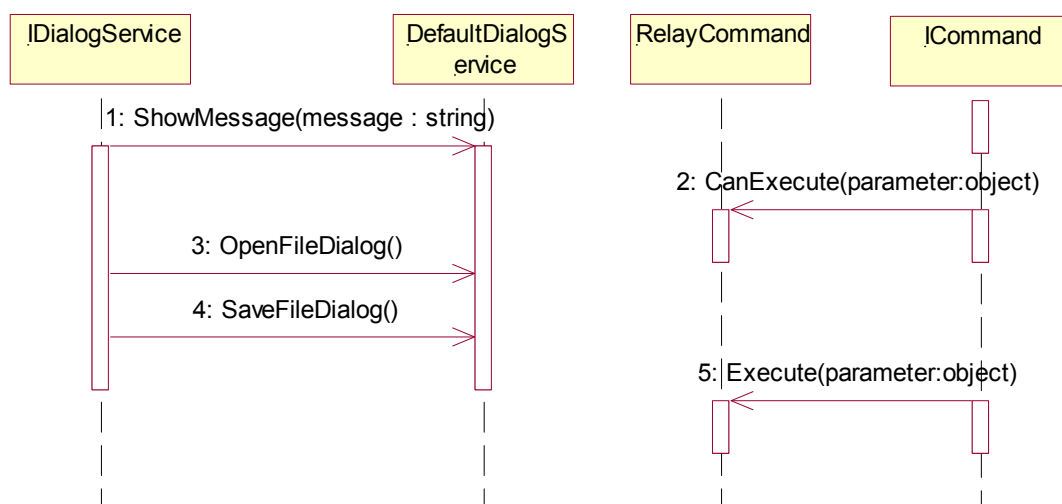


Рис.40. Диаграмма последовательности взаимодействия объектов классов пакета «Functional».

1.5.2.3.3 Построение диаграммы кооперации пакета «Functional»

Диаграмма кооперации рассматриваемого пакета «Functional» представлена на рис.41.

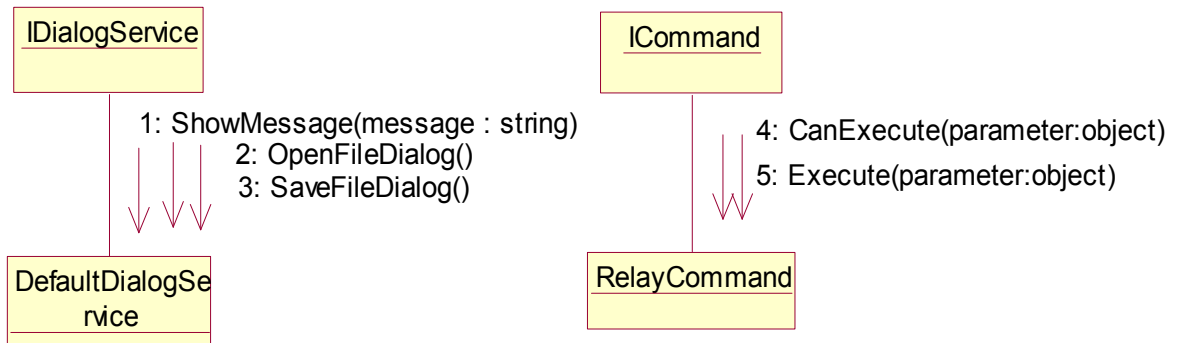


Рис.41. Диаграмма кооперации

1.5.2.3.4 Построение уточнений диаграммы классов пакета «Functional»

Класс DefaultDialogService наследует данные класса IDialogService, класс RelayCommand наследует данные класса ICommand

Уточненная диаграмма классов пакета представлена на рис.42.

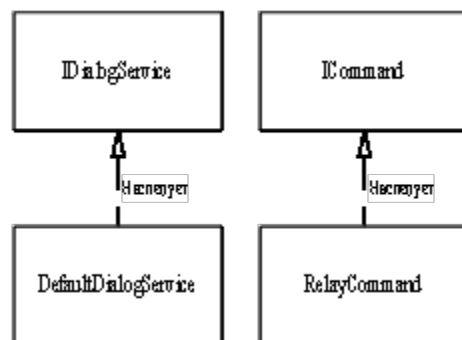


Рис. 42. Уточненная диаграмма классов пакета «Functional»

1.5.2.3.5 Построение детальной диаграммы классов пакета «Functional»

Детальная диаграмма классов представлена на рис. 43.

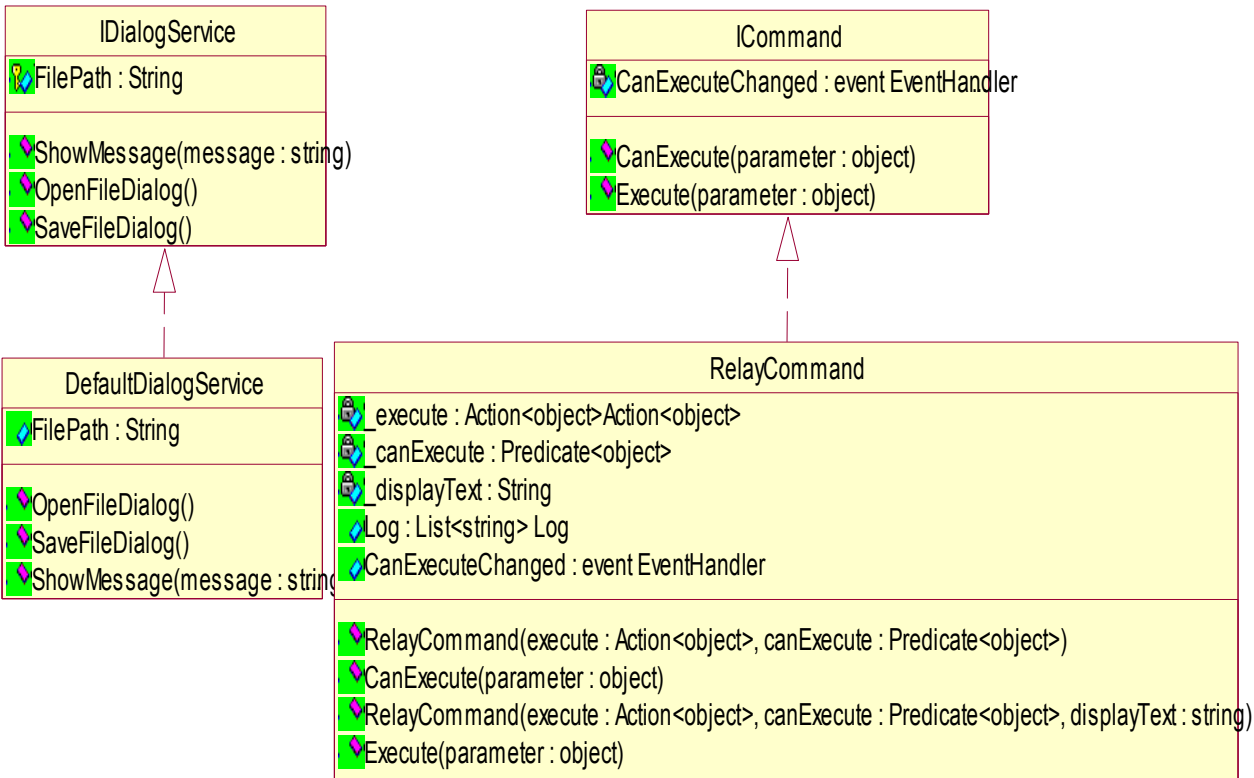


Рис. 43. Детальная диаграмма классов пакета «Functional»

Описание методов и полей класса «IDialogService» представлено в табл. 68,69.

Таблица 68

Описание методов класса «IDialogService»

Название метода	Параметры	Возвращаемое значение	Описание
ShowMessage	message : string	Отсутствует	Метод вывода сообщения
OpenFileDialog	Отсутствует	Отсутствует	Метод работы с файловым диалогом

SaveFileDialog	Отсутствует	Отсутствует	Метод сохранения файла
----------------	-------------	-------------	------------------------

Таблица 69

Описание полей класса «IDialogService»

Название	Описание
FilePath : String	Путь к файлу

Описание методов и полей класса «DefaultDialogService» представлено в табл. 70, 71.

Таблица 70

Описание методов класса «DefaultDialogService»

Название метода	Параметры	Возвращаемое значение	Описание
ShowMessage	message : string	Отсутствует	Метод вывода сообщения
OpenFileDialog	Отсутствует	Отсутствует	Метод работы с файловым диалогом
SaveFileDialog	Отсутствует	Отсутствует	Метод сохранения файла

Таблица 71

Описание полей класса «DefaultDialogService»

Название	Описание
FilePath : String	Путь к файлу

Описание методов и полей класса «ICommand» записи представлено в табл. 72, 73.

Таблица 72

Описание методов класса «ICommand»

Название метода	Параметры	Возвращаемое значение	Описание
CanExecute	parameter:object	bool	Метод определяющий, может ли команда выполняться
Execute	parameter:object	Отсутствует	Метод, выполняющий логику команды

Таблица 73

Описание полей класса «ICommand»

Название	Описание
CanExecuteChanged: event EventHandler	Событие вызывается при изменении условий, указывающий, может ли команда выполняться

Описание методов и полей класса «RelayCommand» в табл. 74, 75.

Таблица 74

Описание методов класса «RelayCommand»

Название метода	Параметры	Возвращаемое значение	Описание
RelayCommand	execute : Action<object>, canExecute : Predicate<object>	Отсутствует	Конструктор с параметрами
RelayCommand	execute : Action<object>, canExecute : Predicate<object>, displayText : string	Отсутствует	Конструктор с параметрами
CanExecute	parameter:object	bool	Метод определяющий, может ли команда выполняться
Execute	parameter:object	Отсутствует	Метод, выполняющий логику команды

Таблица 75

Описание полей класса «RelayCommand»

Название 1	Описание 2
execute: Action<object>Action<object >	Делегат, выполняющий логику команды

Продолжение табл. 75

1	2
canExecute: Predicate<object>	Делегат, указывающий будет ли доступно выполнение команды
_displayText: String	Выводимый текст
Log : List<string> Log	Список сообщений
CanExecuteChanged :event EventHandler	Событие вызывается при изменении условий, указывающий, может ли команда выполняться

При проектировании всех пакетов программы за исключением пакета БД был применен паттерн MVVM для разделения обязанностей. Model-View-ViewModel (MVVM).

1.5.3 Построение диаграмм компонентов

Диаграммы компонентов применяют при проектировании физической структуры разрабатываемого программного обеспечения. Эти диаграммы показывают, как выглядит программное обеспечение на физическом уровне, т. е. из каких частей оно состоит и как эти части связаны между собой [18].

Диаграммы компонентов оперируют понятиями компонент и зависимость. Под компонентами при этом понимают физические заменяемые части программного обеспечения, которые соответствуют некоторому набору интерфейсов и обеспечивают их реализацию. По сути дела, это отдельные файлы различных типов[7].

Проектируемая система содержит одну систему: десктопное приложение, которое работает с БД. Диаграмма компонентов представлена на рис. 44,45.Описание компонентов представлено в табл. 76,77.



Рис. 44. Диаграмма компонентов

Таблица 76

Описание компонентов

Наименование	Назначение	Входные данные	Выходные данные
Программное обеспечение распознавания номера стальной	Позволяет вести обработку поступающего с камеры видеопотока. Передавать информацию об обнаружении стальной и его номера Камера в системе играет ключевую роль по формированию и передачи видеопотока в систему	Видеопоток	Записи в БД и лог-файл о результатах распознавания
База данных (БД)	Предназначена для хранения результатов детектирования	Информация, подлежащая хранению (записи)	Записи о результатах распознавания

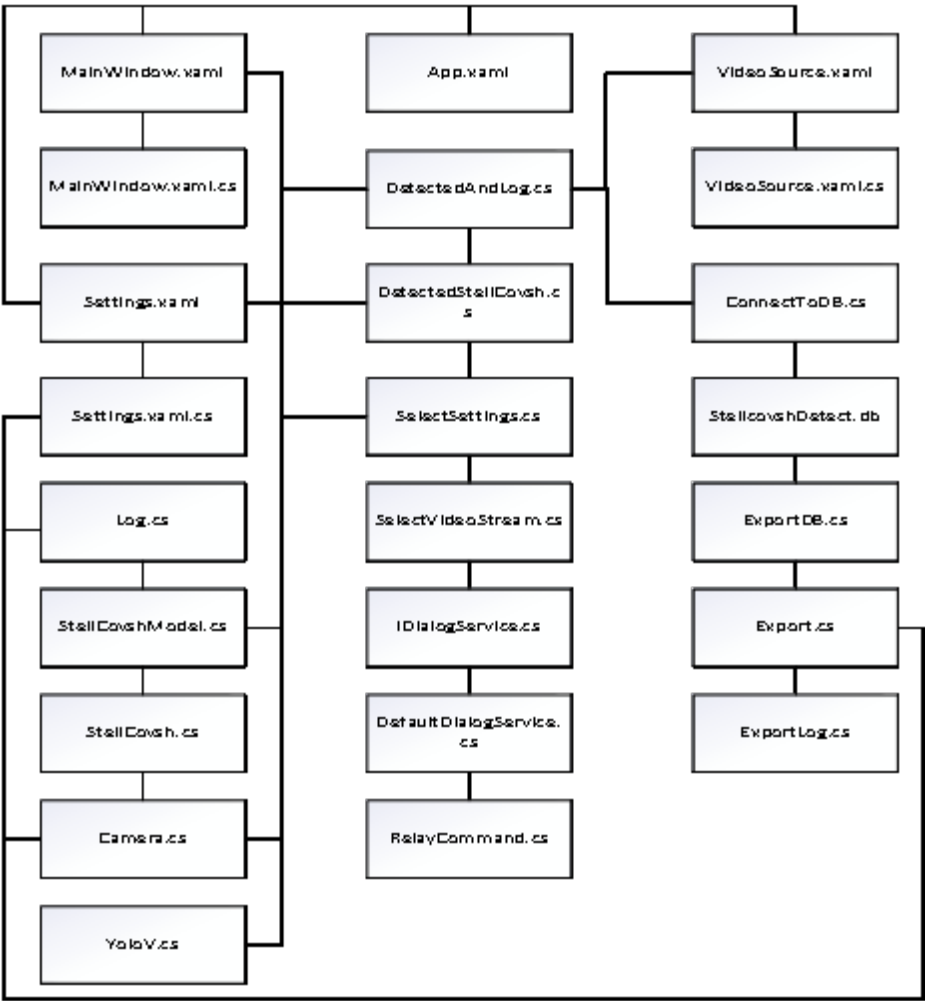


Рис. 45. Диаграмма компонентов

Таблица77

Описание компонентов программы

Наименование	Входные данные	Выходные данные	Назначение
1	2	3	4
DetectedStellCovsh.cs	Отсутствуют	Отсутствуют	Класс, реализует интерфейс INotifyPropertyChanged позволяющий уведомлять систему об изменениях свойств модели. Позволяет связывать данные и интерфейс с логикой программы
DetectedAndLog.cs	Видеопоток, пользовательский интерфейс	Данные распознавания	Класс, наследуемый от базовой модели и определяющий функционал, и логику окна формы детектирования и логирования
SelectVideoStream.cs	Адрес IP-камеры	Адрес IP-камеры	Класс, наследуемый от базовой модели и определяющий функционал, и логику

			окна формы подключения к IP-камере и проверке соединения с ней
SelectSetting s.cs	Пользовательские настройки	Пользовательские настройки	Класс, наследуемый от базовой модели представления и определяющий функционал, и логику окна формы настроек
App.xaml	Параметры запуска приложения	Отсутствуют	Файл конфигурации визуальных ресурсов приложения
ConnectToDB.cs	Отсутствуют	Соединение с БД	Класс с параметрами подключения к БД
Export.cs	Результат распознавания	Данные логирования для БД и лога	Класс логирования, осуществляет форматирование и передачу данных лога разных типов (для БД и файла)
ExportDB.cs	Результат распознавания	Запись в БД	Класс определяющий формат записи в БД
ExportLog.cs	Результат распознавания	Запись в лог файле	Класс определяющий формат записи в лог
MainWindow .xaml	Описание элементов интерфейса	Отсутствуют	Содержит структуру интерфейса оконной формы
MainWindow .xaml.cs	Отсутствуют	Содержимое элементов формы	Содержит класс обработчик пользовательских элементов управления оконной формы
Settings.xaml	Описание элементов интерфейса	Отсутствуют	Содержит структуру интерфейса оконной формы
Settings.xaml .cs	Отсутствуют	Содержимое элементов формы	Содержит класс обработчик пользовательских элементов управления оконной формы

Продолжение табл. 77

1	2	3	4
VideoSource. xaml	Описание элементов интерфейса	Отсутствуют	Содержит структуру интерфейса оконной формы
VideoSource. xaml.cs	Отсутствуют	Содержимое элементов формы	Содержит класс обработчик пользовательских элементов управления оконной формы
Log.cs	Результат распознавания	Передача результата на запись	Класс содержит функционал логирования
StellCovshModel.cs	Результат распознавания	Объект класса StellCovsh	Класс определяет алгоритм фильтрации детектируемых объектов и подготовку к логированию
StellCovsh.cs	Поля класса	Коллекция экземпляров класса StellCovsh	Класс определяет данные объекта предметной области стальковш

Camera.cs	Свойства класса	Свойства класса	Класс определяет данные объекта предметной области камера
YoloV.cs	Конфигурационные файлы	Работа нейросети	Класс содержит параметры нейронной сети
IDialogService.cs	Отсутствуют	Отсутствуют	Класс-интерфейс определяющий функционал диалогового окна
DefaultDialogService.cs	Отсутствуют	Путь к файлу	Класс реализует функционал по работе с диалоговыми окнами
RelayCommand.cs	Выполняемая команды	Выполнение привязанной функции	Класс обеспечивает выполнение логики команды
StellcovshDetect.db	Информация, подлежащая хранению (записи)	Записи о зафиксированных фактах обнаружения стальной	Предназначена для хранения записей о фиксации стальной и их номеров

Подробные спецификации компонентов разработанных компонентов программного обеспечения представлены в табл. 78 - 100.

Таблица 78

Спецификации компонента App.xaml

Функция или свойство	Назначение
InitializeComponent	Инициализирует элементы графического интерфейса

Таблица 79

Спецификации компонента MainWindow.xaml

Функция или свойство	Назначение
InitializeComponent	Инициализирует элементы графического интерфейса формы

Таблица 80

Спецификации компонента MainWindow.cs

Функция или свойство	Назначение
MainWindow	Конструктор класса. Инициализирует новый объект класса MainWindow и вызывает отрисовку элементов интерфейса

Таблица 81

Спецификации компонента Settings.xaml

Функция или свойство	Назначение
InitializeComponent	Инициализует элементы графического интерфейса формы

Таблица 82

Спецификации компонента Settings.xaml.cs

Функция или свойство	Назначение
DBWindow	Конструктор класса. Инициализирует новый объект класса Settings и вызывает отрисовку элементов интерфейса

Таблица 83

Спецификации компонента VideoSource.xaml

Функция или свойство	Назначение
InitializeComponent	Инициализует элементы графического интерфейса формы

Таблица 84

Спецификации компонента VideoSource.xaml.cs

Функция или свойство	Назначение
DBWindow	Конструктор класса. Инициализирует новый объект класса Settings и вызывает отрисовку элементов интерфейса

Таблица 85

Спецификации компонента DetectedStellCovsh.cs

Функция или свойство	Назначение
OnPropertyChanged	Метод изменяет состояние объекта на форме
Set	Метод изменяет значение свойства и меняет его состояние на форме
PropertyChanged	Событие изменения состояния объекта на форме

Таблица 86

Спецификации компонента DetectedAndLog.cs

Функция или свойство	Назначение
1	2
DetectedAndLog	Конструктор с параметрами
StopDetect	Метод останавливает работу системы
OpenFile	Метод вызывает диалоговое окно выбора видеофайла для распознавания
GetFirstFrame	Метод вывода стартового кадра на экран заданного видеопотока
VideoProcessing	Метод воспроизведения видео
ShowOnFormInput	Метод обработки кадра видеопотока и вывода его на экран
Detect	Метод передачи кадра видеопотока для детекции нейросетью, по типу - стальковш
DetectNumber	Метод передачи кадра видеопотока для детекции нейросетью, по типу - число
BitmapToImageSource	Метод перевода пикселей графического изображения в изображение для вывода на экран
OpenVideoStream	Метод вызывает окно подключения к IP-камере
AddCovsh	Метод подсчета распознанных стальковшей
OpenSettings	Метод вызывает окно настроек
countErr : Integer	Количество ошибок распознавания, обнаруженных пользователем
isStarted : Boolean	Флаг запуска системы
disbDetect : Boolean	Флаг отключения системы
logTxt : Boolean	Флаг записи в текстовый лог-файл
logDB : Boolean	Флаг записи в БД
tmp : Mat	Массив пикселей кадра
yoloItems : ObservableCollection<YoloItem>	Список обнаруженных объектов нейросетью на кадре

Продолжение табл. 86

1	2
yoloItemsNumber : ObservableCollection<YoloItem>	Список обнаруженных чисел нейросетью на кадре
detect_item : ObservableCollection<YoloItem>	Список обнаруженных объектов нейросетью на кадре
detect_itemNumber : ObservableCollection<YoloItem>	Список обнаруженных чисел нейросетью на кадре
_StellCovshList : ObservableCollection<StellCovsh>	Список обнаруженных стальных за время работы системой
tokenCancel : CancellationTokenSource	Переменная токена, необходима для прерывания асинхронной операции
selectedPath : String	Путь к источнику видеопотока
countNumber : String	Количество распознанных номеров стальных
countStellCovsh : String	Количество распознанных стальных
countEr : String	Количество ошибок при распознавании
dialogService : IDialogService	Обработчик диалоговых окон
inputVideo : BitmapImage	Входной кадр видеопотока
ShowVideoStream : DelegateCommand	Свойство привязки элемента интерфейса к функции обработки видеопотока
ShowSettings : DelegateCommand	Свойство привязки элемента интерфейса к функции открытия окна настроек
SelectedPath : String	Путь к источнику видеопотока
CountStellCovsh : String	Количество обнаруженных стальных – вывод в интерфейс
CountNumber : String	Количество обнаруженных номеров стальных – вывод в интерфейс
CountEr : String	Количество обнаруженных ошибок – вывод в интерфейс
SystemStarted : Boolean	Свойство флага запуска системы
DisbDetect : Boolean	Свойство флага остановки системы
LogTxt : Boolean	Свойство флага логирования в файл
LogDB : Boolean	Свойство флага записи в БД
StellCovshList : ObservableCollection<StellCovsh>	Список обнаруженных стальных
StartCommand : DelegateCommand	Свойство запуска системы
CheckCovsh : DelegateCommand	Свойство отметки нераспознанного стального
StopCommand : DelegateCommand	Свойство остановки системы
OpenFileDialogCommand : DelegateCommand	Свойство открытия диалогового окна выбора видеофайла
InputVideo : BitmapImage	Свойство, представляющее входной кадр видеопотока

Таблица 87

Спецификации компонента SelectVideoStream.cs

Функция или свойство	Назначение
SetCamera	Метод устанавливает адрес камеры соответствующему классу и запускает метод проверки сигнала
SelectVideoStream	Конструктор с параметром
CheckSourceAvailability	Метод проверки работы IP-камеры
ipAdress: String	IP – адрес камеры
status: Boolean	Флаг подключения к камере
dialogService: IDialogService	Обработчик диалоговых окон
AcceptCamera: DelegateCommand	Свойство подтверждения выбора IP-камеры со стороны пользователя
StatusConnect: Boolean	Свойство подключения к камере

Таблица 88

Спецификации компонента SelectSettings.cs

Функция или свойство	Назначение
SelectSettings	Конструктор с параметром
SaveSettings	Метод применения пользовательских настроек
ReturnSettings	Метод, отменяющий пользовательские настройки
delayS : Integer	Задержка между кадрами видеопотока
heightS : Integer	Разрешение видеопотока (по высоте)
widthS : Integer	Разрешение видеопотока (по ширине)
dialogService : IDialogService	Обработчик диалоговых окон
AcceptSettings : DelegateCommand	Свойство применения пользовательских настроек
CancelSettings : DelegateCommand	Свойство отмены пользовательских настроек
DelayS : Integer	Свойство, устанавливающее задержку между кадрами
HeightS : Integer	Свойство, устанавливающее разрешение видеопотока (по высоте)
WidthS: Integer	Свойство, устанавливающее разрешение видеопотока (по ширине)

Таблица 89

Спецификации компонента SelectSettings.cs

Функция или свойство	Назначение
SelectSettings	Конструктор с параметром
SaveSettings	Метод применения пользовательских настроек
ReturnSettings	Метод, отменяющий пользовательские настройки
delayS : Integer	Задержка между кадрами видеопотока
heightS : Integer	Разрешение видеопотока (по высоте)
widthS : Integer	Разрешение видеопотока (по ширине)
dialogService : IDialogService	Обработчик диалоговых окон
AcceptSettings : DelegateCommand	Свойство применения пользовательских настроек
CancelSettings : DelegateCommand	Свойство отмены пользовательских настроек
DelayS : Integer	Свойство, устанавливающее задержку между кадрами
HeightS : Integer	Свойство, устанавливающее разрешение видеопотока (по высоте)
WidthS: Integer	Свойство, устанавливающее разрешение видеопотока (по ширине)

Таблица 90

Спецификации компонента StellCovshModel.cs

Функция или свойство	Назначение
Logging	Метод определения, фильтрация и передачи данных для записи обнаруженных нейросетью объектов
num: String	Номер стальной ковша
numLast: String	Номер предыдущего стальной ковша

Таблица 91

Спецификации компонента YoloV.cs

Функция или свойство	Назначение
yolo_LoadModel	Метод загрузки нейросети детектирующей стальковши
yoloWrapper : YoloWrapper	Экземпляр класса нейросети детектирующей стальковши

Таблица 92

Спецификации компонента StellCovsh.cs

Функция или свойство	Назначение
StellCovsh	Конструктор с параметрами
StellCovshList: ObservableCollection<StellCovsh>	Список детектированных стальковшей
Point : String	Участок детектирования
Time : String	Время детектирования
Number : String	Номер стальковша
Camera : String	Адрес камеры
X : Double	Координата стальковша
Y : Double	Координата стальковша
Width : Double	Ширина стальковша
Height : Double	Высота стальковша

Таблица 93

Спецификации компонента Log.cs

Функция или свойство	Назначение
Record	Добавление стальковша в список
FormatToDB	Метод, конвертирующий запись в формат пригодный для записи в БД
FormatToLog	Метод, конвертирующий запись в формат

	пригодный для записи в лог файл
count: Integer	Номер записи в файле

Таблица 94

Спецификации компонента Export.cs

Функция или свойство	Назначение
ExportFile	Метод передачи лога на запись в текстовый файл
ExportInDB	Метод передачи лога на запись в БД
filename : String	Путь к лог-файлу
log : ExportLog	Экземпляра класса лог-файл

Таблица 95

Спецификации компонента ConnectToDB.cs

Функция или свойство	Назначение
Connect	Метод подключения к БД
dbFileName : String	Путь к лог-файлу
connect : SQLiteConnectio	Экземпляр класса, отвечающего за подключение к БД

Таблица 96

Спецификации компонента ExportLog.cs

Функция или свойство	Назначение
CreateRecord	Метод записи в лог-файл
ExportLog	Конструктор с параметром
writePath : String	Путь к лог-файлу
sw : StreamWriter	Экземпляр класса потоковой записи в текстовый файл

Таблица 97

Спецификации компонента ExportDB.cs

Функция или свойство	Назначение
CreateRecord	Метод записи в БД

command : SQLiteCommand	Экземпляр класса, отвечающего за выполнение SQL-команд в БД
-------------------------	---

Таблица 98

Спецификации компонента IDialogService.cs

Функция или свойство	Назначение
ShowMessage	Метод вывода сообщения
OpenFileDialog	Метод работы с файловым диалогом
SaveFileDialog	Метод сохранения файла
FilePath : String	Путь к файлу

Таблица 99

Спецификации компонента DefaultDialogService.cs

Функция или свойство	Назначение
ShowMessage	Метод вывода сообщения
OpenFileDialog	Метод работы с файловым диалогом
SaveFileDialog	Метод сохранения файла
FilePath : String	Путь к файлу

Таблица 100

Спецификации компонента RelayCommand.cs

Функция или свойство	Назначение
1	2
RelayCommand	Конструктор с параметрами
RelayCommand	Конструктор с параметрами

Продолжение табл. 100

1	2
CanExecute	Метод определяющий, может ли команда выполняться
Execute	Метод, выполняющий логику команды
execute : Action<object>Action<object>	Делегат, выполняющий логику команды
canExecute : Predicate<object>	Делегат, указывающий будет ли доступно выполнение команды
_displayText : String	Выводимый текст
Log : List<string> Log	Список сообщений

CanExecuteChanged :event EventHandler	Событие вызывается при изменении условий, указывающий, может ли команда выполняться
---------------------------------------	---

1.5.4 Построение диаграмм размещения

При физическом проектировании распределенных программных систем необходимо определить наиболее оптимальный вариант размещения программных компонентов на реальном оборудовании в локальной или глобальной сетях. Для этого используют специальную модель UML - диаграмму размещения[7].

Диаграмма размещения отражает физические взаимосвязи между программными и аппаратными компонентами системы. Каждой части аппаратных средств системы, например, компьютеру или датчику, на диаграмме размещения соответствует узел[7].

Диаграмма размещения проектируемой системы представлена на рис. 46.

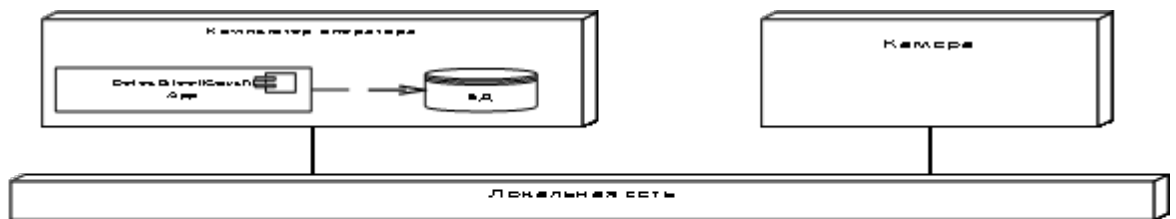


Рис.46. Диаграмма размещения

Компьютер оператора и Камерасвязаны с помощью локальной сети, логированиерезультатов работы ПО,ведется в память компьютера оператора на локальную БД SQLiteи текстовый лог-файл.

1.6. Проектирование интерфейса пользователя

Пользовательский интерфейс (в случае его наличия) проектируются в самом начале создания ПО. Именно он дает наиболее наглядное представление о будущей системе заказчику и позволяет (в случае необходимости) внести правки в проект на самых ранних этапах. Подробно взаимодействие с интерфейсом пользователя рассмотрено в прил.4.

1.6.1. Построение графа диалога

Граф диалога — это граф, каждой вершине которого сопоставлено определенное состояние диалога, характеризующееся набором доступных пользователю действий.

Дуги, исходящие из вершин, показывают возможные изменения состояний при выполнении пользователем определенных действий.

Под диалогом понимают регламентированный обмен информацией между человеком и компьютером, (программным обеспечением) осуществляемый в реальном масштабе времени и направленный на совместное решение конкретных задач: обмен информацией между пользователем и компьютером и координация их действий [17].

Каждый диалог состоит из отдельных процессов ввода-вывода, которые физически обеспечивают связь между пользователем программного обеспечения и компьютером.

Обмен информацией между пользователем и компьютером осуществляется с помощью передачи сообщений и управляющих сигналов.

Сообщение— это определенная порция информации в диалоговом обмене человека и компьютера.

Различают:

- входные сообщения, которые генерируются человеком с помощью средств ввода;
- выходные сообщения, которые генерируются компьютером в виде текстов, звуковых сигналов и/или изображений и выводятся пользователю на экран монитора или другие устройства вывода информации.

Пользователь в основном генерирует сообщения следующих типов: запрос информации, запрос помощи, запрос операции или функции, ввод или изменение информации, выбор поля кадра и т. д.

В ответ он получает: подсказки или справки; информационные сообщения, не требующие ответа; приказы, требующие действий; сообщения об ошибках, нуждающиеся в ответных действиях; изменение формата кадра и т. д.

Так как работа с программой имеет четкую последовательность действий, то тип диалога пользователя с программой является «Управляемый программой».

Граф диалога программы представлен на рис. 47.

После запуска программы оператор попадает на основное окно программы, в котором он может либо загрузить видеофайл для обработки, либо, получить видео в режиме реального времени с установленной IP-камеры. Оператор имеет возможность настройки параметров программы, запускать и останавливать обнаружение.

Возможность выбора записи работы в программы, как в базу данных и лог-файл одновременно, так и по отдельности. Информация о детектированных объектах выводится на экран оператору вместе со статистикой.

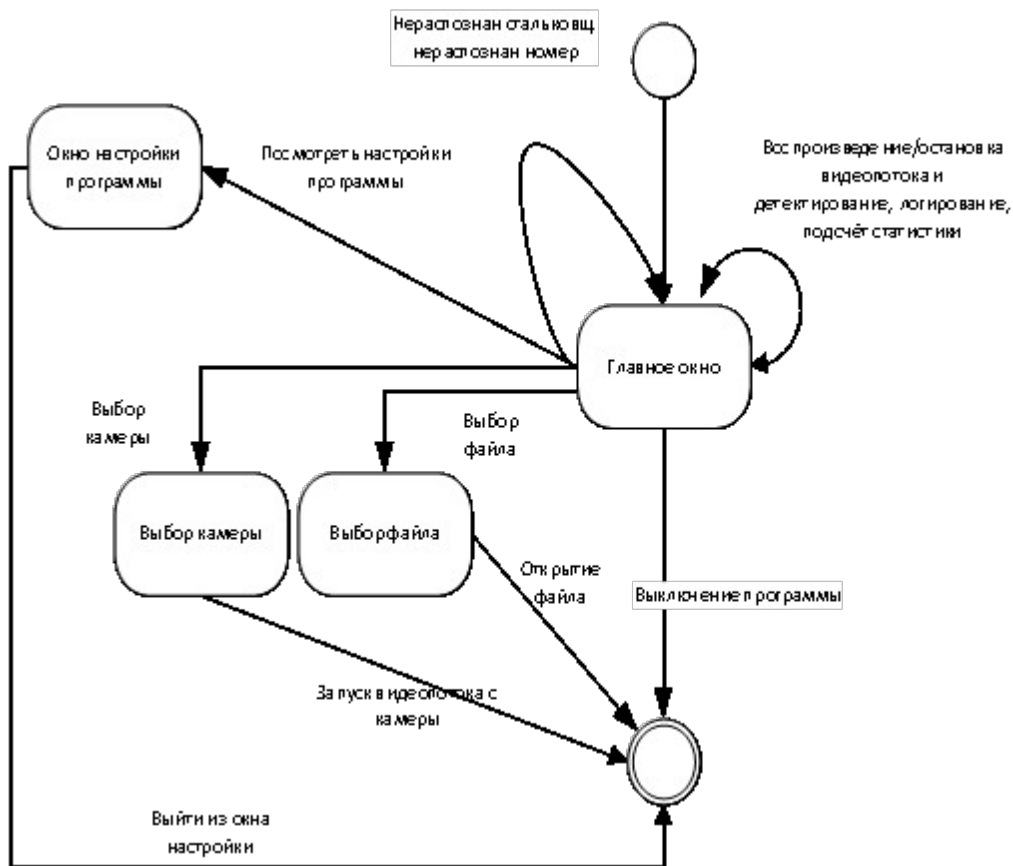


Рис.47. Граф диалога

1.6.2 Разработка форм ввода-вывода информации

Пользовательские интерфейсы большинства современных программ строятся по технологии WIMP: W - Windows (окна), I - Icons (пиктограммы), M - Mouse (мышь), P - Pop-up(всплывающие или выпадающие меню). Основными элементами графических интерфейсов являются [17]:

- окна;
- пиктограммы;
- компоненты ввода-вывода;

- МЫШЬ.

К основным компонентам интерфейса разрабатываемой программы можно отнести:

1. Главное окно программы (рис. 48).

Окно приложения Windows с примененной темой Metro, содержит: системную панель сверху с названием программы и кнопками манипулирования размерами окна, рабочую область, ограниченную рамкой (в ней будет транслироваться видеопоток), зону распознанных данных (таблица с результатами детектирования), зону статистики и управления ошибками, зону основных функций. Ниже будут описаны все рабочие зоны в том же порядке.

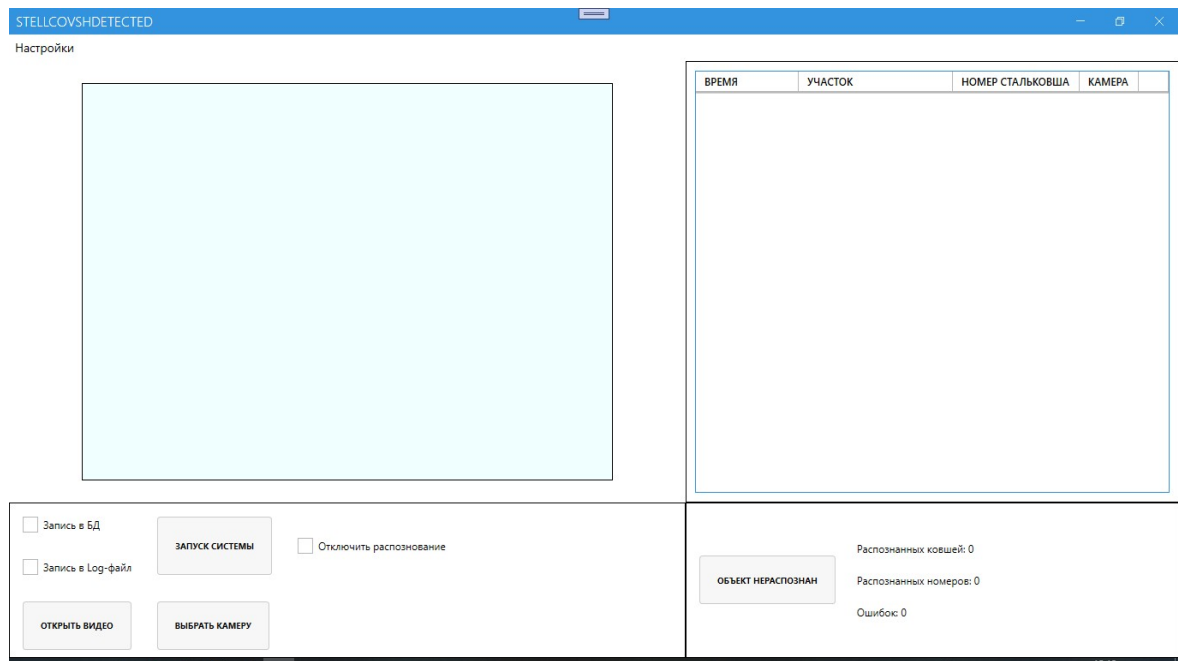


Рис. 48. Главное окно

2. Зона детектирования (рабочая зона) (рис.49). Представляет из область, на которой содержится компонент отрисовки изображения выровненный

относительно центра рабочей зоны. Данный элемент предназначен для предоставления оператору визуальной информации (демонстрация обработанного видеопотока) о работе системе распознавания.

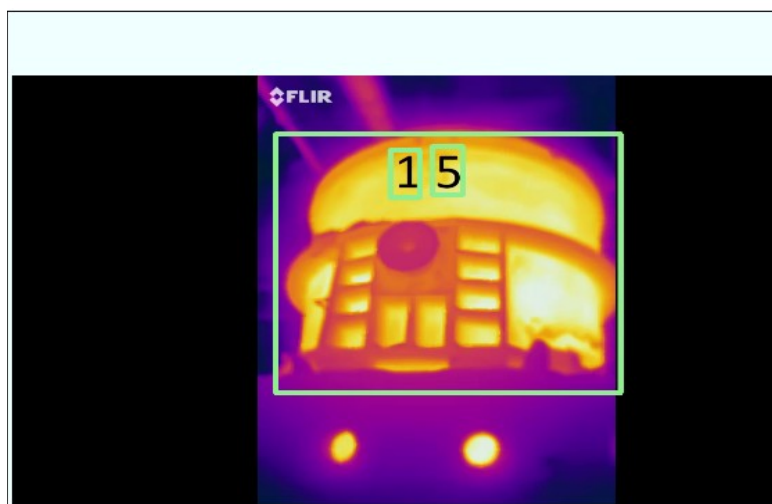


Рис. 49. Зона детектирования

3. Зона распознанных данных, содержит в себе элемент интерфейса таблица с заданными наименованиями полей. Функционал данной зоны заключается в отображении результатов детектирования в табличном представлении (рис.50).

я	УЧАСТОК	НОМЕР СТАЛЬКОВША	КАМЕРА

Рис. 50. Зона распознанных данных

4. Зона статистики и управления ошибками. Содержит следующие элементы интерфейса: 3 надписи и кнопку. Предназначение данной зоны отображать статистику по работе программы и предоставить оператору возможность исправить ошибку программы, а именно в случае нераспознавания стальной ковша программой автоматически, оператор может добавить запись в статистику детектировании о найденном объекте (рис. 51).

ОБЪЕКТ НЕРАСПОЗНАН	Распознанных ковшей: 0
	Распознанных номеров: 0
	Ошибок: 0

15.12

Рис. 51. Зона статистики и управления ошибками

5. Зона основных функций. Содержит следующие элементы интерфейса: 3 чекбокса и 3 кнопки. Данная зона призвана обеспечить доступ оператора к основному функционалу программы, а именно: предоставить возможность запуска и остановки системы, отключения автоматического детектирования, настроить логирование и запись результатов в базу данных (рис.52).

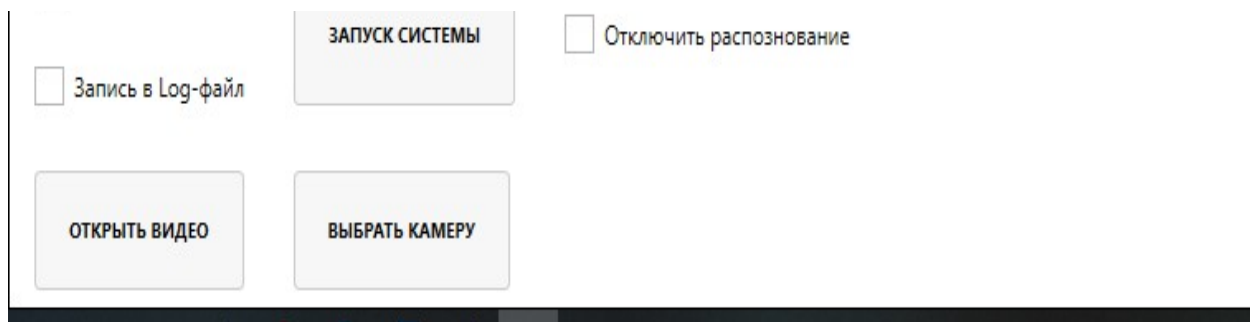


Рис. 52. Зона основных функций

1.7 Выбор стратегии тестирования, разработка тестов, программа и методика испытаний

1.7.1 Объект и цель испытаний

Объектом испытаний является система распознавания номера стальной, компании ПАО «Северсталь».

Цели испытаний:

- поиск максимального количества ошибок;
- проверка правильности работы отдельных частей программы;
- проверка соответствия, разрабатываемого ПО требованиям, заявленным в техническом задании.

1.7.2. Требования к информационному, аппаратно-программному обеспечению

1.7.2.1. Требования к функциональным характеристикам

К составу функций предъявляются следующие требования:

1. Запись видеопотока камерой тепловизора, установленной в точке прохождения крана со стальной шпилькой.
2. Прием видеопотока с IP-камеры разрабатываемой системой.
3. Вывод входящего видеопотока на экран программы оператору.
 - частота кадров не менее 15 кадр. /сек.;
 - размер окна видеопотока должен позволять осуществлять контроль оператору.
4. Обработка видеопотока, распознавание стальной шпильки и его номера.
 - обработанный видеопоток должен выводиться с частотой кадров не менее 12 кадр. /сек.;
 - точность распознавания должна быть не менее 93%.
5. Выделение контуров найденных объектов.
 - контур стальной шпильки или его номера стальной шпильки должны выделяться разным цветом;
 - цвета выделения не должны сливаться с контуром;
 - контур выделения должен примерно сочетаться с контуром реального объекта.
6. Логирование событий.

- информация для логирования: номер стальной, место съемки, модель камеры, дата и время;
- необходимо логирование в БД и лог файл на компьютере оператора

1.7.2.2. Требования к надежности продукта

Разрабатываемая система должна удовлетворять следующим требованиям относительно надежности:

- бесперебойное функционирование при любых входных данных;
- оперативное сохранение информации о результатах работы программы;
- обеспечение проверки вводимых пользователем параметров;
- логирование данных в лог-файл и БД;
- корректный вывод данных и результатов на экран, корректное отображение видеозаписи;
- система должна иметь защиту от некорректных действий оператора и ошибочных исходных данных.

1.7.2.3. Требования к составу и параметрам технических средств

Для корректной работы системы необходимы персональные компьютеры, удовлетворяющие следующим минимальным требованиям:

- процессор с 4 потоками и тактовой частотой 3,2 ГГц или большей;
- не менее 8ГБ оперативной памяти;
- видеокарта – не менее GTX 1660Tiс объемом видеопамати 6 Гб;
- 2 Гб на жестком диске;
- графическое устройство с поддержкой DirectX 9или более поздней

версии;

- устройство ввода (мышь, клавиатура);
- устройства вывода (монитор);
- подключение к локальной сети предприятия (в случае необходимости доступа к камерам и интеграции с базами данных).

1.7.2.4. Требования к программной документации

Программная документация должна включать в себя следующие элементы:

1. Расчетно-пояснительная записка.
2. Техническое задание.
3. Схемы и/или диаграммы.
4. Текст программы.
5. Спецификации.
6. Руководство пользователя.

Документация оформляется на листах формата А4 по действующим стандартам на создание документации к программному обеспечению (ЕСПД).

1.7.3. Состав, порядок и методы испытаний

Для проверки работоспособности разработанного программного обеспечения предполагается следующий состав мероприятий по тестированию:

- тестирование разработанных модулей;
- тестирование программы на соответствие функциональных требований, изложенных в прил. 1;

- тестирование надежности программы на соответствие требований к надежности, изложенных в прил. 1;
- проверка программной документации на соответствие требований ЕСПД.

Тестирование проводилось на персональном компьютере со следующими характеристиками:

- 1.Объем оперативной памяти 8 Гб.
- 2.Монитор с разрешением 2560x1440.
- 3.Операционная система Windows10 Домашняя.
- 4.Твердотельный накопитель объемом 256 ГБ.
- 5.Процессор AMD Ryzen 5 2600 (4 ядер, 8 потоков, тактовая 3.4 ГГц).
- 6.Видеокарта Gigabyte GeForce GTX 1660 с поддержкой DirectX 12.

Для тестирования программной документации использовался метод ручного контроля. Для тестирования программного обеспечения использовалось два метода: функциональное тестирование и системное тестирование.

Для системного тестирования применялся метод ручного контроля. Ручной контроль обычно используется на ранних этапах разработки, так как с его помощью можно находить от 30 до 70 % ошибок логического проектирования и кодирования. Исходными данными для таких проверок являются: техническое задание, спецификации, структурная и функциональная схемы программного продукта, схемы отдельных компонентов.

Из методов ручного контроля был выбран метод проверки за столом. Этот метод не требует наличия группы специалистов. Проверка исходного текста проводится одним человеком, который читает текст программы и проверяет его на наличие возможных ошибок по списку наиболее часто встречающихся

ошибок.

Для тестирования модулей был выбран один из методов функционального тестирования – метод граничных значений. Исходными данными для этого метода являются: программные модули, спецификации на компоненты, правильные и неправильные данные для модулей.

1.7.4. Результаты проведения испытаний

В результате проверки программной документации методом ручного контроля были исправлены найденные грамматические и пунктуационные ошибки, было проверено соответствие документации требованиям ЕСПД. Также были проверены построенные схемы и диаграммы на соответствие ГОСТ.

Результаты тестирования компонентов ПО представлены в табл. 101.

Таблица 101

Тестирование

Дата	Тестируемый модуль	Кто проводил тестирование	Способ тестирования	Результат теста
1	2	3	4	5
20.05.21	DetectedStellCovsh.cs	Разработчик	Функциональное тестирование	Успех
20.05.21	DetectedAndLog.cs	Разработчик	Функциональное тестирование	Неудача, некорректное взаимодействие пользовательского интерфейса с логикой программы

20.05.21	DetectedAndLog.cs	Разработчик	Функциональное тестирование	Неудача, некорректный вывод видеопотока
20.05.21	DetectedAndLog.cs	Разработчик	Функциональное тестирование	Неудача, некорректное взаимодействие с перосетью
20.05.21	DetectedAndLog.cs	Разработчик	Функциональное тестирование	Успех
20.05.21	SelectVideoStream.cs	Разработчик	Функциональное тестирование	Успех
20.05.21	SelectSettings.cs	Разработчик	Функциональное тестирование	Успех
20.05.21	App.xaml	Разработчик	Функциональное тестирование	Успех

Продолжение табл. 101

1	2	3	4	5
20.05.21	ConnectToDB.cs	Разработчик	Функциональное тестирование	Успех
20.05.21	Export.cs	Разработчик	Функциональное тестирование	Неудача, ошибка некорректные данные для логирования
20.05.21	Export.cs	Разработчик	Функциональное тестирование	Успех
20.05.21	ExportDB.cs	Разработчик	Функциональное тестирование	Неудача, ошибка при записи в БД
20.05.21	ExportDB.cs	Разработчик	Функциональное тестирование	Успех
20.05.21	ExportLog.cs	Разработчик	Функциональное тестирование	Неудача, ошибка при именовании

				файла
20.05.21	ExportLog.cs	Разработчик	Функциональное тестирование	Успех
20.05.21	MainWindow.xaml	Разработчик	Функциональное тестирование	Успех
20.05.21	MainWindow.xaml.cs	Разработчик	Функциональное тестирование	Успех
21.05.21	Settings.xaml	Разработчик	Функциональное тестирование	Успех
21.05.21	Settings.xaml.cs	Разработчик	Функциональное тестирование	Успех
21.05.21	VideoSource.xaml	Разработчик	Функциональное тестирование	Успех
21.05.21	VideoSource.xaml.cs	Разработчик	Функциональное тестирование	Успех
21.05.21	Log.cs	Разработчик	Функциональное тестирование	Неудача, использовались некорректные типы данных
21.05.21	Log.cs	Разработчик	Функциональное тестирование	Успех
22.05.21	StellCovshModel.cs	Разработчик	Функциональное тестирование	Неудача, множественное распознавания одного и того же стальковша
22.05.21	StellCovshModel.cs	Разработчик	Функциональное тестирование	Успех
22.05.21	StellCovsh.cs	Разработчик	Функциональное тестирование	Успех

Продолжение табл. 101

1	2	3	4	5
22.05.21	Camera.cs	Разработчик	Функциональное тестирование	Успех
22.05.21	YoloV.cs	Разработчик	Функциональное тестирование	Неудача, нейросеть не

				использует ресурсы графического процессора
22.05.21	YoloV.cs	Разработчик	Функциональное тестирование	Успех
22.05.21	IDialogService.cs	Разработчик	Функциональное тестирование	Успех
22.05.21	DefaultDialogService.cs	Разработчик	Функциональное тестирование	Успех
22.05.21	RelayCommand.cs	Разработчик	Функциональное тестирование	Успех
22.05.21	StellcovshDetect.db	Разработчик	Функциональное тестирование	Неудача, некорректные атрибуты у ряда полей
22.05.21	StellcovshDetect.db	Разработчик	Функциональное тестирование	Успех

Результаты тестирования программного обеспечения на выполнение требований к функциональным характеристикам представлены в табл. 102.

Таблица 102

Тестирование функциональных характеристик

Дата	Тестируемое требование из ТЗ	Кто проводил тестирование	Способ тестирования	Результат тестирования
1	2	3	4	5
23.05.21	Запись видеопотока камерой тепловизора, установленной в точке прохождения крана со стальковшом	Разработчик	Системное тестирование	Успех

Продолжение табл. 102

1	2	3	4	5
23.05.21	Прием видеопотока с IP-камеры разрабатываемой системой.	Разработчик	Системное тестирование	Успех
23.05.21	Вывод входящего видеопотока на экран программы оператору	Разработчик.	Системное тестирование	Успех
23.05.21	Обработка видеопотока, распознавание стальнойковша и его номера	Разработчик	Системное тестирование	Неудача, номер стальнойковша не распознаётся
23.05.21	Обработка видеопотока, распознавание стальнойковша и его номера	Разработчик	Системное тестирование	Успех
23.05.21	Выделение контуров найденных объектов	Разработчик	Системное тестирование	Успех
23.05.21	Логирование событий	Разработчик.	Системное тестирование	Неудача, отсутствие записи в БД
23.05.21	Логирование событий	Разработчик.	Системное тестирование	Успех

Результаты тестирования программного обеспечения на выполнение требований к надежности продукта представлены в табл. 103.

Таблица 103

Тестирование требований к надежности

Дата и время тестирования	Тестируемое требование из ТЗ	Кто проводил тестирование	Способ тестирования	Результаты тестирования
1	2	3	4	5

20.05.21	Бесспорное функционирование при любых входных данных	Разработчик	Системное тестирование	Успех
----------	--	-------------	------------------------	-------

Продолжение табл. 103

1	2	3	4	5
20.05.21	Оперативное сохранение информации о результатах работы программы	Разработчик	Системное тестирование	Успех
20.05.21	Обеспечение проверки вводимых пользователем данных	Разработчик	Системное тестирование	Успех
20.05.21	Обеспечение проверки вводимых пользователем данных	Разработчик	Системное тестирование	Успех
20.05.21	Логирование данных в лог-файл и БД	Разработчик	Системное тестирование	Неудача при записи в БД
20.05.21	Логирование данных в лог-файл и БД	Разработчик	Системное тестирование	Успех
20.05.21	Корректный вывод данных и результатов на экран, корректное отображение видеозаписи;	Разработчик	Системное тестирование	Неудача, некорректное распознавание объектов
20.05.21	Корректный вывод данных и результатов на экран, корректное отображение видеозаписи;	Разработчик	Системное тестирование	Успех
21.05.21	Система должна иметь защиту от некорректных действий оператора и ошибочных исходных данных	Разработчик	Системное тестирование	Успех

Так как тестирование ПО проводилось на персональном компьютере, имеющем следующие характеристики:

1.Объем оперативной памяти 16 Гб.

2. Монитор с разрешением 1920x1080.
3. Операционная система Windows 10 Домашняя.
4. Твердотельный накопитель объемом 1 Тб.
5. Процессор AMD Ryzen 5 2600 (4 ядра, 8 потоков, тактовая 3.4 ГГц).
6. Видеокарта Gigabyte GeForce GTX 1660Ti с поддержкой DirectX 12.

Данные характеристики соответствуют минимальным системным требованиям, предъявляемым к оборудованию.

В результате модульного тестирования пакетов были выявлены недостатки в разработанном программном обеспечении, которые были устранены в ходе доработки ПО. В ходе системного тестирования также были обнаружены ошибки в работе системы, которые были устранены. В ходе тестирования был установлен факт соблюдения всех требований технического задания. Таким образом, можно сделать вывод, что разработанное программное обеспечение выполняет все заявленные требования технического задания в полном объеме.

2 Технико-экономическое обоснование выполняемой разработки

2.1 Организация работ

Для выполнения проектирования ВКР были составлены следующие задачи и разработана иерархическая структура (рис. 53).

		Решение задачи	Название задачи	Длительность	Начало	Окончание
0			ВКР	188 дней	Чт 01.10.20	Вт 22.06.21
1			Получено задание	0 дней	Чт 01.10.20	Чт 01.10.20
2			Разработка ТЗ	11 дней	Пт 02.10.20	Сб 17.10.20
3			Изучение предметной области	93 дней	Сб 17.10.20	Вт 23.02.21
4			Поиск информации в литературе	10 дней	Сб 17.10.20	Вс 01.11.20
5			Поиск информации в сети	5 дней	Ср 17.02.21	Вт 23.02.21
6			Сравнительный анализ отечественных и зарубежных	3 дней	Ср 17.02.21	Пт 19.02.21
7			Выбор технологии, среды и языка программирования	2 дней	Чт 19.11.20	Сб 21.11.20
8			Анализ процесса обработки информации, выбор	6 дней	Сб 21.11.20	Пн 30.11.20
9			Разработка спецификаций архитектуры ПО	79 дней	Вт 01.12.20	Пт 19.03.21
10			Построение диаграмм вариантов использования	2 дней	Вт 23.02.21	Ср 24.02.21
11			Построение контекстных диаграмм классов	2 дней	Чт 25.02.21	Пт 26.02.21
12			Построение диаграмм последовательности	2 дней	Пн 01.03.21	Вт 02.03.21
13			Построение диаграмм деятельности сценариев ВИ	1 день	Ср 03.03.21	Ср 03.03.21
14			Проектирование структур данных и построение диаграмм отношений компонентов данных	1 день	Чт 04.03.21	Чт 04.03.21
15			Построение диаграмм переходов состояний	2 дней	Пт 05.03.21	Пн 08.03.21
16			Разметка датасета	9 дней	Вт 09.03.21	Пт 19.03.21
17			Обучение нейросети	10 дней	Пн 22.03.21	Пт 02.04.21
18			Проектирование системы	60 дней	Пт 05.03.21	Чт 27.05.21
19			Проектирование системы и построение диаграмм пакетов	2 дней	Пн 05.04.21	Вт 06.04.21

Рис. 53. Список задач

53. Продолжение

20	✓	→	✦ Проектирование классов	4 дней	Ср 07.04.21	Пн 12.04.21
21	✓	→	Построение исходных диаграмм классов в пакетах	1 день	Ср 07.04.21	Ср 07.04.21
22	✓	→	Построение диаграмм последовательности действий	1 день	Чт 08.04.21	Чт 08.04.21
23	✓	→	Построение диаграмм кооперации	1 день	Пт 09.04.21	Пт 09.04.21
24	✓	→	Построение уточненных диаграмм классов в пакетах	1 день	Пн 12.04.21	Пн 12.04.21
25	✓	→	✦ Детальное проектирование классов	27 дней	Пн 15.03.21	Вт 20.04.21
26	✓	→	Разработка детальной структуры класса	1 день	Вт 13.04.21	Вт 13.04.21
27	✓	→	Построение диаграмм состояний объектов	1 день	Ср 14.04.21	Ср 14.04.21
28	✓	→	Построение диаграмм деятельности методов класса	1 день	Пт 16.04.21	Пт 16.04.21
29	✓	→	Построение диаграмм компонентов	1 день	Пн 19.04.21	Пн 19.04.21
30	✓	→	Построение диаграмм размещения	1 день	Вт 20.04.21	Вт 20.04.21
31	✓	→	✦ Проектирование интерфейса пользователя	7 дней	Чт 15.04.21	Пт 23.04.21
32	✓	→	Построение графа диалога	1 день	Чт 15.04.21	Чт 15.04.21
33	✓	→	Разработка форм ввода-вывода информации	3 дней	Ср 21.04.21	Пт 23.04.21
34	✓	→	✦ Разработка кода программы	15 дней	Пн 26.04.21	Пт 14.05.21
35	✓	→	✦ Написание пакета	6 дней	Пн 26.04.21	Пн 03.05.21
36	✓	→	Создание классов пакета	2 дней	Пн 26.04.21	Вт 27.04.21
37	✓	→	Создание функционала пакета	4 дней	Ср 28.04.21	Пн 03.05.21
38	✓	→	Разработка интерфейса	4 дней	Вт 04.05.21	Пт 07.05.21
39	✓	→	Связка компонентов программы	5 дней	Пн 10.05.21	Пт 14.05.21
40	✓	→	✦ Тестирование ПО	15 дней	Пн 17.05.21	Пт 04.06.21
41	✓	→	✦ Выбор стратегии тестирования, разработка тестов, Выбор стратегии тестирования, разработка тестов,	6 дней	Пн 17.05.21	Пн 24.05.21
42	✓	→	Объект и цель испытаний	1 день	Пн 17.05.21	Пн 17.05.21
43	✓	→	Требования к информационном аппаратно-програ обеспечению и документации	2 дней	Вт 18.05.21	Ср 19.05.21
44	✓	→	Состав, порядок и методы испытаний	2 дней	Чт 20.05.21	Пт 21.05.21
45	✓	→	Результаты проведения испытаний	1 день	Пн 24.05.21	Пн 24.05.21
46	✓	→	Тестирование пакета	1 день	Ср 02.06.21	Ср 02.06.21
47	✓	→	Тестирование функционала	1 день	Чт 03.06.21	Чт 03.06.21
48	✓	→	Тестирование интерфейса	1 день	Пт 04.06.21	Пт 04.06.21
49	✓	→	Комплексное тестирование	6 дней	Вт 25.05.21	Вт 01.06.21
50	✓	→	Технико-экономическое обоснование	5 дней	Пн 17.05.21	Пт 21.05.21
51	✓	→	Демонстрация первой версии	0 дней	Вт 01.06.21	Вт 01.06.21
52	✓	→	Исправление недостатков	5 дней	Ср 02.06.21	Вт 08.06.21
53		→	Тестирование обновленной версии	2 дней	Ср 09.06.21	Чт 10.06.21
54	☑	→	Оформление документации	4 дней	Чт 17.06.21	Вт 22.06.21
55	☑	→	Защита ВКР	0 дней	Вт 22.06.21	Вт 22.06.21

Иерархическая структура работ представлена на рис. 54.

		Ре- зад ▾	Название задачи ▾	Длительнс ▾	Начало ▾	Окончани
0			ВКР	188 дней	Чт 01.10.20	Вт 22.06.
1	✓		Получено задание	0 дней	Чт 01.10.20	Чт 01.10.2
2	✓		Разработка ТЗ	11 дней	Пт 02.10.20	Сб 17.10.2
3	✓		Изучение предметной области	93 дней	Сб 17.10.20	Вт 23.02.2
4	✓		Поиск информации в литературе	10 дней	Сб 17.10.20	Вс 01.11.2
5	✓		Поиск информации в сети	5 дней	Ср 17.02.21	Вт 23.02.2
6	✓		Сравнительный анализ отечественных и зарубежных	3 дней	Ср 17.02.21	Пт 19.02.2
7	✓		Выбор технологии, среды и языка программирования	2 дней	Чт 19.11.20	Сб 21.11.2
8	✓		Анализ процесса обработки информации, выбор	6 дней	Сб 21.11.20	Пн 30.11.2
9	✓		Разработка спецификаций архитектуры ПО	79 дней	Вт 01.12.20	Пт 19.03.2
10	✓		Построение диаграмм вариантов использования	2 дней	Вт 23.02.21	Ср 24.02.2
11	✓		Построение контекстных диаграмм классов	2 дней	Чт 25.02.21	Пт 26.02.2
12	✓		Построение диаграмм последовательностей	2 дней	Пн 01.03.21	Вт 02.03.2
13	✓		Построение диаграмм деятельности сценариев ВИ	1 день	Ср 03.03.21	Ср 03.03.2
14	✓		Проектирование структур данных и построение диаграмм отношений компонентов данных	1 день	Чт 04.03.21	Чт 04.03.2
15	✓		Построение диаграмм переходов состояний	2 дней	Пт 05.03.21	Пн 08.03.2
16	✓		Разметка датасета	9 дней	Вт 09.03.21	Пт 19.03.2
17	✓		Обучение нейросети	10 дней	Пн 22.03.21	Пт 02.04.2
18	✓		Проектирование системы	60 дней	Пт 05.03.21	Чт 27.05.2
19	✓		Проектирование системы и построение диаграмм пакетов	2 дней	Пн 05.04.21	Вт 06.04.2
20	✓		Проектирование классов	4 дней	Ср 07.04.21	Пн 12.04.2
21	✓		Построение исходных диаграмм классов в	1 день	Ср 07.04.21	Ср 07.04.2

Рис. 54. Иерархическая структура работ

28	✓	→	Построение диаграмм деятельности методов класса	1 день	Пт 16.04.21	Пт 16.04.21
29	✓	→	Построение диаграмм компонентов	1 день	Пн 19.04.21	Пн 19.04.21
30	✓	→	Построение диаграмм размещения	1 день	Вт 20.04.21	Вт 20.04.21
31	✓	→	▣ Проектирование интерфейса пользователя	7 дней	Чт 15.04.21	Пт 23.04.21
32	✓	→	Построение графа диалога	1 день	Чт 15.04.21	Чт 15.04.21
33	✓	→	Разработка форм ввода-вывода информации	3 дней	Ср 21.04.21	Пт 23.04.21
34	✓	→	▣ Разработка кода программы	15 дней	Пн 26.04.21	Пт 14.05.21
35	✓	→	▣ Написание пакета	6 дней	Пн 26.04.21	Пн 03.05.21
36	✓	→	Создание классов пакета	2 дней	Пн 26.04.21	Вт 27.04.21
37	✓	→	Создание функционала пакета	4 дней	Ср 28.04.21	Пн 03.05.21
38	✓	→	Разработка интерфейса	4 дней	Вт 04.05.21	Пт 07.05.21
39	✓	→	Связка компонентов программы	5 дней	Пн 10.05.21	Пт 14.05.21
40	✓	→	▣ Тестирование ПО	15 дней	Пн 17.05.21	Пт 04.06.21
41	✓	→	▣ Выбор стратегии тестирования, разработка тестов, Выбор стратегии тестирования, разработка тестов,	6 дней	Пн 17.05.21	Пн 24.05.21
42	✓	→	Объект и цель испытаний	1 день	Пн 17.05.21	Пн 17.05.21
43	✓	→	Требования к информационному, аппаратно-программному обеспечению и документации	2 дней	Вт 18.05.21	Ср 19.05.21
44	✓	→	Состав, порядок и методы испытаний	2 дней	Чт 20.05.21	Пт 21.05.21
45	✓	→	Результаты проведения испытаний	1 день	Пн 24.05.21	Пн 24.05.21
46	✓	→	Тестирование пакета	1 день	Ср 02.06.21	Ср 02.06.21
47	✓	→	Тестирование функционала	1 день	Чт 03.06.21	Чт 03.06.21
48	✓	→	Тестирование интерфейса	1 день	Пт 04.06.21	Пт 04.06.21
49	✓	→	Комплексное тестирование	6 дней	Вт 25.05.21	Вт 01.06.21
50	✓	→	Технико-экономическое обоснование	5 дней	Пн 17.05.21	Пт 21.05.21
51	✓	→	Демонстрация первой версии	0 дней	Вт 01.06.21	Вт 01.06.21
52	✓	→	Исправление недостатков	5 дней	Ср 02.06.21	Вт 08.06.21
53		→	Тестирование обновленной версии	2 дней	Ср 09.06.21	Чт 10.06.21
54	📅	→	Оформление документации	4 дней	Чт 17.06.21	Вт 22.06.21
55	📅	→	Защита ВКР	0 дней	Вт 22.06.21	Вт 22.06.21

Для выполнения работ были назначены и выровнены ресурсы, установлены предшественники у задач, связи между ними. Наглядно это можно увидеть на рис.55.

Название ресурса ▼	Тип ▼
Литература	Материальный
Интернет	Материальный
Разработчик	Трудовой
Rational Rose	Материальный
MS Visio	Материальный
ПО Darknet	Материальный
MS Visual Studio	Материальный
Тестолог	Трудовой
Прототип ПО	Материальный
MS Excel	Материальный
Заказчик	Трудовой
MS Word	Материальный

Рис. 55. Ресурсы ВКР

После необходимо выравнивать и назначить ресурсы задачам проекта (рис. 56).

	i	Рес- зад	Название задачи	Названия ресурсов
0			ВКР	
1	✓		Получено задание	Разработчик[200%]
2	✓		Разработка ТЗ	Разработчик;Заказчик
3	✓		Изучение предметной области	
4	✓		Поиск информации в литературе	Литература[1 Книг]
5	✓		Поиск информации в сети	Интернет[1 Гб]
6	✓		Сравнительный анализ отечественных и зарубежных	Интернет[1 Гб]; Литература[1 Книг]
7	✓		Выбор технологии, среды и языка программирования	Разработчик; Интернет[1 Гб]
8	✓		Анализ процесса обработки информации, выбор	Разработчик; Интернет[1 Гб]; Литература[1 Книг]

Рис. 56. Ресурсы проекта

56. Продолжение

9	✓	✈	4 Разработка спецификаций архитектуры ПО		
10	✓	→	Построение диаграмм вариантов использования	Rational Rose[1]; Разработчик	
11	✓	→	Построение контекстных диаграмм классов	MS Visio[1]; Rational Rose[1]; Разработчик	
12	✓	→	Построение диаграмм последовательности	Rational Rose[1]; Разработчик	
13	✓	→	Построение диаграмм деятельностей сценариев ВИ	Rational Rose[1]; Разработчик	
14	✓	→	Проектирование структур данных и построение диаграмм отношений компонентов данных	MS Visio[1]; Rational Rose[1]; Разработчик	
15	✓	→	Построение диаграмм переходов состояний	Rational Rose[1]; Разработчик	
16	✓	→	Разметка датасета	ПО Darknet[1];Разработчик	
17	✓	→	Обучение нейросети	ПО Darknet[1];Разработчик	
18	✓	✈	4 Проектирование системы		
19	✓	→	Проектирование системы и построение диаграмм пакетов	Rational Rose[1]; Разработчик	
20	✓	→	4 Проектирование классов		
21	✓	→	Построение исходных диаграмм классов в пакетах	MS Visio[1]; Rational Rose[1]; Разработчик	
22	✓	→	Построение диаграмм последовательности действий	MS Visio[1]; Rational Rose[1]; Разработчик	
23	✓	→	Построение диаграмм кооперации	MS Visio[1]; Разработчик	
24	✓	→	Построение уточненных диаграмм классов в пакетах	MS Visio[1]; Rational Rose[1]; Разработчик	
25	✓	✈	4 Детальное проектирование классов		
26	✓	→	Разработка детальной структуры класса	MS Visual Studio[1]; MS Visio[1]; Rational Rose[1];Разработчик	
27	✓	→	Построение диаграмм состояний объектов	MS Visio[1]; MS Visual Studio[1]; Rational Rose[1]; Разработчик	
28	✓	→	Построение диаграмм деятельностей методов класса	MS Visio[1]; MS Visual Studio[1]; Разработчик	
29	✓	→	Построение диаграмм компонентов	MS Visio[1]; MS Visual Studio[1]; Разработчик;Rational	
30	✓	→	Построение диаграмм размещения	MS Visio[1]; MS Visual Studio[1]; Rational Rose[1];Разработчик	
31	✓	→	4 Проектирование интерфейса пользователя		
32	✓	→	Построение графа диалога	MS Visio[1]; MS Visual Studio[1];Разработчик	
33	✓	→	Разработка форм ввода-вывода информации	MS Visio[1]; MS Visual Studio[1]; Разработчик	
34	✓	→	4 Разработка кода программы		

35	✓	→	✦ Написание пакета	
36	✓	→	Создание классов пакета	MS Visual Studio[1]; Разработчик
37	✓	→	Создание функционала пакета	MS Visual Studio[1]; Разработчик
38	✓	→	Разработка интерфейса	MS Visual Studio[1]; Разработчик
39	✓	→	Связка компонентов программы	MS Visual Studio[1]; Разработчик
40	✓	→	✦ Тестирование ПО	
41	✓	→	✦ Выбор стратегии тестирования, разработка тестов, Выбор стратегии тестирования, разработка тестов,	
42	✓	→	Объект и цель испытаний	MS Visual Studio[1]; Тестолог;Прототип
43	✓	→	Требования к информационном аппаратно-програ обеспечению и документации	MS Visual Studio[1]; Тестолог; Прототип ПО[1]
44	✓	→	Состав, порядок и методы испытаний	MS Visual Studio[1]; Тестолог; Прототип ПО[1]
45	✓	→	Результаты проведения испытаний	MS Visual Studio[1]; Тестолог; Прототип ПО[1]
46	✓	→	Тестирование пакета	MS Visual Studio[1]; Тестолог;Прототип
47	✓	→	Тестирование функционала	MS Visual Studio[1]; Тестолог;Прототип
48	✓	→	Тестирование интерфейса	MS Visual Studio[1]; Тестолог;Прототип
49	✓	→	Комплексное тестирование	MS Visual Studio[1]; Тестолог;Прототип
50	✓	→	Технико-экономическ обоснование	Разработчик; MS Excel[1]
51	✓	→	Демонстрация первой версии	Прототип ПО[0]; Разработчик
52	✓	→	Исправление недостатков	MS Visual Studio[1]; Разработчик
53		→	Тестирование обновленной версии	Прототип ПО[1]; Тестолог
54	📅	→	Оформление документации	MS Word[1]; Разработчик
55	📅	→	Защита ВКР	Разработчик;Прототип

Помимо этого, для грамотного планирования проекта необходимо иметь представление о его критическом пути.

Критический путь — это последовательность связанных задач, от которых непосредственно зависит дата окончания проекта. Если какая-либо задача на критическом пути выполняется с опозданием, задерживается весь проект.

Критические пути данного проекта представлены на рис. 57.

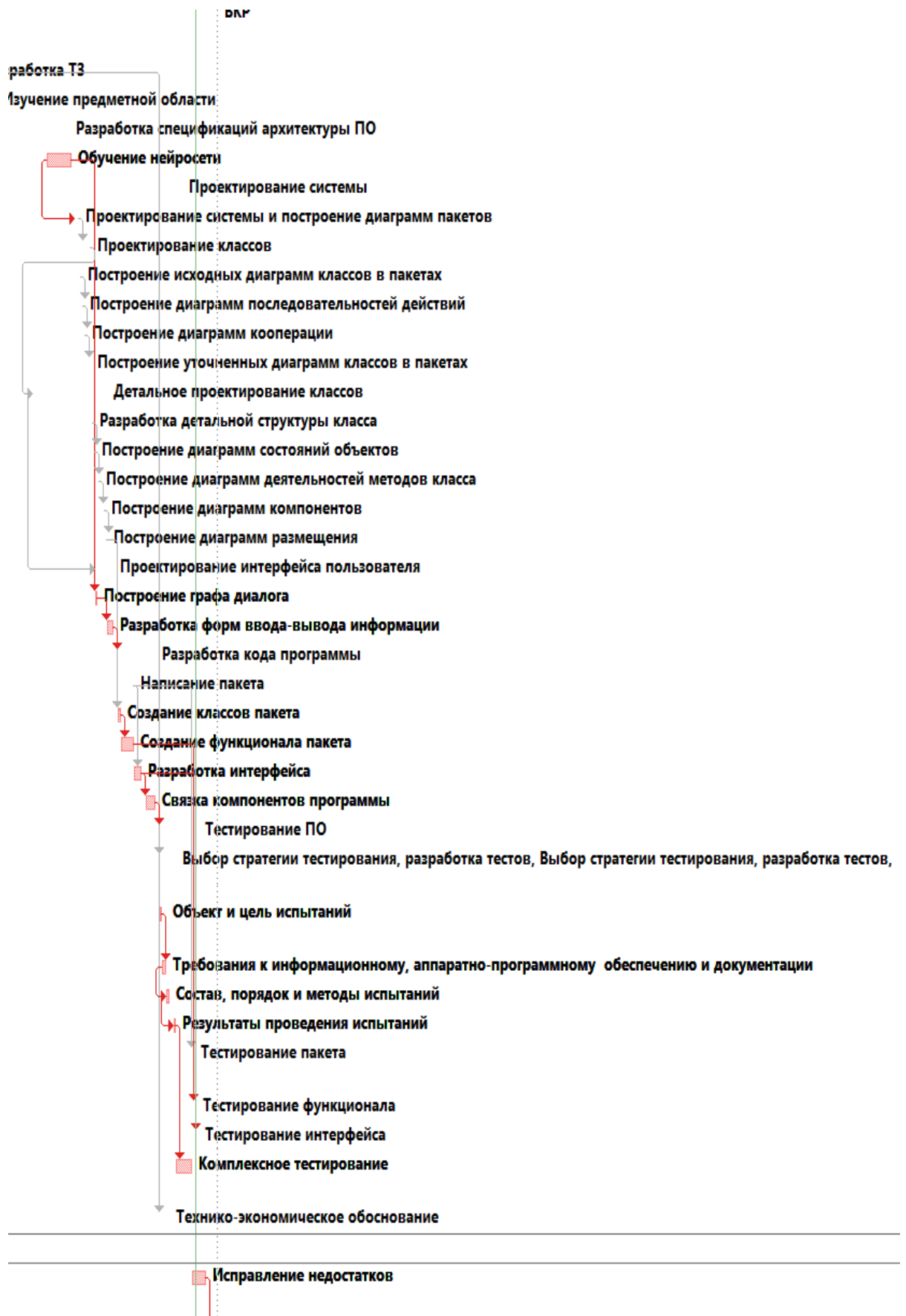


Рис. 57. Критический путь

Связь между установленными предшественниками у задач можно увидеть в диаграмме Ганта (рис.58).

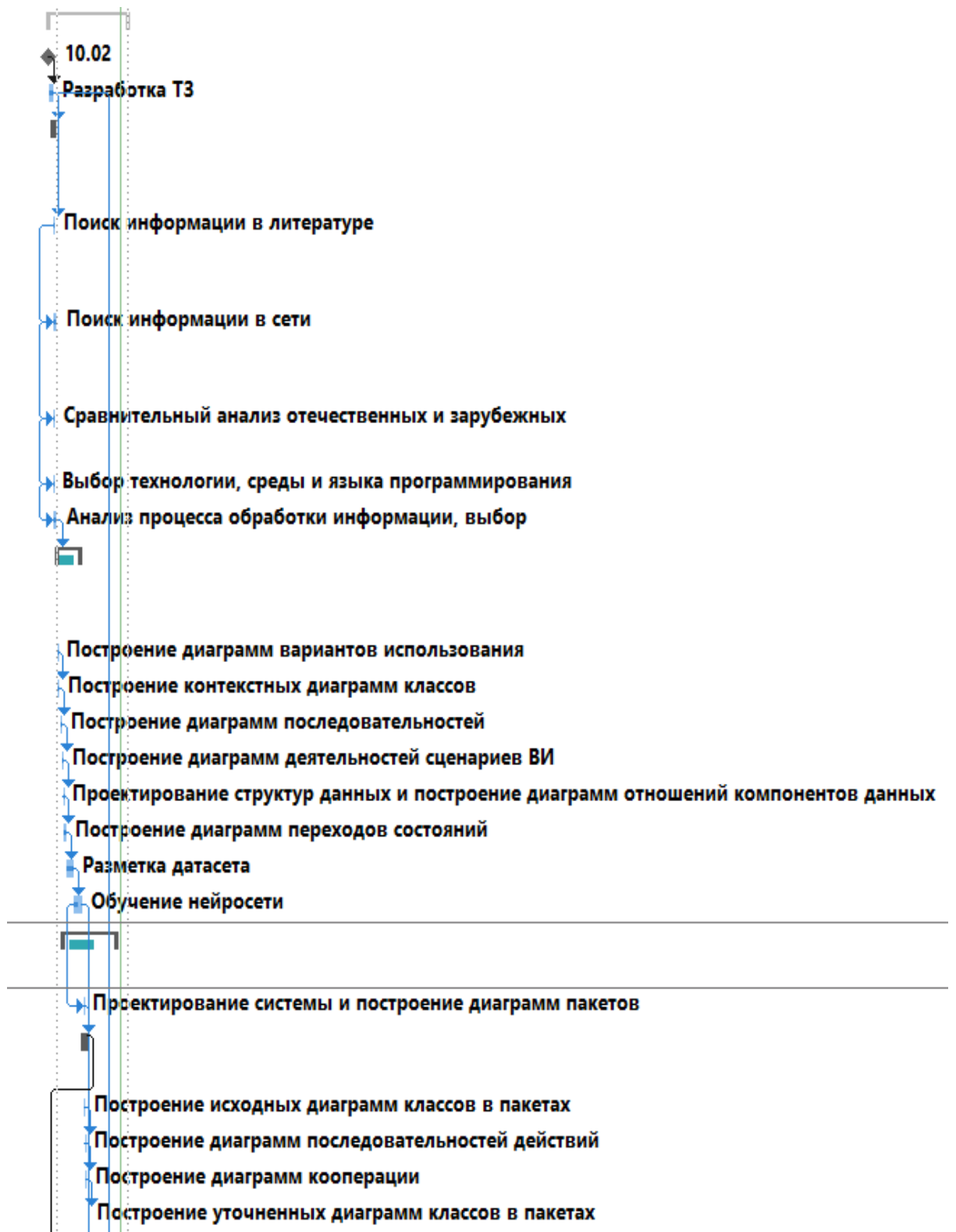
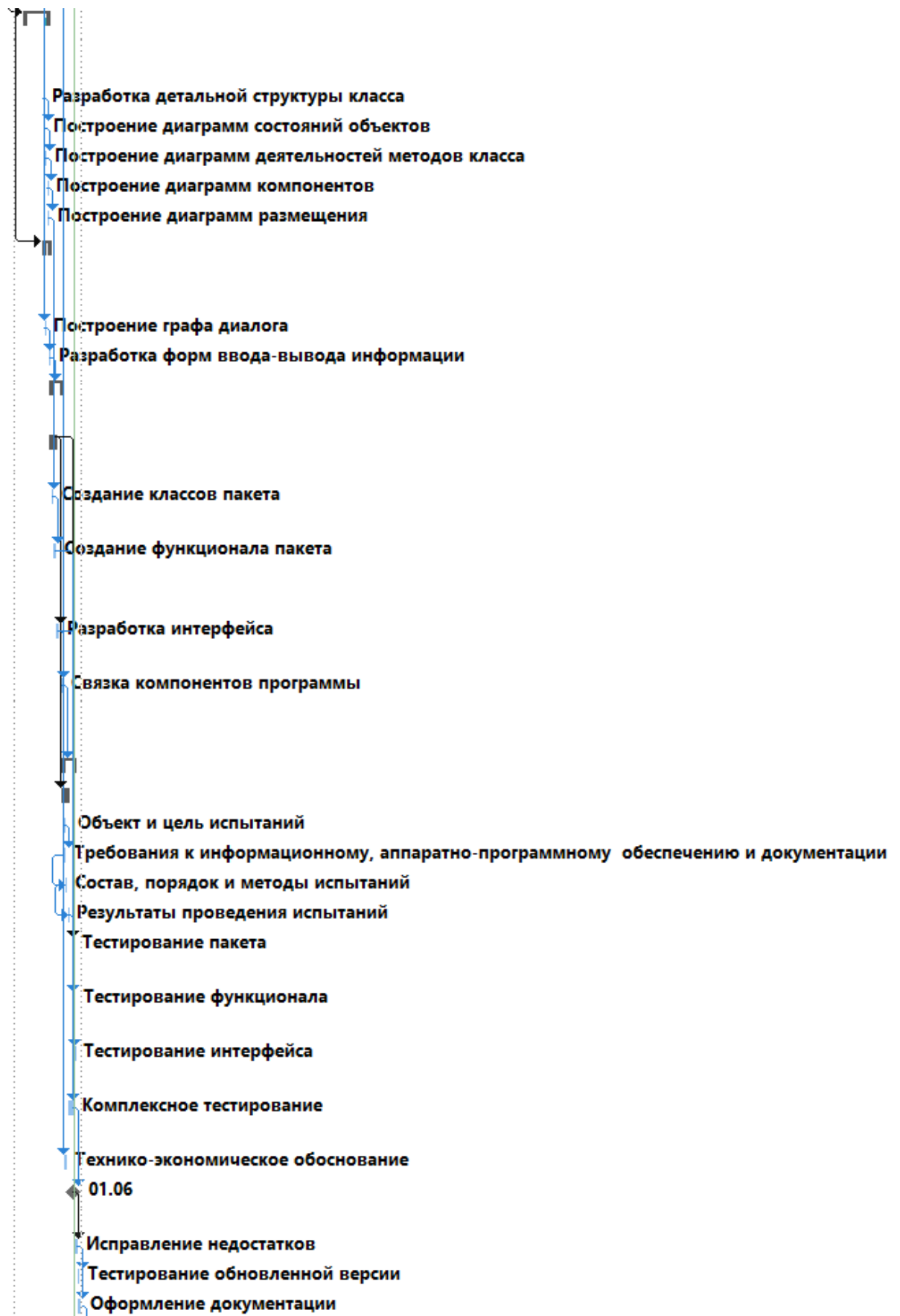


Рис. 58. Диаграмма Ганта



2.2 Расчет себестоимости программного продукта

Состав разработчиков: программист-дипломник, руководитель ВКР.

Затраты на оплату труда при разработке программного продукта вычисляются по формуле:

$$Z_{\text{тр}} = \frac{(Z_{\text{общ}} + \text{Отч}) \cdot T_{\text{н}}}{100} \quad (1)$$

где $Z_{\text{общ}}$ – общая зарплата работника за час, руб.;

Отч – отчисления с зарплаты, %;

$T_{\text{н}}$ – время написания программы, час.

Заработная плата дополнительная определяется по следующей формуле:

$$Z_{\text{доп}} = \frac{Z_{\text{пр}} \cdot H_{\text{доп}}}{100} \quad (2)$$

где $Z_{\text{пр}}$ – заработная плата программиста;

$H_{\text{доп}}$ – норма отчислений на дополнительную зарплату (10%).

Зарплата общая вычисляется по следующей формуле:

$$Z_{\text{общ}} = Z_{\text{пр}} + Z_{\text{доп}} \quad (3)$$

Отчисления на соцстрах, фонд занятости и пенсионный фонд вычисляются по следующей формуле:

$$O_{\text{тч}} = O_{\text{сс}} + O_{\text{фз}} + O_{\text{пф}} \quad (4)$$

где $O_{\text{сс}}$ – отчисления на соцстрах (0,5% от $Z_{\text{общ}}$);

$O_{\text{фз}}$ – отчисления в фонд занятости (0,5% от $Z_{\text{общ}}$);

$O_{\text{пф}}$ – отчисления в пенсионный фонд (2% от $Z_{\text{общ}}$).

Ставка программиста равна 35000 руб., ставка руководителя ВКР составляет 31700 руб.

Дополнительная заработная плата равна 3500 у программиста и 3170 у руководителя ВКР. Зарплаты общие вычисляются по следующей формуле (4):

$$Z_{\text{общ}} = 35000 + 3500 = 38500 \text{ руб.}$$

$$Z_{\text{общ}} = 31700 + 3170 = 34870 \text{ руб.}$$

Отчисления на соцстрах, фонд занятости и пенсионный фонд вычисляются по формуле (5):

$$\text{Отч} = 38500 * 0,005 + 38500 * 0,005 + 38500 * 0,02 = 1155 \text{ руб.}$$

$$\text{Отч} = 34870 * 0,005 + 34870 * 0,005 + 34870 * 0,02 = 1046 \text{ руб.}$$

По формуле (1) рассчитываются затраты на оплату труда при разработке программного продукта:

$$З_{\text{пр}} = (35000 + 1050) * 5 = 180250 \text{ руб.}$$

$$З_{\text{тр}} = (31700 + 951) * 5 = 163255 \text{ руб.}$$

Все данные по заработной плате сводятся в табл. 104.

Таблица 104

Данные по заработной плате

Должность разработчика	Разряд	Время работы,	Ст _{пр} , руб.	З _{пр} , руб.	З _{доп} , руб.	З _{общ} , руб.	Отч, руб.	З _{тр} , руб.
Программист-дипломник		5	35000	35000	3500	38500	1155	198275
Руководитель ВКР		5	31700	31700	3170	34870	1046	179580,5
Итого:			66700	66700	6670	73370	2201,1	377855,5

Стоимость одного часа машинного времени рассчитывается по формуле:

$$C_{\text{м.вр}} = \frac{Ц_{\kappa}}{C_{\text{сл.к}} * K_{\text{р.д}} * B_{\text{р.с}}} + C_{\text{т.э}} * M_{\text{вс}} \quad (5)$$

где $C_{\text{м.вр}}$ – стоимость одного часа машинного времени, руб./ч;

C_k – покупная цена компьютера, руб.;

$C_{сл.к}$ – срок службы компьютера, год;

$K_{р.д}$ – количество рабочих дней в году;

$Вр_c$ – время работы компьютера в течение суток, ч;

$C_{тэ}$ – стоимость одного кВт*ч электроэнергии, руб.;

$M_{вс}$ – мощность вычислительной системы, кВт.

По формуле (5) рассчитаем стоимость одного часа машинного времени:

$$C_{м.вр} = \frac{50000}{5 \cdot 247 \cdot 4} + 4.87 \cdot 0.8 = 14.018 \text{ руб./ч.}$$

Время использования вычислительной техники рассчитывается по следующей формуле:

$$Вр_{в.т} = K_{д.р} \cdot Вр_c \quad (6)$$

где $Вр_{в.т}$ – время использования вычислительной техники, ч;

$K_{д.р}$ – количество дней разработки ПО.

По формуле (7) рассчитаем время использования вычислительной техники:

$$Вр_{в.т} = 120 \cdot 5 = 600 \text{ ч.}$$

Затраты на носители информации принимаются в размере 2 % от цены вычислительной техники.

$$Z_{н.и.} = C_k \cdot 2, \quad (7)$$

По формуле (7) рассчитаем затраты на носители информации:

$$Z_{н.и.} = 50000 \cdot 2 = 1000 \text{ руб.}$$

Затраты на использование машинного времени вычисляются по формуле:

$$Z_{м.вр} = C_{м.вр} \cdot V_{в.т} \quad (8)$$

где $Z_{м.вр}$ – затраты на использование машинного времени, руб.;

$C_{м.вр}$ – стоимость одного часа машинного времени, руб./ч;

$V_{в.т}$ – время использования вычислительной техники, ч.

По формуле (8) рассчитаем затраты на использование машинного времени:

$$Z_{м.вр} = 14,018 \cdot 600 = 8410,48 \text{ руб.}$$

Затраты на текущий и профилактический ремонт составляют 4% от цены вычислительной техники и вычисляются по следующей формуле:

$$Z_{рем} = C_k \cdot 4 \quad (9)$$

По формуле (9) рассчитаем затраты на текущий и профилактический ремонт:

$$Z_{\text{рем}} = 50000 \cdot 4\% = 2000 \text{ руб.}$$

Прочие эксплуатационные расходы включают в себя затраты на освещение, отопление, охрану, уборку и текущий ремонт помещений. Они принимаются в размере 10 % от стоимости помещения (или его аренды), где происходит разработка программного продукта.

$$Z_{\text{пр}} = C_{\text{пм}} \cdot 10\% \quad (10)$$

где $C_{\text{пм}}$ — стоимость помещения, руб.

По формуле (10) рассчитаем прочие эксплуатационные расходы:

$$Z_{\text{пр}} = 12000 \cdot 0,1 = 1200 \text{ руб.}$$

В себестоимость разработки входят: основная заработная плата разработчиков, дополнительная заработная плата, единый социальный налог, затраты на использование машинного времени, затраты на носители информации, затраты на текущий и профилактический ремонт вычислительной техники, прочие эксплуатационные расходы.

$$C_{\text{пп}} = Z_{\text{тр}} + Z_{\text{м.вр}} + Z_{\text{н.и}} + Z_{\text{рем}} + Z_{\text{пр}} \quad (11)$$

По формуле (11) рассчитаем себестоимость разработки:

$$C_{\text{пп}} = 377855,5 + 8410,48 + 1000 + 2000 + 1200 = 390465,98 \text{ руб.}$$

Расчет цены программного продукта

Для определения минимальной цены, ниже которой разработчику будет невыгодно продавать программный продукт, используется следующая формула:

$$Ц_{\text{п.п}} = C_{\text{п.п}} \cdot (1 + H_{\text{пр}}) \quad (12)$$

где $Ц_{\text{п.п}}$ – цена программного продукта, руб.;

$C_{\text{п.п}}$ – себестоимость программного продукта, руб.;

$H_{\text{пр}}$ – норматив прибыли (20 %, в формуле $H_{\text{пр}} = 0,2$).

Рассчитаем минимальную цену по формуле (12):

$$Ц_{\text{п.п}} = 390465,98 \cdot (1 + 0,2) = 468559,17 \text{ руб.}$$

Расчет экономической эффективности

Стоимость одного часа машинного времени для потребителя рассчитывается по формуле (5):

$$C_{\text{м.вр}} = \frac{50000}{4 \cdot 365 \cdot 24} + 2,87 \cdot 0,8 = 3,72 \text{ руб./ч.}$$

Рассчитаем объем машинного времени в течении года необходимый для решения задачи по формуле (6):

$$V_{\text{р.п.}} = 365 \cdot 24 = 8760 \text{ ч.}$$

Расходы потребителя, связанные с эксплуатацией программы, определяются по следующей формуле:

$$P_{\text{э.п}} = V_{\text{р.п.}} \cdot C_{\text{м.вр}} + \frac{Ц_{\text{п.п.}}}{C_{\text{сл}}} \quad (13)$$

где $P_{\text{э.п}}$ – эксплуатационные расходы потребителя, руб.;

$V_{\text{р.п.}}$ – объем машинного времени в течение года, необходимый для решения данной задачи с использованием программы, ч;

$C_{\text{м.вр}}$ – стоимость одного часа машинного времени, руб./ч;

$Ц_{\text{п.п.}}$ – цена программного продукта, руб.;

$C_{\text{сл}}$ – срок службы программного продукта, год. Обычно составляет 1 – 2 года, затем выпускается новая версия программного продукта.

Рассчитаем расходы потребителя связанные с эксплуатацией программы по формуле (13):

$$P_{\text{э.п}} = 8760 \cdot 3,72 + \frac{468559,17}{2} = 266892,54 \text{ руб.}$$

Если на момент внедрения программного продукта у потребителя все работы выполнялись вручную, тогда капитальные затраты рассчитываются по формуле:

$$P_{\text{кап}} = \frac{B p_{\text{п.п}} \cdot K_{\text{ЭВМ}}}{\Phi_{\text{вр}}} + \Pi_{\text{п.п}} \quad (14)$$

где $P_{\text{кап}}$ – капитальные расходы потребителя, руб.;

$\Phi_{\text{вр}}$ – полезный годовой фонд времени работы вычислительной техники, принимается условно 2000 ч в год;

$K_{\text{ЭВМ}}$ – капитальные затраты на вычислительную технику, для которой предназначена программа, руб.

Капитальные затраты на вычислительную технику рассчитываются по формуле:

$$K_{\text{ЭВМ}} = \Pi_{\text{ЭВМ}} + P_{\text{п.п}} \quad (15)$$

где $\Pi_{\text{ЭВМ}}$ – цена вычислительной техники, руб.; $P_{\text{п.п}}$ – прочие расходы потребителя, связанные с помещением (отопление, освещение, уборка и т.д.),

принимаются в размере 10 % от стоимости помещения потребителя (или его аренды), руб.

Рассчитаем капитальные затраты на вычислительную технику по формуле (15):

$$K_{\text{ЭВМ}} = 50\,000 + 12\,000 = 62\,000 \text{ руб.}$$

По формуле (14) рассчитаем капитальные затраты:

$$P_{\text{кап}} = \frac{8760 * 62\,000}{2000} + 468559,17 = 740119,17 \text{ руб.}$$

Для расчета годовой экономии эксплуатационных расходов потребителя вычисляются эксплуатационные затраты потребителя при решении задачи вручную:

$$P_{\text{э.руч}} = 1,21 * \text{ФЗП} * 12 \quad (16)$$

где $P_{\text{э.руч}}$ – эксплуатационные расходы потребителя при решении задачи вручную, руб.;

ФЗП – фонд заработной платы персонала, обслуживающего решение задачи вручную, руб.;

12 – количество месяцев в году;

1,21 – поправочный коэффициент.

Рассчитаем эксплуатационные расходы потребителя при решении задачи вручную по формуле (16):

$$P_{\text{э.руч}} = 1,21 * 43000 * 12 = 624360 \text{ руб.}$$

Тогда годовая экономия эксплуатационных расходов у одного потребителя рассчитывается по формуле:

$$\text{Э} = P_{\text{э.руч}} - P_{\text{э.п.}} \quad (17)$$

Рассчитаем годовую экономию эксплуатационных расходов у одного потребителя по формуле (17):

$$\text{Э} = 624360 - 266892,54 = 357467,46 \text{ руб.}$$

Срок окупаемости программного продукта рассчитывается по формуле:

$$T_{\text{ок}} = \frac{P_{\text{кап}}}{\text{Э}} \quad (18)$$

Рассчитаем срок окупаемости программного продукта по формуле (18):

$$T_{\text{ок}} = \frac{740119,17}{357467,46} = 2,07 \text{ года}$$

Годовой экономический эффект, получаемый одним потребителем, рассчитывается по формуле:

$$\Xi\Xi = \Xi - E_n \cdot P_{\text{kap}} \quad (19)$$

где E_n – нормативный коэффициент эффективности дополнительных капитальных вложений, равный 0,15.

Рассчитаем годовой экономический эффект, получаемый одним потребителем по формуле (19):

$$\Xi\Xi = 357467,46 - 0,15 \cdot 740119,17 = 246449,58 \text{ руб.}$$

Заключение

В результате выполнения выпускной квалификационной работы было спроектированной и реализовано программное обеспечение распознавания номера стальной, на производстве компании ПАО «Северсталь». Она представляет собой приложение, способное выполнять распознавание номера и непосредственно самого стальной с помощью установленных камер тепловизора. Также позволяет фиксировать, время, номер распознанного объекта для дальнейшего анализа.

Данная разработка позволит сократить затраты предприятия за счет сокращения времени простоя стальной и оптимизации их перемещения.

Программное обеспечение реализовано посредством языка программирования C# в среде разработки MS VisualStudio 2019с использованием библиотеки машинного зрения OpenCV и нейронной сети YOLOv3.

В ходе выпускной квалификационной работы были приобретены следующие компетенции:

- владение навыками использования различных технологий разработки программного обеспечения[6];
- владение концепциями и атрибутами качества программного обеспечения (надежности, безопасности, удобства использования), в том числе, роли людей, процессов, методов, инструментов и технологий обеспечения качества[6];
- владение классическими концепциями и моделями менеджмента в управлении проектами[6];
- способность к формализации в своей предметной области с учетом ограничений используемых методов исследования[6];
- готовность к использованию методов и инструментальных средств исследования объектов профессиональной деятельности[6];
- готовность обосновать принимаемые проектные решения, осуществлять постановку и выполнение экспериментов по проверке их корректности и эффективности[6];
- способность готовить презентации, оформлять научно-технические отчеты по результатам выполненной работы, публиковать результаты исследований в виде статей и докладов на научно-технических конференциях[6];
- способность формализовать предметную область программного проекта и разработать спецификации для компонентов программного продукта[6];
- способность выполнить начальную оценку степени трудности, рисков, затрат и сформировать рабочий график[6];
- способность готовить коммерческие предложения с вариантами решения[6].

Список литературы

1. Бадд Т. Объектно-ориентированное программирование в действии. — СПб: Питер, 1997. — ISBN 5-88782-270-8.
2. Бишоп, Дж. С# в кратком изложении / Дж. Бишоп, Н. Хорспул. — М.: Бином. Лаборатория знаний, 2011. 472 с.
3. Боггс, М. UML и RationalRose / М. Боггс. — Москва: РГГУ, 2010. — 385с.
4. Буч, Г., Язык UML. Руководство пользователя / Грейди Буч, Джеймс Рамбо, Айвар Джекобсон. — М.: ДМК, 2015. — 432 с.
5. Ершов Е.В., д-р техн. наук, проф.; Виноградова Л.Н. и др. Методика и организация самостоятельной работы студентов — Коллектив авторов, ФГБОУ ВПО «Череповецкий государственный университет», 2012. —208 с.
6. Ершов Е.В., Виноградова Л.Н., Веселов Ю.В. Программа итоговой (государственной итоговой) аттестации. / Ершов Е.В. - Череповец.: Череповецкий государственный университет, 2015 г. — 27 с.
7. Иванова, Г.С. Технология программирования: Учебник для вузов. - М.: Изд-во МГТУ им. Н.Э. Баумана, 2002. -320 с.: ил. (Сер. Информатика в техническом университете.)
8. Камаев, В.А. Технологии программирования / В.А. Камаев, В.В. Костерин. — М.: Высшая школа, 2006. — 454 с.
9. Киммел, Пол. UML. Основы визуального анализа и проектирования / Пол Киммел. — М.: НТ Пресс, 2008. — 272 с.
10. Леоненков А.В. Объектно-ориентированный анализ и проектирование с использованием UML и IBM Rational Rose : учебное пособие— Москва :

Интернет-Университет Информационных Технологий (ИНТУИТ), Ай Пи Ар Медиа, 2020. — 317 с. — ISBN 978-5-4497-0667-6.

11. Либерти Д. Язык программирования C# . Программирование на C#. — Санкт-Петербург. — 2003: Символ-Плюс, 2003. — С. 26. — 688 с.
12. Мак-Дональд М. WPF – WindowsPresentationFoundation в .NET 4.5 с примерами на C# 5.0 для профессионалов / М. Мак-Дональд. – ИД Вильямс, 2013. – 1024 с.
13. Назаров, С.В. Архитектура и проектирование программных систем: Монография. / С.В. Назаров. - М.: ИНФРА-М, 2013. - 351 с.
14. Хорев, П.Б. Технологии объектно-ориентированного программирования / П.Б. Хорев. - М.: Academia, 2013. - 286 с.
15. Орлов С.А. Технологии разработки программного обеспечения / С. Орлов. – СПб.: Питер, 2002. – 464 с.
16. Пироцкая В.Н., Градусов Д.А. Основы тестирования программного обеспечения: Учебное пособие – Владимирский государственный университет, 2017. – 100 с.
17. Рассел, Дж., Жизненный цикл программного обеспечения / Джесси Рассел. – М.: Книга по Требованию, 2012. – 89 с.
18. Рассел, Дж., Интерфейс программирования приложений / Джесси Рассел. – М.: Книга по Требованию, 2012. – 574 с.
19. Форсайт Дэвид, Понс Жан. Компьютерное зрение. Современный подход / пер. Назаренко А.В. – М.: Вильямс, 2014. – 928 с.
20. Studopedia [Электронный ресурс] — Режим доступа: URL: https://studopedia.ru/3_175686_proektirovanie-po-pri-ob-ektnom-podhode.html
21. Studopedia [Электронный ресурс] — Режим доступа: URL: https://studopedia.ru/3_175686_proektirovanie-po-pri-ob-ektnom-podhode.html

podhode.html

22. AutoTRASSIR — описание модуля распознавания номеров автомобильных номеров в TRASSIR — База знаний DSSL [Электронный ресурс] — Режим доступа: URL: <https://confluence.trassir.com/display/WD/AutoTRASSIR>

23. TRASSIRFaceDetector [Электронный ресурс] — Режим доступа: URL: https://trassir.ru/products/programmnoe_obespechenie/trassir_face_detector/

24. Распознавание автомобильных номеров — ОБЗОР 16 производителей [Электронный ресурс] — Режим доступа: URL: <https://securityrussia.com/blog/avto-nomera.html>

25. IItVision [Электронный ресурс] — Режим доступа: URL: <http://iitvision.ru/?p=191>.

26. RB [Электронный ресурс] — Режим доступа: URL: <https://rb.ru/list/computer-vision-in-practice/>

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Череповецкий государственный университет»

Институт информационных технологий
Кафедра математического и программного обеспечения ЭВМ

УТВЕРЖДАЮ

Зав. кафедрой МПО ЭВМ,

д.т.н., профессор _____ Ершов Е.В.

«___» _____ 2021 г.

Разработка программного обеспечения распознавания номера стальной

Техническое задание на выпускную квалификационную работу

Руководитель: Заведующий кафедрой
МПО ЭВМ Ершов Е.В.

Исполнитель: студент гр.

1ПИБ-01-41оп Краев А.А.

Череповец, 2021 г.
Введение

Задача оптимизации неразрывно связана с развитием промышленного производства с момента зарождения первых мануфактур, вместе с развитием промышленных технологий менялись и инструменты оптимизации. В 21 веке с развитием цифровых технологий данные группы людей стало дополнять программное обеспечение способное оперативно обрабатывать огромные объемы данных.

Программное обеспечение (ПО) — программа или множество программ, используемых для управления компьютером.

До появления ПО и автоматизированного производства, вопросом проверки качества выполнения обязанностей рабочим персоналом занимались люди. Существовали специальные отделы (или команды), которые следили за ходом выполнения работ и их качеством.

Для решения одной из проблем оптимизации в данной работе будет спроектировано ПО для распознавания номера стальной, с помощью камеры тепловизора и инструментов машинного зрения, нейронной сети.

На предприятии «Северсталь» существует проблема, связанная с контролем перемещения стальной, а именно требуется программное обеспечение для фиксации стальной и его номера на разных участках производства с указанием времени.

Для решения данной проблемы, было принято решение спроектировать и разработать ПО, которое с помощью камер и компьютерного зрения в связке с нейронной сетью будет распознавать сталь и его номер.

Заказчиком данного ПО выступила «Северсталь Менеджмент».

1. Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу, выданное на кафедре МПО ЭВМ.

Дата утверждения: 10.02.2020 года.

Наименование темы разработки: Разработка программного обеспечения распознавания номера стальной.

2. Назначение разработки

Данное программное обеспечение предназначено для решения проблемы на предприятии АО «Северсталь», связанная с контролем движения стальной по территории производства.

Цели создания системы является повышение эффективности производства за счет отслеживания перемещения стальной по территории предприятия

3. Требования к разработке

3.1. Требования к функциональным характеристикам

Программное обеспечение включает в себе приложение, реализующее следующий функционал:

1. Запись видеопотока камерой тепловизора, установленной в точке прохождения крана со сталью.
2. Прием видеопотока с IP-камеры разрабатываемой системой.
3. Вывод входящего видеопотока на экран программы оператору.
 - частота кадров не менее 15 кадр. /сек.;
 - размер окна видеопотока должен позволять осуществлять контроль оператору.
4. Обработка видеопотока, распознавание стальной и его номера.
 - обработанный видеопоток должен выводиться с частотой кадров не менее 12 кадр. /сек.;
 - точность распознавания должна быть не менее 93%.
5. Выделение контуров найденных объектов.

- контур стальной и номера стальной должны выделяться разным цветом;

- цвета выделения не должны сливаться с контуром;

- контур выделения должен сочетаться с контуром реального объекта.

6. Логирование событий.

- информация для логирования: номер стальной, место съемки, модель камеры, дата и время;

- необходимо логирование в БД и лог файл на компьютере оператора.

3.2. Требования к надежности

Разрабатываемая система должна удовлетворять следующим требованиям относительно надежности:

- беспробойное функционирование при любых входных данных;
- оперативное сохранение информации о результатах работы программы;
- обеспечение проверки вводимых пользователем параметров;
- логирование данных в лог-файл и БД;
- корректный вывод данных и результатов на экран, корректное отображение видеозаписи;
- система должна иметь защиту от некорректных действий оператора и ошибочных исходных данных.

3.3. Требования к эксплуатации

Программа предназначена для работы на персональном компьютере одним пользователем.

Программа работает в автономном режим после запуска, однако оператор обязан следить за зафиксированными данными и выполнять передачу информации о них.

Камеры тепловизора, формирующие видеопоток для программы должны обеспечивать качественную съемку необходимых для системы кадров.

Требования к пользовательскому интерфейсу программы выдвигаются следующие:

- лаконичность интерфейса - минимальное количество элементов на экране;
- возможность выделять зоны на видеопотоке;
- интерфейс должен быть выполнен в спокойных цветах (голубой и его оттенки);
- наличие настроек программы.

3.4. Требования к составу и параметрам технических средств

Для корректной работы системы необходимы персональные компьютеры, удовлетворяющие следующим минимальным требованиям:

- процессор с 4 потоками и тактовой частотой 3,2 ГГц или большей;
- не менее 8ГБ оперативной памяти;
- видеокарта – не менее GTX 1660Tiс объемом видеопамяти 6 Гб;
- 2 ГБ на жестком диске;
- графическое устройство с поддержкой DirectX 9или более поздней версии;
- устройство ввода (мышь, клавиатура);
- устройства вывода (монитор);
- подключение к локальной сети предприятия (в случае необходимости доступа к камерам и интеграции с базами данных).

3.5. Требования к информационной и программной совместимости

Программа должна быть разработана на языке программирования C# в среде разработки MSVisualStudio 2019, при поддержке открытой библиотеки OpenCV.

Работа с программой будет осуществляться на персональном компьютере, управляемом операционной системой Windows 8,10.

База данных должна быть реализована в СУБД SQLite.

3.6. Требования к маркировке и упаковке

Диск, на котором будет храниться программа, должен быть подписан и находиться в специальном бумажном конверте, имеющем прозрачную пластиковую вставку для показа диска.

3.7. Требования к транспортированию и хранению

Передача программы осуществляется строго заказчику в руки на диске, или флеш – устройстве.

3.8. Специальные требования

Не предъявляются.

4. Требования к программной документации

Разработка данной программы предполагает написание следующей документации:

- расчетно-пояснительной записки;
- приложения (текст программы, спецификация, руководство пользователя).

5. Стадии и этапы разработки

Стадии и этапы разработки представлены в табл. П1.1.

Таблица П1.1

Стадии и этапы разработки

Наименование этапа разработки	Сроки разработки	Результат выполнения	Отметка выполнения
1	2	3	4
1.Разработка ТЗ	07.02.2021 – 10.02.2021	1.Техническое задание	х
2.Изучение предметной области	11.02.2021 – 26.02.2021	2.Представление о предметной области	х
3.Выбор технологии, среды и языка программирования	27.02.2021 – 1.03.2021	3. Выбраны технологии, среда и язык программирования	х
4.Анализ процесса обработки информации, выбор структур данных для ее хранения, выбор методов и алгоритмов решения	2.03.2021 – 16.03.2021	4. Выбраны структуры данных и алгоритмы решения поставленной задачи	х
5.Разработка спецификации и архитектуры ПО	17.03.2021 – 28.03.2021	5. Разработана спецификация архитектуры ПО	х
6. Проектирование	29.03.2021 – 4.04.2021	6. Спроектирован	

пользовательского интерфейса		пользовательский интерфейс	
7.Разработка ПО	5.04.2021 – 14.05.2021	7.Готовое ПО	х
8.Тестирование ПО	15.05.2021 – 19.05.2021	8.Рабочее ПО	х
9.Технико-экономическое обоснование разработки (Организация работ)	20.05.2021 – 21.05.2021	9.Выполнена организация работ	х
10. Расчет себестоимости программного продукта	22.05.2021	10. Рассчитана себестоимость программного продукта	х
11. Расчет цены программного продукта	23.05.2021	11. Рассчитана цена программного продукта	х
12. Расчет экономической эффективности	24.05.2021	12. Рассчитана экономическая эффективность	х

Продолжение табл. П1.1.

1	2	3	4
13.Составление (ВКР)	25.05.2021 – 3.06.2021	13.Готовая РПЗ	х
14. Защита ВКР	25.06.2021	14. Успешная защита ВКР	

6. Порядок контроля и приемки

Порядок контроля и приемки представлен в табл. П1.2.

Таблица П1.2

Порядок контроля и приемки

Наименование контрольного этапа выполнения	Сроки разработки	Результат выполнения	Отметка о приемке результатов контрольного этапа
1	2	3	4
1. Проверка технического задания	10.02.2021	1. Техническое задание	х
2. Изучение предметной области	26.02.2021	2. Понятия предметной области	х
3. Выбор технологии, среды и языка программирования	1.03.2021	3. Выбраны технологии, среда и язык программирования	х
4. Анализ процесса обработки информации, выбор структур данных для ее хранения, выбор методов и алгоритмов решения	16.03.2021	4. Выбраны структуры данных и алгоритмы решения поставленной задачи	х
5. Разработка спецификации и архитектуры ПО	28.03.2021	5. Разработана спецификация архитектуры ПО	х
6. Проектирование пользовательского интерфейса	4.04.2021	6. Спроектирован пользовательский интерфейс	х
7. Разработка ПО	14.05.2021	7. Разработанное ПО	х
8. Тестирование ПО	19.05.2021	8. Протестировано ПО	х
9. Техничко-экономическое обоснование разработки (Организация работ)	21.05.2021	9. Выполнена организация работ	х

Продолжение табл. П1.2.

1	2	3	4
10. Расчет себестоимости программного продукта	22.05.2021	10. Рассчитана себестоимость	х

		программного продукта	
11. Расчет цены программного продукта	23.05.2021	11. Рассчитана цена программного продукта	х
12. Расчет экономической эффективности	24.05.2021	12. Рассчитана экономическая эффективность	х
13. Составление ВКР	03.06.2021	13. Готовая ВКР	х
14. Защита ВКР	25.06.2021	14. Успешная защита ВКР	

Текст программы

```

<mah:MetroWindow
  x:Class="StellcovshDetected.MainWindow"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mah="clr-namespace:MahApps.Metro.Controls;assembly=MahApps.Metro"

xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  xmlns:local="clr-namespace:StellcovshDetected"
  Title="StellCovshDetected"
  Width="1160"
  Height="760"
  WindowStartupLocation="CenterScreen"
  WindowState="Maximized"
  mc:Ignorable="d">

  <Grid>
    <Grid.RowDefinitions>
      <RowDefinition Height="Auto" />
      <RowDefinition Height="*" />
    </Grid.RowDefinitions>
    <Menu DockPanel.Dock="Top" Grid.Row="0">
      <MenuItem Command="{Binding ShowSettings}"
        Header="Настройки" />
    </Menu>
    <Grid Grid.Row="1">
      <Grid.RowDefinitions>
        <RowDefinition Height="500"/>
        <RowDefinition Height="*" />
      </Grid.RowDefinitions>
      <Grid.ColumnDefinitions>
        <ColumnDefinition Width="790"/>
        <ColumnDefinition Width="*" />
      </Grid.ColumnDefinitions>
      <Border Grid.Column="0" Grid.Row="0"
        BorderBrush="Black" Height="450" Width="620"
        BorderThickness="1" VerticalAlignment="Center"
        Padding="3" Margin="5"
        Background="Azure">
        <Image x:Name="inputVideo" Source="{Binding InputVideo}" />
      </Image>
    </Border>

    <Border Grid.Column="1" Grid.Row="0"
      BorderThickness="1" BorderBrush="Black">
      <DataGrid Margin="10" AutoGenerateColumns="False"
        BorderThickness="1,1,1,1"
        ItemsSource="{Binding StellCovshList}"><DataGrid.Columns>
        <DataGridTextColumn Width="120"
          Binding="{Binding Time}"
          Header="Время"
          IsReadOnly="True" />
        <DataGridTextColumn Width="180"
          Binding="{Binding Point}"
          Header="Участок" IsReadOnly="True" />
        <DataGridTextColumn
          Width="auto"
          Binding="{Binding Number}"
          Header="Номерстальковша"
          IsReadOnly="True" />
        <DataGridTextColumn
          Width="auto"
          Binding="{Binding Camera}" Header="Камера"
          IsReadOnly="True" />
      </DataGrid.Columns>
    </DataGrid>
    </Border>
    <Border Grid.Column="0" Grid.Row="1"
      BorderThickness="1" BorderBrush="Black">
      <StackPanel Orientation="Vertical"
        HorizontalAlignment="Stretch">
        <StackPanel Orientation="Horizontal"
          VerticalAlignment="Center">
          <StackPanel Orientation="Vertical"
            HorizontalAlignment="Center">
            <CheckBox Margin="15" IsChecked="{Binding LogDB}">ЗаписьвБД</CheckBox>
            <CheckBox Margin="15" IsChecked="{Binding LogTxt}">Записьв Log-файл</CheckBox>
          </StackPanel>
          <Button Margin="15" Padding="20"
            HorizontalAlignment="Right">
            <Button.Style>
              <Style BasedOn="{StaticResource MahApps.Styles.Button}" TargetType="Button">
                <Style.Triggers>
                  <DataTrigger Binding="{Binding SystemStarted}"
                    Value="false">
                    <Setter Property="Command" Value="{Binding StartCommand}" />
                    <Setter Property="Content"
                      Value="Запусксистемы" /></DataTrigger>
                  <DataTrigger Binding="{Binding SystemStarted}"
                    Value="true">
                    <Setter Property="Command" Value="{Binding StopCommand}" /><Setter Property="Content"
                      Value="Остановитьсистему" />
                  </DataTrigger></Style.Triggers></Style>
                </Button.Style>

```

Рис. П2.1. Текст программы модуля MainWindow.xaml

П2.1. Продолжение

```

</Button>
<CheckBoxMargin="15"HorizontalAlignment="Right"IsC
hecked="{BindingDisbDetect}">Отключитьраспознавани
e</CheckBox>
</StackPanel>
<StackPanel Orientation="Horizontal"
VerticalAlignment="Center">
<Button Margin="15" Padding="20" Command="{Binding
OpenFileDialogCommand}">Открытьвидео</Button>
<Button Margin="15" Padding="20" Command="{Binding
ShowVideoStream}">Выбратькамеру</Button>

</StackPanel>
</StackPanel>

</Border>
<Border Grid.Column="1" Grid.Row="1"
BorderThickness="1" BorderBrush="Black">
<StackPanel Orientation="Horizontal">

```

```

<StackPanel Orientation="Vertical"
HorizontalAlignment="Left"
VerticalAlignment="Center">
<Button Margin="15" Padding="20" Command="{Binding
CheckCovsh}">Объектнераспознан</Button>
</StackPanel>
<StackPanel Orientation="Vertical"
HorizontalAlignment="Right"
VerticalAlignment="Center">
<Label Padding="10" Content="{Binding
CountStellCovsh}" />
<Label Padding="10" Content="{Binding
CountNumber}" />
<Label Padding="10" Content="{Binding CountEr}" />
</StackPanel>
</StackPanel>
</Border>
</Grid>
</Grid>
</mah:MetroWindow>

```

```

using Alturos.Yolo;
using Alturos.Yolo;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace StellcovshDetected.BusinessLayer
{
    publicstaticclassYoloV
    {
        publicstatic YoloWrapper yoloWrapper;
        ///<summary>
        /// Функция загрузки Yolo-модели.
        ///</summary>
        publicstaticvoid yolo_LoadModel()
        {
            string configFile = "Yolo\\stellcovshS.cfg";
            string weightsFile = "Yolo\\stellcovshS.weights";
            string namesFile = "Yolo\\obj2.names";
            YoloConfiguration yoloconf = new
            YoloConfiguration(configFile, weightsFile,
            namesFile);

```

```

        yoloWrapper = new
        YoloWrapper(yoloconf,false);
    }
    publicstatic YoloWrapper yoloWrapperNumber;
    ///<summary>
    /// Функция загрузки Yolo-модели.
    ///</summary>
    publicstaticvoid yoloNumber_LoadModel()
    {
        string configFile = "Yolo\\stellcovshNumber.cfg";
        string weightsFile =
        "Yolo\\stellcovshNumber.weights";
        string namesFile = "Yolo\\objNumber.names";
        YoloConfiguration yoloconf = new
        YoloConfiguration(configFile, weightsFile,
        namesFile);
        yoloWrapperNumber = new
        YoloWrapper(yoloconf, false);
    }
}

```

Рис. П2.2. Текст программы модуля YoloV.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace StellcovshDetected.BusinessLayer
{
    publicstaticclassCamera
    {

```

```

        publicstaticstring Point { get; set; }="Default";
        publicstaticstring Model { get; set; } = "FLIR-
        Default";
        publicstaticint Delay { get; set; } = 30;
        publicstaticint Width { get; set; } = 1280;
        publicstaticint Height { get; set; } = 720;
        publicstaticstring Address { get; set; }
    }
}

```

Рис. П2.3. Текст программы модуля Camera.cs

```

using Alturos.Yolo.Model;
using Microsoft.Win32;
using OpenCvSharp;
using StellcovshDetected.Functional;
using StellcovshDetected.BusinessLayer;
using StellcovshDetected.PresentationLayer;
using Prism.Commands;
using System;
using System.IO;
using System.Linq;
using System.Threading;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Media.Imaging;
using System.Collections.ObjectModel;
using Newtonsoft.Json;
using System.Windows.Shapes;
using System.Drawing;
using System.Collections.Generic;

namespace StellcovshDetected.BusinessLayer
{
    publicstaticclass Log
    {
        {
            privatestaticint count = 0;
            publicstaticvoid Record(StellCovsh rec)
            {
                StellCovsh.StellCovshList.Add(rec);
            }
            publicstaticvoid FormatToDB(StellCovsh rec) {
                DataLayer.Export.ExportInDB(rec);
            }
            publicstaticvoid FormatToLog(StellCovsh rec)
            {
                count++;
                string record = $"{count} | [{rec.Time}] номер:
                {rec.Number}участок: {rec.Point}камера:
                {rec.Camera}";
                DataLayer.Export.ExportFile(record);
            }
        }
    }
}

```

Рис. П2.4. Текст программы модуля Log.cs

```

using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace StellcovshDetected.BusinessLayer
{
    publicclass StellCovsh
    {
        publicstatic ObservableCollection<StellCovsh>
        StellCovshList=new
        ObservableCollection<StellCovsh>();
        publicstring Point { get; set; }
        publicstring Time { get; set; }
        publicstring Number { get; set; }
        publicstring Camera { get; set; }

        publicdouble X { get; set; }
        publicdouble Y { get; set; }

        publicdouble Width { get; set; }
        publicdouble Height { get; set; }
        publicStellCovsh(string location, DateTime
        timeDetection, string number,string camera,double
        x,double y,double width, double height)
        {
            this.Point = location;
            this.Time = $"{timeDetection.Hour}:
            {timeDetection.Minute}:{timeDetection.Second}";
            this.Number = number;
            this.Camera = camera;
            this.X = x;
            this.Y = y;
            this.Width = width;
            this.Height = height;
        }
    }
}

```

Рис. П2.5. Текст программы модуля StellCovsh.cs

```

using Alturos.Yolo.Model;
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace StellcovshDetected.BusinessLayer
{
    publicstaticclass StellCovshModel
    {
        { staticstring num="";
        staticstring numLast = "";
        publicstaticvoid
        Logging(ObservableCollection<YoloItem> recS,
        ObservableCollection<YoloItem> recN, bool txtLog =
        false, bool dbLog = false)
        {
            var log = new List<StellCovsh>();
            var time = new DateTime();
            StellCovsh tmp;
            time = DateTime.Now;
            bool flag = false;
            int corect = 50;
            foreach (var itemS in recS)
            {
                foreach (var item in
                StellCovsh.StellCovshList.Reverse().Take(24)){

```

```

if (item.X == itemS.X && item.Y == itemS.Y &&
item.Width == itemS.Width && item.Height ==
itemS.Height) flag = true;

if ((item.X >= (itemS.X - corect) && item.X <=
(itemS.X + corect)) && (item.Y >= (itemS.Y -
corect) && item.Y <= (itemS.Y + corect))
&& (item.Width >= (itemS.Width - corect) &&
item.Width <= (itemS.Width + corect)) &&
(item.Height >=
(itemS.Height - corect) && item.Height <=
(itemS.Height + corect))) flag = true;

}
if (!flag){
    StringBuilder number = new
    StringBuilder("");

    ObservableCollection<YoloItem> nums = new
    ObservableCollection<YoloItem>();

    foreach (var itemN in recN){
        if (itemS.X < itemN.Center().X && (itemS.X +
        itemS.Width) > itemN.Center().X)
            {

```

```

if (itemS.Y < itemN.Center().Y && (itemS.Y +
itemS.Height) > itemN.Center().Y && num !=
itemN.Type)
            {
                number.Insert(0,
                itemN.Type);
                num = itemN.Type;
            }
        }
    }
    if (numLast != number.ToString())
        {
            if (number.ToString() != ""){
                numLast = number.ToString();
                tmp = new
                StellCovsh(Camera.Point, time, number.ToString(),
                Camera.Model, itemS.X, itemS.Y, itemS.Width,
                itemS.Height);
            }
            else tmp = new StellCovsh(Camera.Point, time,
            "Отсутствует", Camera.Model, itemS.X, itemS.Y,
            itemS.Width, itemS.Height);

            Log.Record(tmp);flag = false;
            if (txtLog) Log.FormatToLog(tmp);
            if (dbLog) Log.FormatToDB(tmp);}}}}

```

Рис. П2.6. Текст программы модуля StellCovshModel.cs

```

using System;
using System.Collections.Generic;
using System.Data.SQLite;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace StellcovshDetected.DataLayer
{
    public static class ConnectToDB
    {
        private static String dbFileName =
        "StellcovshDetect.db";
        public static SQLiteConnection connect;

        public static void Connect()
        {
            if (!File.Exists(dbFileName))

```

```

                Console.WriteLine("Please, create
                DB and blank table (Push \"Create\" button)");
            try
            {
                connect = new
                SQLiteConnection("Data Source=" + dbFileName +
                ";MultipleActiveResultSets=true;Version=3;");
                connect.Open();
            }
            catch (SQLiteException ex)
            {
                Console.WriteLine("Error: " +
                ex.Message);
            }
        }
    }
}

```

Рис. П2.7. Текст программы модуля ConnectToDB.cs

```

using StellcovshDetected.BusinessLayer;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace StellcovshDetected.DataLayer
{
    public static class Export
    {
        private static string filename=
        $"{DateTime.Now.ToShortDateString()}__{DateTime.Now
        .Hour}_{DateTime.Now.Minute}";

```

```

        private static ExportLog log = new
        ExportLog(filename);

        public static void ExportFile(string rec)
        {
            log.CreateRecord(rec);
        }
        public static void ExportInDB(StellCovsh rec)
        {
            ExportDB.CreateRecord(rec);
        }
    }
}

```

Рис. П2.8. Текст программы модуля Export.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Data.SQLite;
using System.Data;
using StellcovshDetected.BusinessLayer;

namespace StellcovshDetected.DataLayer
{
    public static class ExportDB
    {
        private static SQLiteCommand command;
        public static void CreateRecord(StellCovsh rec)
        {
            if (ConnectToDB.connect.State !=
                ConnectionState.Open) return;
            command = new SQLiteCommand();
            command.Connection =
                ConnectToDB.connect;

            try
            {
                command = new SQLiteCommand();
                command.Connection =
                    ConnectToDB.connect;
                command.CommandText = $"INSERT
                INTO Camera (Model,Location,Address) VALUES
                ('{Camera.Model}','{Camera.Point}','{Camera.Address
                }')";
                command.ExecuteNonQuery();
            }

            catch (SQLiteException ex)
            {
                Console.WriteLine("Error: " +
                    ex.Message);
            }

            try
            {
                command.CommandText = $"INSERT
                INTO StellCovsh (Number) VALUES
                ('{rec.Number}')";
                command.ExecuteNonQuery();
            }

            catch (SQLiteException ex)
            {
                Console.WriteLine("Error: " +
                    ex.Message);
            }

            try
            {
                command.CommandText = $"INSERT
                INTO Region (Camera,StellCovsh,TimeDetect) VALUES
                ((SELECT Camera.ID FROM Camera WHERE Camera.Address
                = '{Camera.Address}'),(SELECT StellCovsh.ID FROM
                StellCovsh WHERE StellCovsh.Number =
                '{rec.Number}'),'{DateTime.Now.ToString()}')";
                command.ExecuteNonQuery();
            }

            catch (SQLiteException ex)
            {
                Console.WriteLine("Error: " +
                    ex.Message);
            }
        }
    }
}

```

Рис. П2.9. Текст программы модуля ExportDB.cs

```

namespace StellcovshDetected.Functional
{
    public interface IDialogService
    {
        void ShowMessage(string message); //
        // показ сообщения
    }

    string FilePath { get; set; } //
    // путь к выбранному файлу
    bool OpenFileDialog(); // открытие файла
    bool SaveFileDialog(); // сохранение файла
}

```

Рис. П2.10. Текст программы модуля IDialogService.cs

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace StellcovshDetected.DataLayer
{
    public class ExportLog
    {
        private string writePath;
        private StreamWriter sw;

        public ExportLog(string name)
        {
            writePath = $"@logs\\log_{name}.txt";
        }

        public void CreateRecord(string rec)
        {
            try
            {
                sw = new StreamWriter(writePath,
                    true, System.Text.Encoding.Default);
                sw.WriteLine(rec);
                sw.Close();
            }

            catch (Exception ex) { }
        }
    }
}

```

```

    }
}
}

```

Рис. П2.11. Текст программы модуля ExportLog.cs

```

using Microsoft.Win32;
using System.Windows;

namespace StellcovshDetected.Functional
{
    public class DefaultDialogService : IDialogService
    {
        public string FilePath { get; set; }

        public bool OpenFileDialog()
        {
            OpenFileDialog openFileDialog = new
            OpenFileDialog();
            if (openFileDialog.ShowDialog() == true)
            {
                FilePath =
                openFileDialog.FileName;
                return true;
            }
            return false;
        }

        public bool SaveFileDialog()
        {
            SaveFileDialog saveFileDialog = new
            SaveFileDialog();
            if (saveFileDialog.ShowDialog() == true)
            {
                FilePath =
                saveFileDialog.FileName;
                return true;
            }
            return false;
        }

        public void ShowMessage(string message)
        {
            MessageBox.Show(message);
        }
    }
}

```

Рис. П2.12. Текст программы модуля DefaultDialogService.cs

```

<Application x:Class="StellcovshDetected.App"                                </Application>

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

xmlns:controls="http://metro.mahapps.com/winfx/xaml/controls"
  xmlns:local="clr-namespace:StellcovshDetected"
  StartupUri="MainWindow.xaml">

<Application.Resources>
<ResourceDictionary>
<ResourceDictionary.MergedDictionaries>
<!-- MahApps.Metro resource dictionaries. Make
sure that all file names are Case Sensitive! -->
<ResourceDictionary
Source="pack://application:,,,/MahApps.Metro;component/Styles/Controls.xaml" />
<ResourceDictionary
Source="pack://application:,,,/MahApps.Metro;component/Styles/Fonts.xaml" />
<!-- Theme setting -->
<ResourceDictionary
Source="pack://application:,,,/MahApps.Metro;component/Styles/Themes/Light.Blue.xaml" />
</ResourceDictionary.MergedDictionaries>
</ResourceDictionary>
</Application.Resources>

```


Рис. П2.13. Текст программы модуля App.xaml

```

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Windows.Input;

namespace StellcovshDetected.Functional
{
    public class RelayCommand : ICommand
    {
        #region Fields

        private readonly Action<object> _execute;
        private readonly Predicate<object> _canExecute;
        private string _displayText;

        public static List<string> Log = new
        List<string>();
        private Action<object> action;

        #endregion // Fields

        #region Constructors

        ///<summary>
        /// Creates a new command that can always execute.
        ///</summary>
        ///<param name="execute">The execution
        logic.</param>
        public RelayCommand(Action<object> execute)
            : this(execute, null){}

        ///<summary>
        /// Creates a new command.
        ///</summary>
        ///<param name="execute">The execution
        logic.</param>
        ///<param name="canExecute">The execution status
        logic.</param>
        public RelayCommand(Action<object> execute,
        Predicate<object> canExecute)
            : this(execute, canExecute, "") {}

        public RelayCommand(Action<object> execute,
        Predicate<object> canExecute, string displayText)
        {
            if (execute == null)
                throw new ArgumentNullException("execute");

            _execute = execute;
            _canExecute = canExecute;
            _displayText = displayText;
        }

        public string DisplayText
        {
            get { return _displayText; }
            set { _displayText = value; }
        }

        #endregion // Constructors

        #region ICommand Members

        [DebuggerStepThrough]
        public bool CanExecute(object parameter)
        {
            return _canExecute == null ? true :
            _canExecute(parameter);
        }

        public event EventHandler CanExecuteChanged
        {
            add { CommandManager.RequerySuggested += value; }
            remove { CommandManager.RequerySuggested -= value; }
        }

        public void Execute(object parameter)
        {
            _execute(parameter);
        }

        #endregion // ICommand Members
    }
}

```

Рис. П2.14. Текст программы модуля RelayCommand.xaml

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;
using StellcovshDetected.BusinessLayer;
using StellcovshDetected.DataLayer;
using StellcovshDetected.PresentationLayer;
using StellcovshDetected.Functional;

using MahApps.Metro.Controls;
namespace StellcovshDetected
{
    ///<summary>
    /// Логика взаимодействия для MainWindow.xaml
    ///</summary>
    public partial class MainWindow : MetroWindow
    {
        public MainWindow()
        {
            DataContext = new DetectedAndLog (new
            DefaultDialogService());
            //YoloV.yoloNumber_LoadModel();
            YoloV.yolo_LoadModel();
            InitializeComponent();
        }
    }
}

```

Рис. П2.15. Текстпрограммы модуля MainWindow.xaml.cs

```

<mah:MetroWindow
  x:Class="StellcovshDetected.Settings"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mah="clr-namespace:MahApps.Metro.Controls;assembly=MahApps.Metro"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  xmlns:local="clr-namespace:StellcovshDetected"
  mc:Ignorable="d"
  Title="Настройка" Height="300" Width="270">
<Grid><StackPanel Orientation="Vertical">
<StackPanel Orientation="Horizontal"><TextBlock
Margin="10">Введите задержку между кадрами:</TextBlock>

<TextBox Margin="5" Text="{Binding DelayS}"/>
</StackPanel>
<StackPanel Orientation="Horizontal"
HorizontalAlignment="Center">
<TextBlock Margin="10">Разрешение видео</TextBlock>
<StackPanel>
<StackPanel Orientation="Horizontal">
<TextBlock Margin="10">Ширина:</TextBlock>
<TextBox Margin="5" Text="{Binding WidthS}"/>
</StackPanel>
<StackPanel Orientation="Horizontal">
<TextBlock Margin="10">Высота:</TextBlock>
<TextBox Margin="5" Text="{Binding HeightS}"/>
</StackPanel>
<Button Content="Сохранить" Margin="5"
Command="{Binding AcceptSettings}"
VerticalAlignment="Bottom"/>
<Button Content="Отмена" Margin="5"
Command="{Binding CancelSettings}"
VerticalAlignment="Bottom"/>
</StackPanel>
</Grid></mah:MetroWindow>

```

Рис. П2.16. Текстпрограммы модуля Settings.xaml

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;
using StellcovshDetected.BusinessLayer;
using StellcovshDetected.DataLayer;

using StellcovshDetected.PresentationLayer;
using StellcovshDetected.Functional;
using MahApps.Metro.Controls;
namespace StellcovshDetected
{
  ///<summary>
  ///Логика взаимодействия для Settings.xaml
  ///</summary>
  public partial class Settings : MetroWindow
  {
    public Settings()
    {
      DataContext = new SelectSettings(new
        DefaultDialogService());
      InitializeComponent();
    }
  }
}

```

Рис. П2.17. Текстпрограммы модуля Settings.xaml.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;
using StellcovshDetected.BusinessLayer;
using StellcovshDetected.DataLayer;

using StellcovshDetected.PresentationLayer;
using StellcovshDetected.Functional;
using MahApps.Metro.Controls;
namespace StellcovshDetected
{
  ///<summary>
  ///Логика взаимодействия для VideoSource.xaml
  ///</summary>
  public partial class VideoSource : MetroWindow
  {
    public VideoSource()
    {
      DataContext = new
        SelectVideoStream(new DefaultDialogService());
      InitializeComponent();
    }
  }
}

```

Рис. П2.18. Текст программы модуля VideoSource.xaml.cs

```

<mah:MetroWindow
  x:Class="StellcovshDetected.VideoSource"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mah="clr-namespace:MahApps.Metro.Controls;assembly=MahApps.Metro"

xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  xmlns:local="clr-namespace:StellcovshDetected"
  mc:Ignorable="d"
  Title="Видеопоток" Height="350" Width="300">
<Grid>
<StackPanel Orientation="Vertical">
<TextBlock Margin="15">Введите адрес IP -
камеры:</TextBlock>

<TextBox Margin="15" Text="{Binding
CameraAddress}"/>
<TextBlock Margin="15">
<TextBlock.Style>
<Style BasedOn="{StaticResource
MahApps.Styles.TextBlock}" TargetType="TextBlock">
<Style.Triggers>
<DataTrigger Binding="{Binding StatusConnect}"
Value="false">
<Setter Property="Text"
Value="Неудалось подключиться" />
</DataTrigger>
<DataTrigger Binding="{Binding StatusConnect}"
Value="true">
<Setter Property="Text" Value="Подключение успешно" />
</DataTrigger>
</Style.Triggers>
</Style>
</TextBlock.Style>
</TextBlock>
<Button Content="Применить" Margin="5"
Command="{Binding AcceptCamera}"
VerticalAlignment="Bottom"/>
</StackPanel>
</Grid>
</mah:MetroWindow>

```

Рис. П2.19. Текст программы модуля VideoSource.xaml

```

using Alturos.Yolo.Model;
using Microsoft.Win32;
using OpenCvSharp;
using StellcovshDetected.Functional;
using StellcovshDetected.BusinessLayer;
using Prism.Commands;
using System;
using System.IO;
using System.Linq;
using System.Threading;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Media.Imaging;
using System.Collections.ObjectModel;
using Newtonsoft.Json;
using System.Windows.Shapes;
using System.Drawing;
namespace StellcovshDetected.PresentationLayer
{
class SelectVideoStream : DetectedStellCovsh
{
    {
        private string ipAddress;
        private bool status;
        public DelegateCommand AcceptCamera { get;
        protected set; }
        IDialogService dialogService;
        public SelectVideoStream(IDialogService
        dialogService)
        {
            this.dialogService = dialogService;
            AcceptCamera = new
            DelegateCommand(SetCamera);
        }
        public string CameraAddress
        {
            get { return ipAddress; }
            set { ipAddress = value;
            OnPropertyChanged(nameof(CameraAddress)); }
        }
        private void SetCamera()
        {
            CameraAddress = ipAddress;
            CheckSourceAvailability();
        }
        public bool StatusConnect
        {
            get { return status; }
            set { status = value; OnPropertyChanged(); }
        }
        public void CheckSourceAvailability()
        {
            var stream = new VideoCapture(CameraAddress);
            var mat = new Mat();
            try
            {
                if (stream.Read(mat) && !mat.Empty())
                {
                    stream.Dispose();
                    mat.Dispose();
                    StatusConnect = true;
                    Camera.Address = CameraAddress;
                    stream.Dispose(); mat.Dispose();
                }
                StatusConnect = false;
            }
            catch { stream.Dispose(); mat.Dispose();
            StatusConnect = false; }
        }
    }
}

```

Рис. П2.20. Текст программы модуля SelectVideoStream.cs

```

using Alturos.Yolo.Model;
using Microsoft.Win32;
using OpenCvSharp;
using StellcovshDetected.Functional;
using StellcovshDetected.BusinessLayer;
using Prism.Commands;
using System;
using System.IO;
using System.Linq;
using System.Threading;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Media.Imaging;
using System.Collections.ObjectModel;
using Newtonsoft.Json;
using System.Windows.Shapes;
using System.Drawing;

namespace StellcovshDetected.PresentationLayer
{
    class SelectSettings : DetectedStellCovsh
    {
        private int delayS = Camera.Delay;
        private int heightS = Camera.Height;
        private int widthS = Camera.Width;
        public DelegateCommand AcceptSettings { get;
        protected set; }
        public DelegateCommand CancelSettings { get;
        protected set; }

        public int DelayS
        {
            get { return delayS; }
            set { delayS = value; OnPropertyChanged(); }
        }

        public int HeightS
        {
            get { return heightS; }
            set { heightS = value; OnPropertyChanged(); }
        }

        public int WidthS
        {
            get { return widthS; }
            set { widthS = value; OnPropertyChanged(); }
        }

        IDialogService dialogService;

        public SelectSettings(IDialogService dialogService)
        {
            this.dialogService = dialogService;
            AcceptSettings = new
            DelegateCommand(SaveSettings);
            CancelSettings = new
            DelegateCommand(ReturnSettings);
        }

        public void SaveSettings()
        {
            Camera.Delay = DelayS;
            Camera.Height = HeightS;
            Camera.Width = WidthS;
        }

        public void ReturnSettings()
        {
            DelayS = Camera.Delay;
            HeightS = Camera.Height;
            WidthS = Camera.Width;
        }
    }
}

```

Рис. П2.21. Текст программы модуля SelectSettings.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace StellcovshDetected.PresentationLayer
{
    class DetectedStellCovsh : INotifyPropertyChanged
    {
        public event PropertyChangedEventHandler
        PropertyChanged;

        public bool Set<T>(ref T field, T value,
        [CallerMemberName] string propertyName = null)
        {
            if (EqualityComparer<T>.Default.Equals(field,
            value))
                return false;

            field = value;
            OnPropertyChanged(propertyName);
            return true;
        }

        public void OnPropertyChanged([CallerMemberName]
        string prop = "")
        {
            PropertyChanged?.Invoke(this, new
            PropertyChangedEventArgs(prop));
        }
    }
}

```

Рис. П2.22. Текст программы модуля DetectedStellCovsh.cs

```

using Alturos.Yolo.Model;
using Microsoft.Win32;
using OpenCvSharp;
using StellcovshDetected.Functional;
using StellcovshDetected.BusinessLayer;
using Prism.Commands;
using System;
using System.IO;
using System.Linq;
using System.Threading;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Media.Imaging;
using System.Collections.ObjectModel;
using Newtonsoft.Json;
using System.Windows.Shapes;
using System.Drawing;
using System.Collections.Generic;
using System.Threading.Tasks;
namespace StellcovshDetected.PresentationLayer
{
    class DetectedAndLog : DetectedStellCovsh
    {
        private int countErr = 0;
        private bool isStarted;
        private bool disbDetect = false;
        private bool logTxt = false;
        private bool logDB = false;
        private Mat tmp = new Mat(); // обработка
        private ObservableCollection<YoloItem> yoloItems;
        private ObservableCollection<YoloItem>
        yoloItemsNumber;
        ObservableCollection<YoloItem> detect_item
        = new ObservableCollection<YoloItem>();
        ObservableCollection<YoloItem>
        detect_itemNumber = new
        ObservableCollection<YoloItem>();
        private ObservableCollection<StellCovsh>
        _StellCovshList = new
        ObservableCollection<StellCovsh>();
        private CancellationTokenSource tokenCancel;
        private string selectedPath;
        private string countStellCovsh =
        "Распознанныхковшей: 0";
        private string countNumber = "Распознанныхномеров:
        0";
        private string countEr = "Ошибка: 0";
        public DelegateCommand ShowVideoStream { get;
        protected set; }
        public DelegateCommand ShowSettings { get;
        protected set; }
        //Свойства
        public string SelectedPath{
        get { return selectedPath; }
        set { selectedPath = value;
        OnPropertyChanged(nameof(SelectedPath)); }}
        public string CountStellCovsh
        {get { return countStellCovsh; }
        set { countStellCovsh = value;
        OnPropertyChanged(nameof(CountStellCovsh)); }}
        public string CountNumber
        { get { return countNumber; }
        set { countNumber = value;
        OnPropertyChanged(nameof(CountNumber)); }}
        public string CountEr{
        get { return countEr; }
        set { countEr = value;
        OnPropertyChanged(nameof(CountEr)); }
        }
        public bool SystemStarted
        {
        get { return isStarted; }
        set { isStarted = value; OnPropertyChanged(); }
        }
        public bool DisbDetect
        {
        get { return disbDetect; }
        set { disbDetect = value; OnPropertyChanged(); }
        }
        public bool LogTxt
        {
        get { return logTxt; }
        set { logTxt = value; OnPropertyChanged(); }
        }
        public bool LogDB
        {
        get { return logDB; }
        set { logDB = value; OnPropertyChanged(); }
        }
        public ObservableCollection<StellCovsh>
        StellCovshList
        {
        get { return _StellCovshList; }
        set { _StellCovshList = value;
        OnPropertyChanged(); }
        }
        public DelegateCommand StartCommand { get;
        protected set; }
        public DelegateCommand CheckCovsh { get;
        protected set; }
        public DelegateCommand StopCommand { get;
        protected set; }

        public DelegateCommand OpenFileDialogCommand {
        get; protected set; }
        IDialogService dialogService;
        public DetectedAndLog(IDialogService dialogService)
        {
        this.dialogService = dialogService;
        OpenFileDialogCommand = new
        DelegateCommand(OpenFile);
        StartCommand = new
        DelegateCommand(VideoProcessing);
        StopCommand = new
        DelegateCommand(StopDetect);
        ShowVideoStream = new
        DelegateCommand(OpenVideoStream);
        ShowSettings = new
        DelegateCommand(OpenSettings);
        CheckCovsh = new
        DelegateCommand(AddCovsh);
        DataLayer.ConnectToDB.Connect();
        private void StopDetect()
        { tokenCancel.Cancel(); }
    }
}

```

Рис. П2.23. Текст программы модуля DetectedAndLog.cs

П2.23. Продолжение

```

#region Свойства, представляющие кадр
private BitmapImage inputVideo;
///<summary>
/// Свойство, представляющее входящий кадр.
///</summary>
public BitmapImage InputVideo
{
    get
    {
        return inputVideo;
    }
    set
    {
        inputVideo = value;

        OnPropertyChanged(nameof(InputVideo));
    }
}
#endregion
private void OpenFile()
{
    if (dialogService.OpenFileDialog() == true)
    {
        SelectedPath =
        dialogService.FilePath;
        Camera.Address =
        dialogService.FilePath; ;
        GetFirstFrame();
    }
}
private void GetFirstFrame()
{
    var video = VideoCapture.FromFile(SelectedPath);
    Mat matF = new OpenCvSharp.Mat();
    video.Read(matF);
    BitmapImage firstFrame =
    BitmapToImageSource(OpenCvSharp.Extensions.BitmapC
onverter.ToBitmap(matF));
    InputVideo = firstFrame;
    matF.Dispose();
    video.Dispose();
}

public async void VideoProcessing()
{
    tokenCancel = new
    CancellationTokensource();
    OpenFileDialog oFD = new
    OpenFileDialog();
    oFD.FileName = SelectedPath;
    SystemStarted = true;
    VideoCapture video = new
    VideoCapture(Camera.Address);

    System.Diagnostics.Debug.WriteLine("Количество кадр
ов видео " + video.FrameCount);
    try
    {
        while (!tokenCancel.IsCancellationRequested)
        //while считывает кадр и для него накладывается
        //установленная обработка
        { video.Read(tmp);
            ShowOnFormInput(tmp);
            int character = Cv2.WaitKey(Camera.Delay);

            CountStellCovsh = $"Распознанных ковшей:
            {StellCovshList.Count}";

```

```

        }
    }
    catch (Exception ex)
    {
        Console.WriteLine("Error: " +
        ex.Message);
    }
    SystemStarted = false;
    tokenCancel.Dispose();

    return;
}

public void ShowOnFormInput(Mat mat)
{
    if (mat != null)
    {
        mat = mat.Resize(new
        OpenCvSharp.Size(Camera.Width, Camera.Height));
        if (DisbDetect == false)
        {
            detect_item = Detect(mat);
            if (yoloItems != null)
            {
                var itemS = new ObservableCollection<YoloItem>();
                var itemN = new ObservableCollection<YoloItem>();

                foreach (var item in detect_item)
                {
                    var Width = item.Width;
                    var Height = item.Height;

                    var Center = item.Center();
                    Cv2.Rectangle(mat, new
                    OpenCvSharp.Point(item.X, item.Y),
                    new OpenCvSharp.Point(item.X + Width, item.Y +
                    Height), Scalar.LightGreen, 7);
                    if (item.Confidence > 0.55)
                    {
                        if (item.Type == "StellCovsh") {
                            if (item.Confidence > 0.7) itemS.Add(item);
                        }
                        else itemN.Add(item);
                    }
                }

                StellCovshModel.Logging(itemS, itemN, LogTxt,
                LogDB);
            }
        }
    }
}

```

П2.23. Продолжение

```

BitmapImage b =
BitmapToImageSource(OpenCvSharp.Extensions.BitmapC
onverter.ToBitmap(mat));
        InputVideo = b;
        StellCovshList =
StellCovsh.StellCovshList;
    }
    }
#region Детектирование проверка нахождения в зоне
privatebool isReadyYolo = false;
public ObservableCollection<YoloItem> Detect(Mat
mat_upac)
{
    Thread thread_auto = new Thread(() =>
    {
        if (!isReadyYolo)
        {
            isReadyYolo = true;
            yoloItems = new
ObservableCollection<YoloItem>(YoloV.yoloWrapper.D
etect(mat_upac.ToBytes()));
            isReadyYolo = false;
            Thread.Sleep(30);
        }
    });
    thread_auto.Start();
return yoloItems;
}

privatebool isReadyYoloNumber = false;
public ObservableCollection<YoloItem>
DetectNumber(Mat mat_upac)
{
    Thread thread_autoNumber = new
Thread(() =>
    {
        if (!isReadyYoloNumber)
        {
            isReadyYoloNumber = true;
            yoloItemsNumber = new
ObservableCollection<YoloItem>(YoloV.yoloWrapperNu
mber.Detect(mat_upac.ToBytes()));
            isReadyYoloNumber = false;
            Thread.Sleep(50);
        }
    });
    thread_autoNumber.Start();
return yoloItemsNumber;
}

#endregion
///<summary>
/// Это метод перевода пикселей графического
изображения в само изображение.
///</summary>
///<param name="bitmap">Пиксели графического
изображения</param>
///<returns>Изображение, оптимизированное для
загрузки изображений с помощью языка
XAML</returns>
public BitmapImage BitmapToImageSource(Bitmap
bitmap)// функция отображение видео на форме
{
using (MemoryStream memory = new MemoryStream())
{
    bitmap.Save(memory,
System.Drawing.Imaging.ImageFormat.Bmp);
    memory.Position = 0;
        BitmapImage bitmapimage = new
BitmapImage();
        bitmapimage.BeginInit();
        bitmapimage.StreamSource = memory;
        bitmapimage.CacheOption =
BitmapCacheOption.OnLoad;
        bitmapimage.EndInit();
return bitmapimage;
    }
}
privatevoid OpenVideoStream()
{
}

var window = new VideoSource();
window.Show();
var w = new Settings { DataContext = window };

}

privatevoid AddCovsh()
{
    CountEr = $"Ошибка: {++countErr}";
}

privatevoid OpenSettings()
{
}

var window = new Settings();
window.Show();

}
}

```

}

Спецификация

Настоящее приложение содержит спецификацию на разработанную программную документацию и программное обеспечение. Спецификация приведена в табл. ПЗ.1.

Таблица ПЗ.1

Спецификации на разработанную программную документацию

Обозначение	Назначение	Примечание
1	2	3
Документация		
Краев_ВКР.docx	Расчетно-пояснительная записка выпускной квалификационной работы	
Компоненты		
DetectedStellCovsh.cs	Класс, реализует интерфейс INotifyPropertyChanged позволяющий уведомлять систему об изменениях свойств модели. Позволяет связывать данные и интерфейс с логикой программы	
DetectedAndLog.cs	Класс, наследуемый от базовой модели и определяющий функционал, и логику окна формы детектирования и логирования	
SelectVideoStream.cs	Класс, наследуемый от базовой модели и определяющий функционал, и логику окна формы подключения к IP-камере и проверке соединения с ней	
SelectSettings.cs	Класс, наследуемый от базовой модели представления и определяющий функционал, и логику окна формы настроек	
App.xaml	Файл конфигурации визуальных ресурсов приложения	

Продолжение табл. ПЗ.1

1	2	3
ConnectToDB.cs	Класс с параметрами подключения к БД	
Export.cs	Класс логирования, осуществляет	

	форматирование и передачу данных лога разных типов (для БД и файла)	
ExportDB.cs	Класс определяющий формат записи в БД	
ExportLog.cs	Класс определяющий формат записи в лог	
MainWindow.xaml	Содержит структуру интерфейса оконной формы	
MainWindow.xaml.cs	Содержит класс обработчик пользовательских элементов управления оконной формы	
Settings.xaml	Содержит структуру интерфейса оконной формы	
Settings.xaml.cs	Содержит класс обработчик пользовательских элементов управления оконной формы	
VideoSource.xaml	Содержит структуру интерфейса оконной формы	
VideoSource.xaml.cs	Содержит класс обработчик пользовательских элементов управления оконной формы	
Log.cs	Класс содержит функционал логирования	
StellCovshModel.cs	Класс определяет алгоритм фильтрации детектируемых объектов и подготовку к логированию	
StellCovsh.cs	Класс определяет данные объекта предметной области стальковш	
Camera.cs	Класс определяет данные объекта предметной области камера	
YoloV.cs	Класс содержит параметры нейронной сети	

Продолжение табл. ПЗ.1

1	2	3
IDialogService.cs	Класс-интерфейс определяющий функционал диалогового окна	
DefaultDialogService.cs	Класс реализует функционал по работе с диалоговыми окнами	
RelayCommand.cs	Класс обеспечивает выполнение логики команды	
StellcovshDetect.db	Предназначена для хранения записей о фиксации стальковшей и их номеров	

Руководство пользователя

1. Описание программного обеспечения

Программный продукт предназначен для решения проблемы на предприятии АО «Северсталь БСС», связанная с контролем движения стальной по территории производства. Производится детектирование стальной и его номера с выбранного пользователем источника видеопотока, а так же запись результатов в БД или лог файл.

2. Минимальные системные требования

Компьютер, на котором будет установлена система должен обладать следующими минимальными требованиями:

- объем оперативной памяти 16 Гб;
- монитор с разрешением 1920x1080;
- операционная система Windows10 Домашняя;
- твердотельный накопитель объемом 1 Тб;
- процессор AMD Ryzen 5 2600 (4 ядра, 8 потоков, тактовая 3.4 ГГц);
- видеокарта Gigabyte GeForce GTX 1660Ti с поддержкой DirectX 12.

3. Запуск программы

Для запуска программы необходимо открыть папку с файлами программы и в ней найти файл “StellcovshDetected.exe” (рис. П4.1). После чего открыть его.

 StellcovshDetected.exe	15.04.2021 10:53	Приложение	1 007 КБ
 StellcovshDetected.exe.confia	15.04.2021 10:53	Исходный файл ...	19 КБ

Рис. П4.1. Файл запуска ПО

4. Выполнение программы. После запуска программы перед пользователем появляется основное окно программы (рис. П4.2).

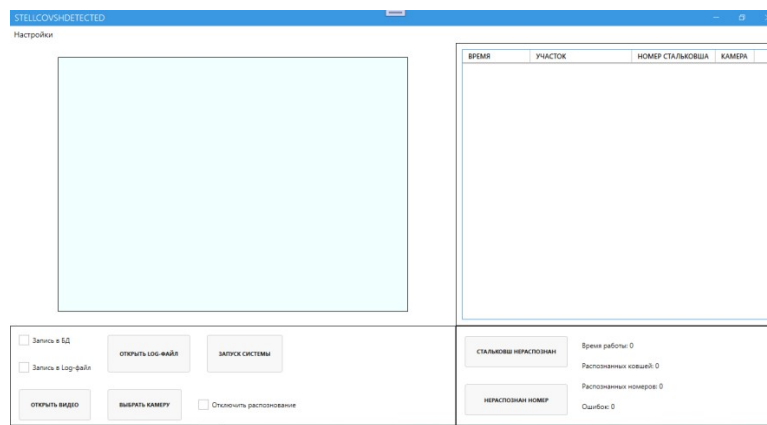


Рис. П4.2. Главное окно программы

Затем необходимо выбрать источник видеопотока нажав на кнопку «Открыть видео» или «Выбор камеры» (Рис. П4.3), после запустить систему (кнопка «Запуск системы»). Результат представлен на рис. П4.4.

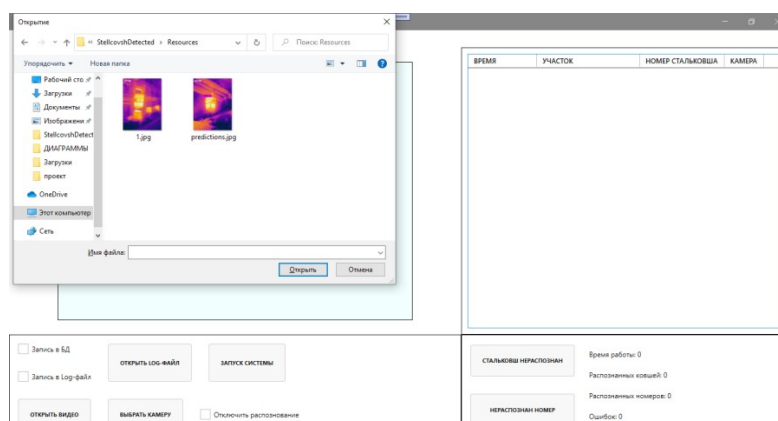


Рис. П4.3. Выбор файла

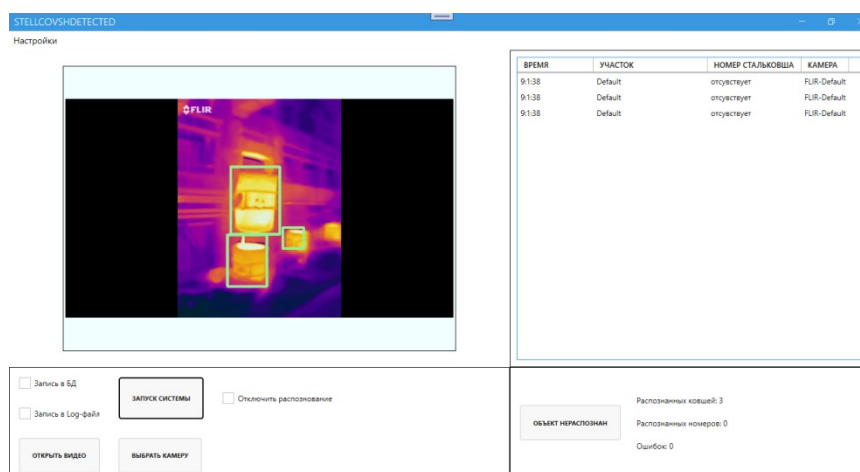


Рис. П4.4. Результат работы